

badpages The remainder of the page is used to store the indices of up to `MAX_SWAP_BADPAGES` number of bad page slots. These slots are filled in by the **mkswap** system program if the `-c` switch is specified to check the area.

`MAX_SWAP_BADPAGES` is a compile time constant that varies if the struct changes, but it is 637 entries in its current form as given by the simple equation.

$$\text{MAX_SWAP_BADPAGES} = \frac{\text{PAGE_SIZE} - 1,024 - 512 - 10}{\text{sizeof}(\text{long})}$$

Where 1,024 is the size of the bootblock, 512 is the size of the padding and 10 is the size of the magic string identifying the format of the swap file.

11.2 Mapping Page Table Entries to Swap Entries

When a page is swapped out, Linux uses the corresponding PTE to store enough information to locate the page on disk again. Obviously, a PTE is not large enough in itself to store precisely where on disk the page is located, but it is more than enough to store an index into the `swap_info` array and an offset within the `swap_map`. This is precisely what Linux does.

Each PTE, regardless of architecture, is large enough to store a `swp_entry_t`, which is declared as follows in `<linux/shmem_fs.h>`:

```
16 typedef struct {
17     unsigned long val;
18 } swp_entry_t;
```

Two macros are provided for the translation of PTEs to swap entries and vice versa. They are `pte_to_swp_entry()` and `swp_entry_to_pte()`, respectively.

Each architecture has to be able to determine if a PTE is present or swapped out. For illustration, I show how this is implemented on the x86. In the `swp_entry_t`, two bits are always kept free. On the x86, Bit 0 is reserved for the `_PAGE_PRESENT` flag, and Bit 7 is reserved for `_PAGE_PROTNONE`. The requirement for both bits is explained in Section 3.2. Bits 1 through 6 are for the *type*, which is the index within the `swap_info` array and are returned by the `SWP_TYPE()` macro.

Bits 8 through 31 are used to store the *offset* within the `swap_map` from the `swp_entry_t`. On the x86, this means 24 bits are available, which limits the size of the swap area to 64GiB. The macro `SWP_OFFSET()` is used to extract the offset.

To encode a type and offset into a `swp_entry_t`, the macro `SWP_ENTRY()` is available, which simply performs the relevant bit-shifting operations. The relationship between all these macros is illustrated in Figure 11.1.

The six bits for type should allow up to 64 swap areas to exist in a 32-bit architecture instead of the `MAX_SWAPFILES` restriction of 32. The restriction is due to the consumption of the `vmalloc` address space. If a swap area is the maximum possible size, 32MiB is required for the `swap_map` ($2^{24} * \text{sizeof}(\text{short})$); remember that each page uses one short for the reference count. For just `MAX_SWAPFILES`

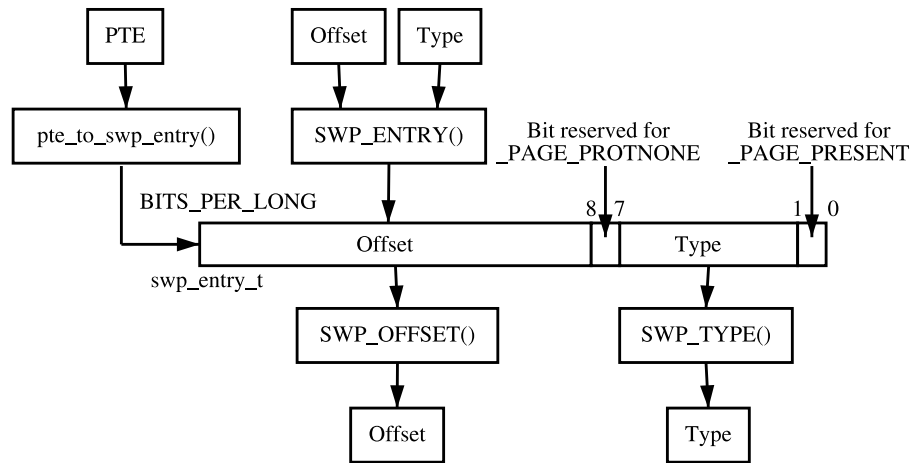


Figure 11.1. Storing Swap Entry Information in `swp_entry_t`

maximum number of swap areas to exist, 1GiB of virtual malloc space is required, which is simply impossible because of the user/kernel linear address space split.

This would imply that supporting 64 swap areas is not worth the additional complexity, but there are cases where a large number of swap areas would be desirable even if the overall swap available does not increase. Some modern machines² have many separate disks, which, between them, can create a large number of separate block devices. In this case, it is desirable to create a large number of small swap areas that are evenly distributed across all disks. This would allow a high degree of parallelism in the page swapping behavior, which is important for swap-intensive applications.

11.3 Allocating a Swap Slot

All page-sized slots are tracked by the array `swap_info_struct→swap_map`, which is of type `unsigned short`. Each entry is a reference count of the number of users of the slot, which happens in the case of a shared page and is 0 when free. If the entry is `SWAP_MAP_MAX`, the page is permanently reserved for that slot. It is unlikely, if not impossible, for this condition to occur, but it exists to ensure the reference count does not overflow. If the entry is `SWAP_MAP_BAD`, the slot is unusable.

The task of finding and allocating a swap entry is divided into two major tasks. The first is performed by the high-level function `get_swap_page()`, shown in Figure 11.2. Starting with `swap_list→next`, it searches swap areas for a suitable slot. After a slot has been found, it records what the next swap area to be used will be and returns the allocated entry.

²A Sun E450 could have in the region of 20 disks in it, for example.

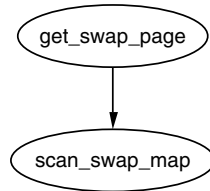


Figure 11.2. Call Graph: `get_swap_page()`

The task of searching the map is the responsibility of `scan_swap_map()`. In principle, it is very simple because it linearly scans the array for a free slot and return. Predictably, the implementation is a bit more thorough.

Linux attempts to organize pages into *clusters* on disk of size `SWAPFILE_CLUSTER`. It allocates `SWAPFILE_CLUSTER` number of pages sequentially in swap, keeps count of the number of sequentially allocated pages in `swap_info_struct→cluster_nr` and records the current offset in `swap_info_struct→cluster_next`. After a sequential block has been allocated, it searches for a block of free entries of size `SWAPFILE_CLUSTER`. If a block large enough can be found, it will be used as another cluster-sized sequence.

If no free clusters large enough can be found in the swap area, a simple first-free search that starts from `swap_info_struct→lowest_bit` is performed. The aim is to have pages swapped out at the same time close together on the premise that pages swapped out together are related. This premise, which seems strange at first glance, is quite solid when it is considered that the page replacement algorithm will use swap space most when linearly scanning the process address space swapping out pages. Without scanning for large free blocks and using them, it is likely that the scanning would degenerate to first-free searches and never improve. With it, processes exiting are likely to free up large blocks of slots.

11.4 Swap Cache

Pages that are shared between many processes cannot be easily swapped out because, as mentioned, there is no quick way to map a `struct page` to every PTE that references it. This leads to the rare condition where a page that is present for one PTE and swapped out for another gets updated without being synced to disk, thereby losing the update.

To address this problem, shared pages that have a reserved slot in backing storage are considered to be part of the *swap cache*. The swap cache has a small API associated with it and is shown in Table 11.1. The swap cache is purely conceptual because it is simply a specialization of the page cache. The first principal difference between pages in the swap cache rather than the page cache is that pages in the swap cache always use `swapper_space` as their `address_space` in `page→mapping`. The second difference is that pages are added to the swap cache with `add_to_swap_cache()`, shown in Figure 11.3, instead of `add_to_page_cache()`.

```
swp_entry_t get_swap_page()
```

This function allocates a slot in a `swap_map` by searching active swap areas. This is covered in greater detail in Section 11.3, but included here because it is principally used in conjunction with the swap cache.

```
int add_to_swap_cache(struct page *page, swp_entry_t entry)
```

This function adds a page to the swap cache. It first checks if it already exists by calling `swap_duplicate()`, and, if not, it adds it to the swap cache using the normal page cache interface function `add_to_page_cache_unique()`.

```
struct page * lookup_swap_cache(swp_entry_t entry)
```

This searches the swap cache and returns the `struct page` corresponding to the supplied `entry`. It works by searching the normal page cache based on `swapper_space` and the `swap_map` offset.

```
int swap_duplicate(swp_entry_t entry)
```

This function verifies a swap entry is valid and, if so, increments its swap map count.

```
void swap_free(swp_entry_t entry)
```

The complement function to `swap_duplicate()`. It decrements the relevant counter in the `swap_map`. When the count reaches zero, the slot is effectively free.

Table 11.1. Swap Cache API

Anonymous pages are not part of the swap cache *until* an attempt is made to swap them out. The variable `swapper_space` is declared as follows in `swap_state.c`:

```
39 struct address_space swapper_space = {
40     LIST_HEAD_INIT(swapper_space.clean_pages),
41     LIST_HEAD_INIT(swapper_space.dirty_pages),
42     LIST_HEAD_INIT(swapper_space.locked_pages),
43     0,
44     &swap_aops,
45 };
```

A page is identified as being part of the swap cache after the `page→mapping` field has been set to `swapper_space`, which is tested by the `PageSwapCache()` macro. Linux uses the exact same code for keeping pages between swap and memory in sync as it uses for keeping file-backed pages and memory in sync. They both share the page cache code, but the differences are just in the functions used.

The address space for backing storage, `swapper_space`, uses `swap_ops` for its `address_space→a_ops`. The `page→index` field is then used to store the `swp_entry_t` structure instead of a file offset, which is its normal purpose. The `address_space_operations` struct `swap_aops` is declared as follows in

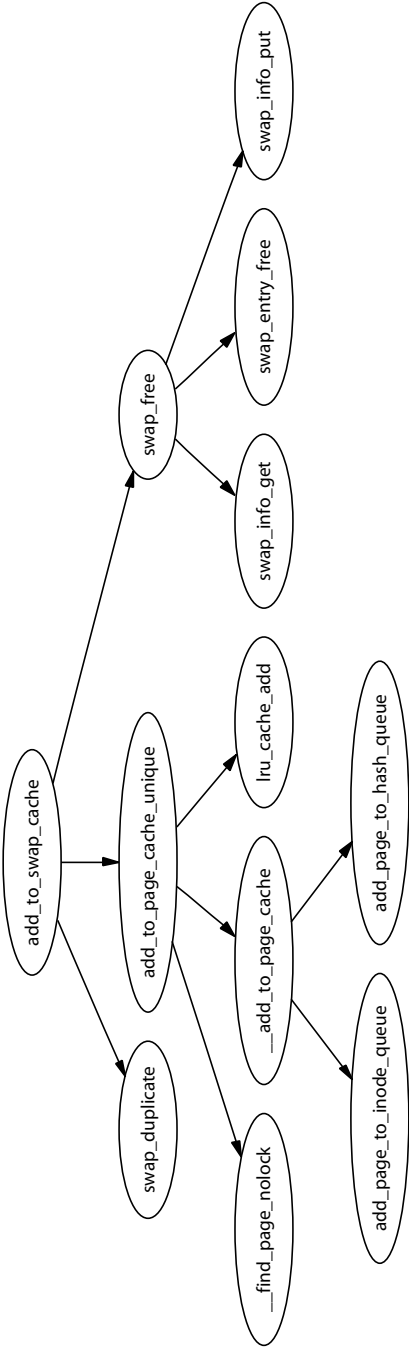


Figure 11.3. Call Graph: add_to_swap_cache()

swap_state.c:

```

34 static struct address_space_operations swap_aops = {
35     writepage: swap_writepage,
36     sync_page: block_sync_page,
37 };

```

When a page is being added to the swap cache, a slot is allocated with `get_swap_page()`, added to the page cache with `add_to_swap_cache()` and then marked dirty. When the page is next laundered, it will actually be written to backing storage on disk as the normal page cache would operate. This process is illustrated in Figure 11.4.

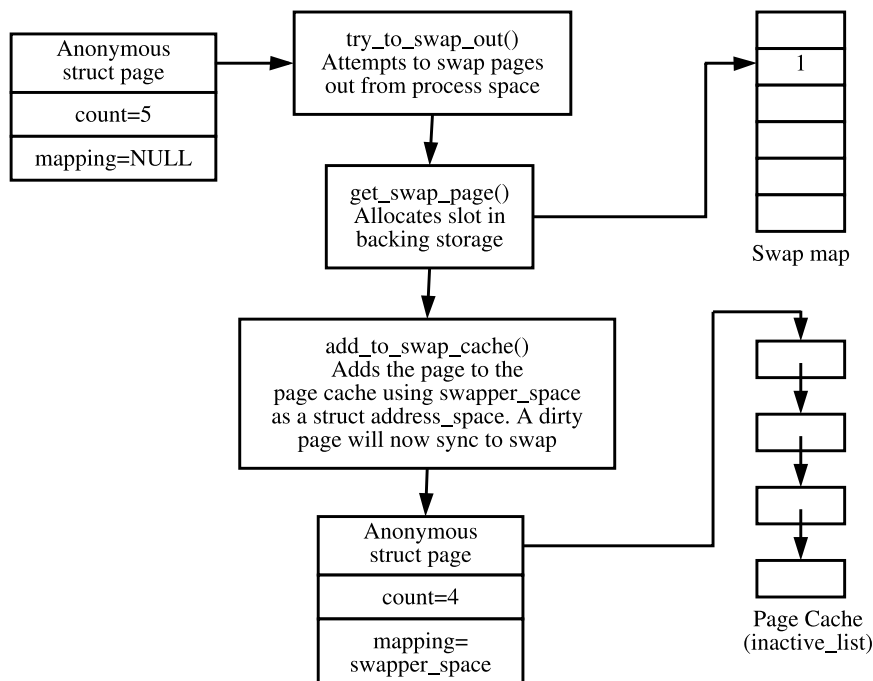


Figure 11.4. Adding a Page to the Swap Cache

Subsequent swapping of the page from shared PTEs results in a call to `swap_duplicate()`, which simply increments the reference to the slot in the `swap_map`. If the PTE is marked dirty by the hardware as a result of a write, the bit is cleared, and the `struct page` is marked dirty with `set_page_dirty()` so that the on-disk copy will be synced before the page is dropped. This ensures that, until all references to the page have been dropped, a check will be made to ensure the data on disk matches the data in the page frame.

When the reference count to the page finally reaches 0, the page is eligible to be dropped from the page cache, and the swap map count will have the count of the number of PTEs the on-disk slot belongs to so that the slot will not be freed

prematurely. It is laundered and finally dropped with the same LRU aging and logic described in Chapter 10.

If, on the other hand, a page fault occurs for a page that is swapped out, the logic in `do_swap_page()` will check to see if the page exists in the swap cache by calling `lookup_swap_cache()`. If it does, the PTE is updated to point to the page frame, the page reference count is incremented and the swap slot is decremented with `swap_free()`.

11.5 Reading Pages From Backing Storage

The principal function used when reading in pages is `read_swap_cache_async()`, which is mainly called during page faulting (see Figure 11.5). The function begins searching the swap cache with `find_get_page()`. Normally, swap cache searches are performed by `lookup_swap_cache()`, but that function updates statistics on the number of searches performed. Because the cache may need to be searched multiple times, `find_get_page()` is used instead.

The page can already exist in the swap cache if another process has the same page mapped or if multiple processes are faulting on the same page at the same time. If the page does not exist in the swap cache, one must be allocated and filled with data from backing storage.

After the page is allocated with `alloc_page()`, it is added to the swap cache with `add_to_swap_cache()` because swap cache operations may only be performed on pages in the swap cache. If the page cannot be added to the swap cache, the swap cache will be searched again to make sure another process has not put the data in the swap cache already.

To read information from backing storage, `rw_swap_page()` is called, which is discussed in Section 11.7. After the function completes, `page_cache_release()` is called to drop the reference to the page taken by `find_get_page()`.

11.6 Writing Pages to Backing Storage

When any page is being written to disk, the `address_space→a_ops` is consulted to find the appropriate write-out function. In the case of backing storage, the `address_space` is `swapper_space`, and the swap operations are contained in `swap_aops`. The struct `swap_aops` registers `swap_writepage()` because of its write-out function (see Figure 11.6).

The function `swap_writepage()` behaves differently depending on whether the writing process is the last user of the swap cache page or not. It knows this by calling `remove_exclusive_swap_page()`, which checks if there are any other processes using the page. This is a simple case of examining the page count with the `pagecache_lock` held. If no other process is mapping the page, it is removed from the swap cache and freed.

If `remove_exclusive_swap_page()` removed the page from the swap cache and freed it, `swap_writepage()` will unlock the page because it is no longer in use.

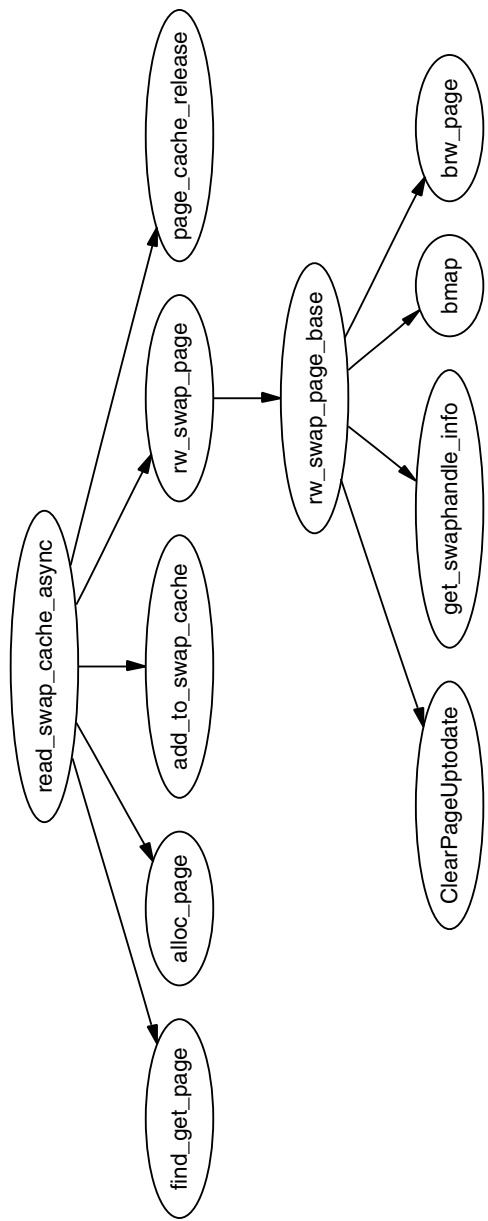


Figure 11.5. Call Graph: `read_swap_cache_async()`

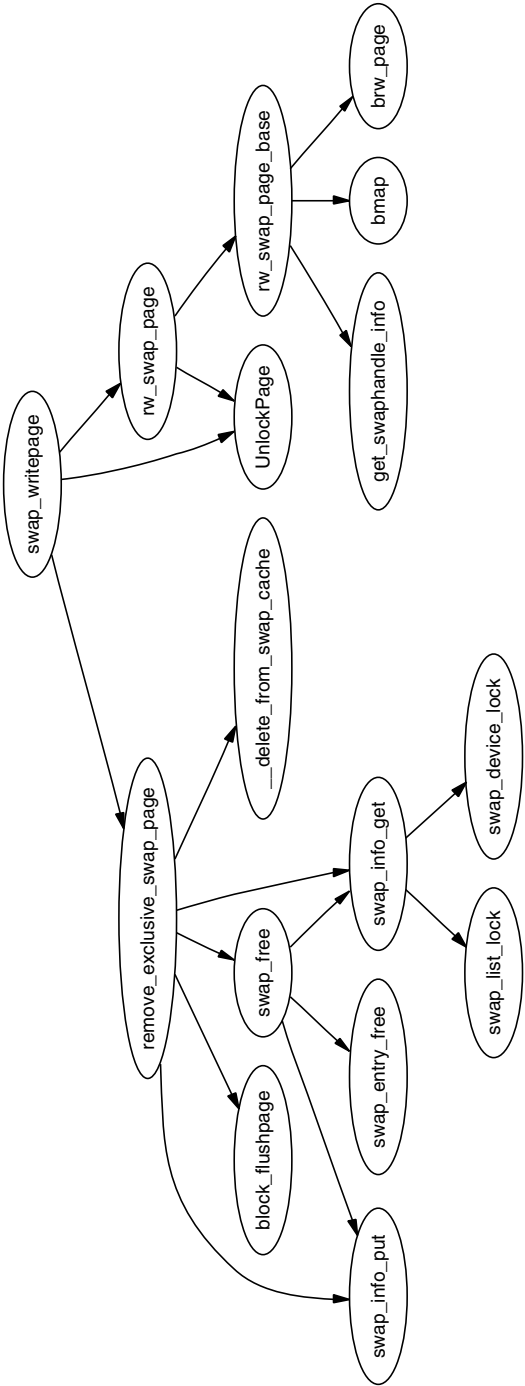


Figure 11.6. Call Graph: `swap_writepage()`

If it still exists in the swap cache, `rw_swap_page()` is called to write the data to the backing storage.

11.7 Reading/Writing Swap Area Blocks

The top-level function for reading and writing to the swap area is `rw_swap_page()`. This function ensures that all operations are performed through the swap cache to prevent lost updates. `rw_swap_page_base()` is the core function that performs the real work.

It begins by checking if the operation is a read. If it is, it clears the `uptodate` flag with `ClearPageUptodate()` because the page is obviously not up to date if I/O is required to fill it with data. This flag will be set again if the page is successfully read from disk. It then calls `get_swaphandle_info()` to acquire the device for the swap partition of the inode for the swap file. These are required by the block layer, which will be performing the actual I/O.

The core function can work with either swap partition or files because it uses the block layer function `brw_page()` to perform the actual disk I/O. If the swap area is a file, `bmap()` is used to fill a local array with a list of all blocks in the filesystem that contain the page data. Remember that filesystems may have their own method of storing files and disk, and it is not as simple as the swap partition where information may be written directly to disk. If the backing storage is a partition, only one page-sized block requires I/O, and, because no filesystem is involved, `bmap()` is unnecessary.

After it is known what blocks must be read or written, a normal block I/O operation takes place with `brw_page()`. All I/O that is performed is asynchronous, so the function returns quickly. After the I/O is complete, the block layer will unlock the page, and any waiting process will wake up.

11.8 Activating a Swap Area

Now that you know what swap areas are, how they are represented and how pages are tracked, it is time to see how they all tie together to activate an area. Activating an area is conceptually quite simple: Open the file, load the header information from disk, populate a `swap_info_struct` and add it to the swap list.

The function responsible for the activation of a swap area is `sys_swapon()`, and it takes two parameters, the path to the special file for the swap area and a set of flags. While swap is being activated, the *Big Kernel Lock (BKL)* is held, which prevents any application from entering kernel space while this operation is being performed. The function is quite large, but can be broken down into the following simple steps:

1. Find a free `swap_info_struct` in the `swap_info` array and initialize it with default values.
2. Call `user_path_walk()`, which traverses the directory tree for the supplied `specialfile` and populates a `namidata` structure with the available data on the file, such as the `dentry` and the filesystem information for where it is stored (`vfsmount`).

3. Populate `swap_info_struct` fields pertaining to the dimensions of the swap area and how to find it. If the swap area is a partition, the block size will be configured to the `PAGE_SIZE` before calculating the size. If it is a file, the information is obtained directly from the `inode`.
4. Ensure the area is not already activated. If not, allocate a page from memory and read the first page-sized slot from the swap area. This page contains information such as the number of good slots and how to populate the `swap_info_struct→swap_map` with the bad entries.
5. Allocate memory with `vmalloc()` for `swap_info_struct→swap_map` and initialize each entry with 0 for good slots and `SWAP_MAP_BAD` otherwise. Ideally, the header information will be a version 2 file format because version 1 was limited to swap areas of just under 128MiB for architectures with 4KiB page sizes like the x86.³
6. After ensuring the information indicated in the header matches the actual swap area, fill in the remaining information in the `swap_info_struct`, such as the maximum number of pages and the available good pages. Update the global statistics for `nr_swap_pages` and `total_swap_pages`.
7. The swap area is now fully active and initialized, so it is inserted into the swap list in the correct position based on priority of the newly activated area.

At the end of the function, the BKL is released, and the system now has a new swap area available for paging to.

11.9 Deactivating a Swap Area

In comparison to activating a swap area, deactivation is incredibly expensive. The principal problem is that the area cannot be simply removed. Every page that is swapped out must now be swapped back in again. Just as there is no quick way of mapping a `struct page` to every PTE that references it, there is no quick way to map a swap entry to a PTE either. This requires that all process page tables be traversed to find PTEs that reference the swap area to be deactivated and swap them in. This, of course, means that swap deactivation will fail if the physical memory is not available.

The function responsible for deactivating an area is, predictably enough, called `sys_swapoff()`. This function is mainly concerned with updating the `swap_info_struct`. The major task of paging in each paged-out page is the responsibility of `try_to_unuse()`, which is *extremely* expensive. For each slot used in the `swap_map`, the page tables for processes have to be traversed searching for it. In the worst case, all page tables belonging to all `mm_structs` may have to be traversed. Therefore, the tasks taken for deactivating an area are the following, broadly speaking:

1. Call `user_path_walk()` to acquire the information about the special file to be deactivated and then take the BKL.

³See the Code Commentary for the comprehensive reason for this.

2. Remove the `swap_info_struct` from the swap list and update the global statistics on the number of swap pages available (`nr_swap_pages`) and the total number of swap entries (`total_swap_pages`). After this is acquired, the BKL can be released again.
3. Call `try_to_unuse()`, which will page in all pages from the swap area to be deactivated. This function loops through the swap map using `find_next_to_unuse()` to locate the next used swap slot. For each used slot it finds, it performs the following:
 - Call `read_swap_cache_async()` to allocate a page for the slot saved on disk. Ideally, it exists in the swap cache already, but the page allocator will be called if it is not.
 - Wait on the page to be fully paged in and lock it. Once locked, call `unuse_process()` for every process that has a PTE referencing the page. This function traverses the page table searching for the relevant PTE and then updates it to point to the `struct page`. If the page is a shared memory page with no remaining reference, `shmem_unuse()` is called instead.
 - Free all slots that were permanently mapped. It is believed that slots will never become permanently reserved, so the risk is taken.
 - Delete the page from the swap cache to prevent `try_to_swap_out()` from referencing a page in the event it still somehow has a reference in swap from map.
4. If there was not enough available memory to page in all the entries, the swap area is reinserted back into the running system because it cannot be simply dropped. If it succeeded, the `swap_info_struct` is placed into an uninitialized state, and the `swap_map` memory is freed with `vfree()`

11.10 What's New in 2.6

The most important addition to the `struct swap_info_struct` is the addition of a linked list called `extent_list` and a cache field called `curr_swap_extent` for the implementation of extents.

Extents, which are represented by a `struct swap_extent`, map a contiguous range of pages in the swap area into a contiguous range of disk blocks. These extents are set up at swapon time by the function `setup_swap_extents()`. For block devices, there will only be one swap extent, and it will not improve performance, but the extent it set up so that swap areas backed by block devices or regular files can be treated the same.

It can make a large difference with swap files, which will have multiple extents representing ranges of pages clustered together in blocks. When searching for the page at a particular offset, the extent list will be traversed. To improve search times, the last extent that was searched will be cached in `swap_extent→curr_swap_extent`.

Shared Memory Virtual Filesystem

Sharing a region of memory backed by a file or device is simply a case of calling `mmap()` with the `MAP_SHARED` flag. However, there are two important cases where an anonymous region needs to be shared between processes. The first is when `mmap()` with `MAP_SHARED` is used without file backing. These regions will be shared between a parent and child process after a `fork()` is executed. The second is when a region is explicitly setting them up with `shmget()` and is attached to the virtual address space with `shmat()`.

When pages within a VMA are backed by a file on disk, the interface used is straightforward. To read a page during a page fault, the required `nopage()` function is found in `vm_area_struct→vm_ops`. To write a page to backing storage, the appropriate `writepage()` function is found in the `address_space_operations` using `inode→i_mapping→a_ops` or alternatively using `page→mapping→a_ops`. When normal file operations are taking place, such as `mmap()`, `read()` and `write()`, the `struct file_operations` with the appropriate functions is found using `inode→i_fop` and so on. These relationships were illustrated in Figure 4.2.

This is a very clean interface that is conceptually easy to understand, but it does not help anonymous pages because there is no file backing. To keep this nice interface, Linux creates an artificial file backing for anonymous pages using a RAM-based filesystem where each VMA is backed by a file in this filesystem. Every inode in the filesystem is placed on a linked list called `shmem_inodes` so that it may always be easily located. This allows the same file-based interface to be used without treating anonymous pages as a special case.

The filesystem comes in two variations called *shm* and *tmpfs*. They both share core functionality and mainly differ in what they are used for. *shm* is for use by the kernel for creating file backings for anonymous pages and for backing regions created by `shmget()`. This filesystem is mounted by `kern_mount()` so that it is mounted internally and not visible to users. *tmpfs* is a temporary filesystem that may be optionally mounted on `/tmp/` to have a fast RAM-based temporary filesystem. A secondary use for *tmpfs* is to mount it on `/dev/shm/`. Processes that `mmap()` files in the *tmpfs* filesystem will be able to share information between them as an alternative to System V Inter-Process Communication (IPC) mechanisms. Regardless of the type of use, *tmpfs* must be explicitly mounted by the system administrator.

This chapter begins with a description of how the virtual filesystem is implemented. From there, I discuss how shared regions are set up and destroyed before talking about how the tools are used to implement System V IPC mechanisms.

12.1 Initializing the Virtual Filesystem

The virtual filesystem is initialized by the function `init_tmpfs()`, shown in Figure 12.1, either during system start or when the module is being loaded. This function registers the two filesystems, `tmpfs` and `shm`, and mounts `shm` as an internal filesystem with `kern_mount()`. It then calculates the maximum number of blocks and inodes that can exist in the filesystems. As part of the registration, the function `shmem_read_super()` is used as a callback to populate a `struct super_block` with more information about the filesystems, such as making the block size equal to the page size.

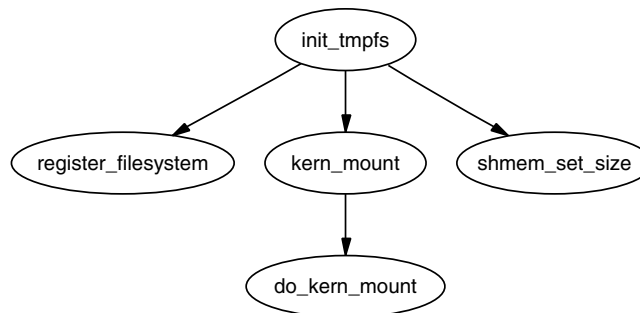


Figure 12.1. Call Graph: `init_tmpfs()`

Every inode created in the filesystem will have a `struct shmem_inode_info` associated with it, which contains private information specific to the filesystem. The function `SHMEM_I()` takes an inode as a parameter and returns a pointer to a struct of this type. It is declared as follows in `<linux/shmem_fs.h>`:

```

20 struct shmem_inode_info {
21     spinlock_t          lock;
22     unsigned long       next_index;
23     swp_entry_t         i_direct[SHMEM_NR_DIRECT];
24     void                **i_indirect;
25     unsigned long       swapped;
26     unsigned long       flags;
27     struct list_head    list;
28     struct inode        *inode;
29 };
  
```

The fields are the following:

lock is a spinlock protecting the inode information from concurrent accesses.

next_index is an index of the last page being used in the file. This will be different from `inode→i_size` while a file is being truncated.

i_direct is a direct block containing the first `SHMEM_NR_DIRECT` swap vectors in use by the file. See Section 12.4.1.

i_indirect is a pointer to the first indirect block. See Section 12.4.1.

swapped is a count of the number of pages belonging to the file that are currently swapped out.

flags is currently only used to remember if the file belongs to a shared region set up by `shmget()`. It is set by specifying `SHM_LOCK` with `shmctl()` and unlocked by specifying `SHM_UNLOCK`.

list is a list of all inodes used by the filesystem.

inode is a pointer to the parent inode.

12.2 Using shmem Functions

Different structs contain pointers for shmem specific functions. In all cases, `tmpfs` and `shm` share the same structs.

For faulting in pages and writing them to backing storage, two structs called `shmem_aops` and `shmem_vm_ops` of type `struct address_space_operations` and `struct vm_operations_struct`, respectively, are declared.

The address space operations `struct shmem_aops` contains pointers to a small number of functions of which the most important one is `shmem_writepage()`, which is called when a page is moved from the page cache to the swap cache. `shmem_removepage()` is called when a page is removed from the page cache so that the block can be reclaimed. `shmem_readpage()` is not used by `tmpfs`, but is provided so that the `sendfile()` system call may be used with `tmpfs` files. `shmem_prepare_write()` and `shmem_commit_write()` are also unused, but are provided so that `tmpfs` can be used with the loopback device. `shmem_aops` is declared as follows in `mm/shmem.c`:

```
1500 static struct address_space_operations shmem_aops = {
1501     removepage:    shmem_removepage,
1502     writepage:     shmem_writepage,
1503 #ifdef CONFIG_TMPFS
1504     readpage:      shmem_readpage,
1505     prepare_write:  shmem_prepare_write,
1506     commit_write:   shmem_commit_write,
1507 #endif
1508 };
```

Anonymous VMAs use `shmem_vm_ops` as the `vm_operations_struct` so that `shmem_nopage()` is called when a new page is being faulted in. It is declared as follows:

```
1426 static struct vm_operations_struct shmem_vm_ops = {
1427     nopage: shmem_nopage,
1428 };
```

To perform operations on files and inodes, two structs, `file_operations` and `inode_operations`, are required. The `file_operations`, called `shmem_file_operations`, provides functions that implement `mmap()`, `read()`, `write()` and `fsync()`. It is declared as follows:

```
1510 static struct file_operations shmem_file_operations = {
1511     mmap:          shmem_mmap,
1512 #ifdef CONFIG_TMPFS
1513     read:          shmem_file_read,
1514     write:         shmem_file_write,
1515     fsync:         shmem_sync_file,
1516 #endif
1517 };
```

Three sets of `inode_operations`, are provided. The first is `shmem_inode_operations`, which is used for file inodes. The second, called `shmem_dir_inode_operations`, is for directories. The last pair, called `shmem_symlink_inline_operations` and `shmem_symlink_inode_operations`, is for use with symbolic links.

The two file operations supported are `truncate()` and `setattr()`, which are stored in a `struct inode_operations` called `shmem_inode_operations`. `shmem_truncate()` is used to truncate a file. `shmem_notify_change()` is called when the file attributes change. This allows, among other things, for a file to be grown with `truncate()` and to use the global zero page as the data page. `shmem_inode_operations` is declared as follows:

```
1519 static struct inode_operations shmem_inode_operations = {
1520     truncate:      shmem_truncate,
1521     setattr:       shmem_notify_change,
1522 };
```

The directory `inode_operations` provides functions such as `create()`, `link()` and `mkdir()`. They are declared as follows:

```
1524 static struct inode_operations shmem_dir_inode_operations = {
1525 #ifdef CONFIG_TMPFS
1526     create:        shmem_create,
1527     lookup:        shmem_lookup,
1528     link:          shmem_link,
```



```

1529     unlink:      shmem_unlink,
1530     symlink:     shmem_symlink,
1531     mkdir:       shmem_mkdir,
1532     rmdir:       shmem_rmdir,
1533     mknod:       shmem_mknod,
1534     rename:      shmem_rename,
1535 #endif
1536 };

```

The last pair of operations are for use with symlinks. They are declared as follows:

```

1354 static struct inode_operations shmem_symlink_inline_operations = {
1355     readlink:     shmem_readlink_inline,
1356     follow_link:  shmem_follow_link_inline,
1357 };
1358
1359 static struct inode_operations shmem_symlink_inode_operations = {
1360     truncate:     shmem_truncate,
1361     readlink:     shmem_readlink,
1362     follow_link:  shmem_follow_link,
1363 };

```

The difference between the two `readlink()` and `follow_link()` functions is related to where the link information is stored. A symlink inode does not require the private inode information `struct shmem_inode_information`. If the length of the symbolic link name is smaller than this struct, the space in the inode is used to store the name, and `shmem_symlink_inline_operations` becomes the inode operations struct. Otherwise, a page is allocated with `shmem_getpage()`, the symbolic link is copied to it and `shmem_symlink_inode_operations` is used. The second struct includes a `truncate()` function so that the page will be reclaimed when the file is deleted.

These various structs ensure that the shmem equivalent of inode-related operations will be used when regions are backed by virtual files. When they are used, the majority of the VM sees no difference between pages backed by a real file and ones backed by virtual files.

12.3 Creating Files in tmpfs

Because `tmpfs` is mounted as a proper filesystem that is visible to the user, it must support directory inode operations such as `open()`, `mkdir()` and `link()`. Pointers to functions that implement these for `tmpfs` are provided in `shmem_dir_inode_operations`, which is shown in Section 12.2.

The implementations of most of these functions are quite small, and, at some level, they are all interconnected as can be seen from Figure 12.2. All of them share the same basic principle of performing some work with inodes in the virtual filesystem, and the majority of the inode fields are filled in by `shmem_get_inode()`.

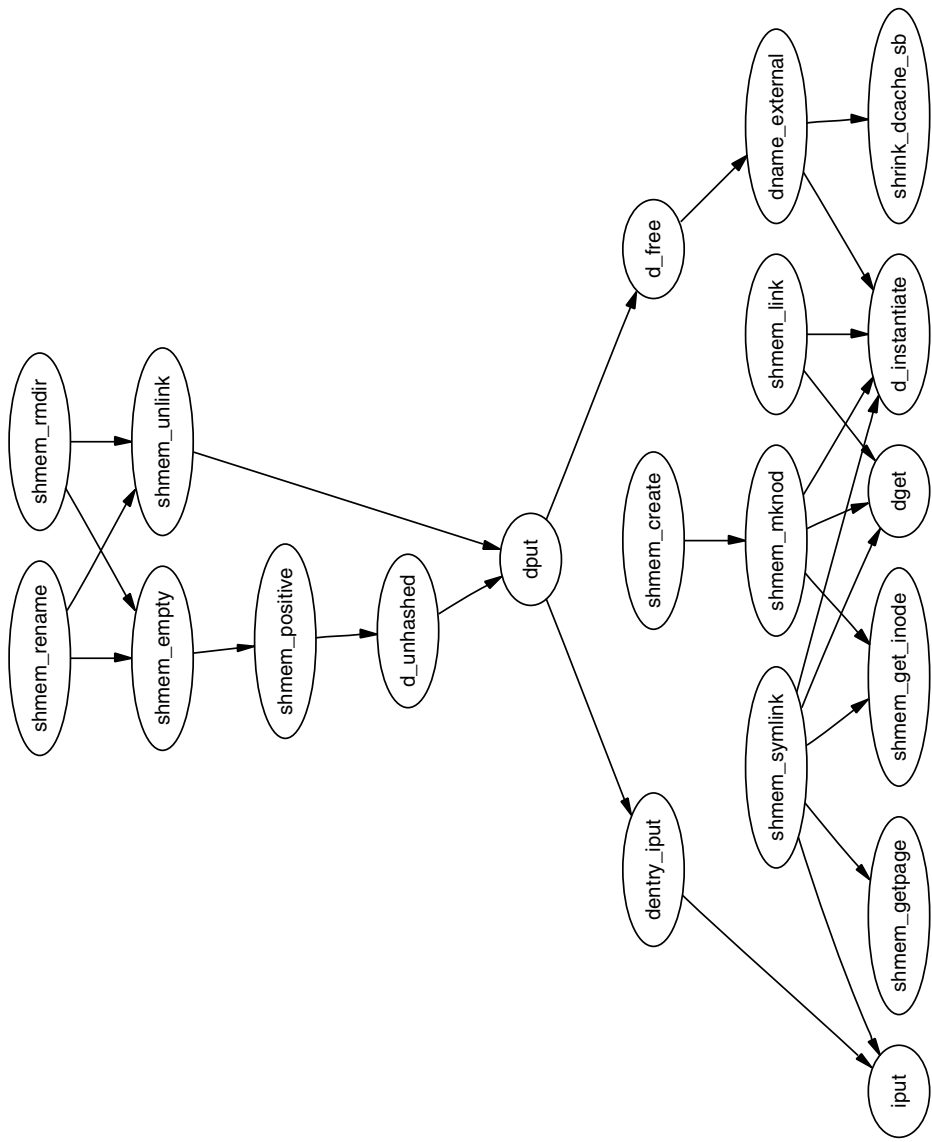


Figure 12.2. Call Graph: shm_create()

When creating a new file, the top-level function called is `shmem_create()`. This small function calls `shmem_mknod()` with the `S_IFREG` flag added so that a regular file will be created. `shmem_mknod()` is little more than a wrapper around the `shmem_get_inode()`, which, predictably, creates a new inode and fills in the struct fields. The three fields of principal interest that are filled are the `inode→i_mapping→a_ops`, `inode→i_op` and `inode→i_fop` fields. After the inode has been created, `shmem_mknod()` updates the directory inode `size` and `mtime` statistics before instantiating the new inode.

Files are created differently in `shm` even though the filesystems are essentially identical in functionality. How these files are created is covered later in Section 12.7.

12.4 Page Faulting Within a Virtual File

When a page fault occurs, `do_no_page()` will call `vma→vm_ops→nopage` if it exists. In the case of the virtual filesystem, this means the function `shmem_nopage()`, with its call graph shown in Figure 12.3, will be called when a page fault occurs.

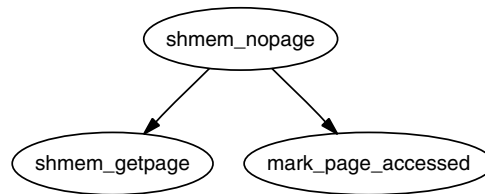


Figure 12.3. Call Graph: `shmem_nopage()`

The core function in this case is `shmem_getpage()`, which is responsible for either allocating a new page or finding it in swap. This overloading of fault types is unusual because `do_swap_page()` is normally responsible for locating pages that have been moved to the swap cache or backing storage using information encoded within the PTE. In this case, pages backed by virtual files have their PTE set to 0 when they are moved to the swap cache. The inode's private filesystem data stores direct and indirect block information, which is used to locate the pages later. This operation is very similar in many respects to normal page faulting.

12.4.1 Locating Swapped Pages

When a page has been swapped out, a `swp_entry_t` will contain information needed to locate the page again. Instead of using the PTEs for this task, the information is stored within the filesystem-specific private information in the inode.

When faulting, the function called to locate the swap entry is `shmem_alloc_entry()`. Its basic task is to perform basic checks and ensure that `shmem_inode_info→next_index` always points to the page index at the end of the virtual file. Its principal task is to call `shmem_swp_entry()`, which searches for the swap vector within the inode information with `shmem_swp_entry()`, and to allocate new pages as necessary to store swap vectors.

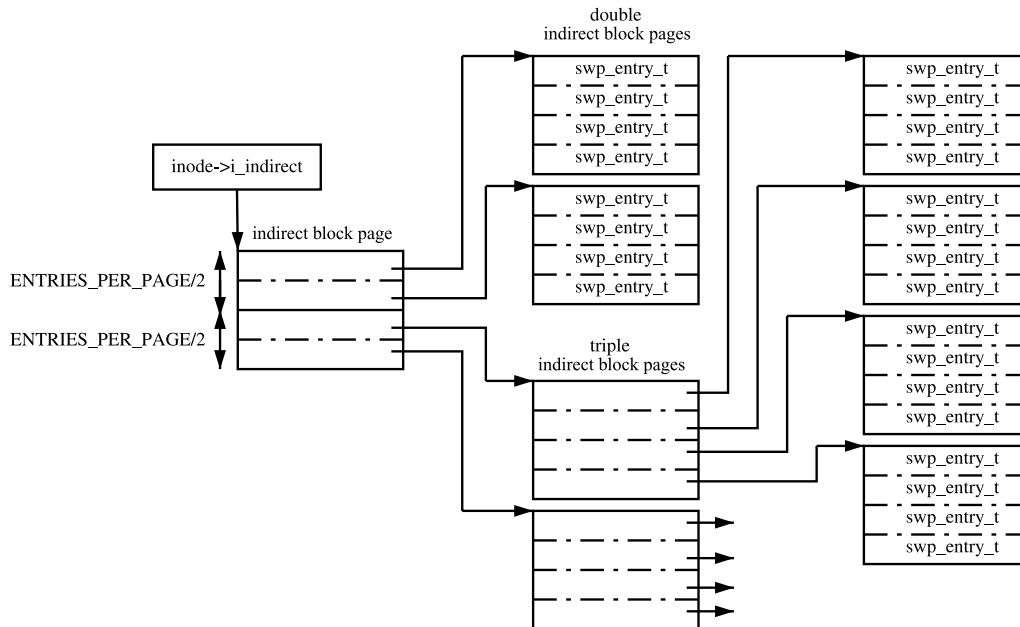


Figure 12.4. Traversing Indirect Blocks in a Virtual File

The first `SHMEM_NR_DIRECT` entries are stored in `inode->i_direct`. This means that, for the x86, files that are smaller than 64KiB (`SHMEM_NR_DIRECT * PAGE_SIZE`) will not need to use indirect blocks. Larger files must use indirect blocks starting with the one located at `inode->i_indirect`.

The initial indirect block (`inode->i_indirect`) is broken into two halves. The first half contains pointers to doubly indirect blocks, and the second half contains pointers to triply indirect blocks. The doubly indirect blocks are pages containing swap vectors (`swp_entry_t`). The triply indirect blocks contain pointers to pages, which in turn are filled with swap vectors. The relationship between the different levels of indirect blocks is illustrated in Figure 12.4. The relationship means that the maximum number of pages in a virtual file (`SHMEM_MAX_INDEX`) is defined as follows in `mm/shmem.c`:

```
44 #define SHMEM_MAX_INDEX (
    SHMEM_NR_DIRECT +
    (ENTRIES_PER_PAGE/2) *
    (ENTRIES_PER_PAGE+1))
```

12.4.2 Writing Pages to Swap

The function `shmem_writepage()` is the registered function in the filesystem's `address_space_operations` for writing pages to swap. The function is responsible for simply moving the page from the page cache to the swap cache. This is

implemented with a few simple steps:

1. Record the current `page→mapping` and information about the inode.
2. Allocate a free slot in the backing storage with `get_swap_page()`.
3. Allocate a `swp_entry_t` with `shmem_swp_entry()`.
4. Remove the page from the page cache.
5. Add the page to the swap cache. If it fails, free the swap slot, add back to the page cache and try again.

12.5 File Operations in tmpfs

Four operations, `mmap()`, `read()`, `write()` and `fsync()`, are supported with virtual files. Pointers to the functions are stored in `shmem_file_operations`, which was shown in Section 12.2.

Little is unusual in the implementation of these operations, and they are covered in detail in the Code Commentary. The `mmap()` operation is implemented by `shmem_mmap()`, and it simply updates the VMA that is managing the mapped region. `read()`, implemented by `shmem_read()`, performs the operation of copying bytes from the virtual file to a userspace buffer, faulting in pages as necessary. `write()`, implemented by `shmem_write()`, is essentially the same. The `fsync()` operation is implemented by `shmem_file_sync()`, but is essentially a NULL operation because it performs no task and simply returns 0 for success. Because the files only exist in RAM, they do not need to be synchronized with any disk.

12.6 Inode Operations in tmpfs

The most complex operation that is supported for inodes is truncation and involves four distinct stages. The first, in `shmem_truncate()`, will truncate a partial page at the end of the file and continually calls `shmem_truncate_indirect()` until the file is truncated to the proper size. Each call to `shmem_truncate_indirect()` will only process one indirect block at each pass, which is why it may need to be called multiple times.

The second stage, in `shmem_truncate_indirect()`, understands both doubly and triply indirect blocks. It finds the next indirect block that needs to be truncated. This indirect block, which is passed to the third stage, will contain pointers to pages, which in turn contain swap vectors.

The third stage in `shmem_truncate_direct()` works with pages that contain swap vectors. It selects a range that needs to be truncated and passes the range to the last stage `shmem_swp_free()`. The last stage frees entries with `free_swap_and_cache()`, which frees both the swap entry and the page containing data.

The linking and unlinking of files is very simple because most of the work is performed by the filesystem layer. To link a file, the directory inode size is incremented,

the `ctime` and `mtime` of the affected inodes is updated and the number of links to the inode being linked to is incremented. A reference to the new dentry is then taken with `dget()` before instantiating the new dentry with `d_instantiate()`. Unlinking updates the same inode statistics before decrementing the reference to the dentry with `dput()`. `dput()` will also call `iput()`, which will clear up the inode when its reference count hits zero.

Creating a directory will use `shmem_mkdir()` to perform the task. It simply uses `shmem_mknod()` with the `S_IFDIR` flag before incrementing the parent directory inode's `i_nlink` counter. The function `shmem_rmdir()` will delete a directory by first ensuring it is empty with `shmem_empty()`. If it is, the function then decrements the parent directory inode's `i_nlink` count and calls `shmem_unlink()` to remove the requested directory.

12.7 Setting Up Shared Regions

A shared region is backed by a file created in `shm`. There are two cases where a new file will be created: during the setup of a shared region with `shmget()` and when an anonymous region is set up with `mmap()` with the `MAP_SHARED` flag. Both functions use the core function `shmem_file_setup()` to create a file.

Because the filesystem is internal, the names of the files created do not have to be unique because the files are always located by inode, not name. Therefore, `shmem_zero_setup()` (see Figure 12.5) always says to create a file called `dev/zero`, which is how it shows up in the file `/proc/pid/maps`. Files created by `shmget()` are called `SYSVNN` where the `NN` is the key that is passed as a parameter to `shmget()`.

The core function `shmem_file_setup()` simply creates a new dentry and inode, fills in the relevant fields and instantiates them.

12.8 System V IPC

The full internals of the IPC implementation are beyond the scope of this book. This section will focus just on the implementations of `shmget()` and `shmat()` and how they are affected by the VM. The system call `shmget()` is implemented by `sys_shmget()`, shown in Figure 12.6. It performs basic checks to the parameters and sets up the IPC-related data structures. To create the segment, it calls `newseg()`. This is the function that creates the file in `shmfs` with `shmem_file_setup()` as discussed in the previous section.

The system call `shmat()` is implemented by `sys_shmat()`. There is little remarkable about the function. It acquires the appropriate descriptor and makes sure all the parameters are valid before calling `do_mmap()` to map the shared region into the process address space. Only two points of note are in the function.

The first is that it is responsible for ensuring that VMAs will not overlap if the caller specifies the address. The second is that the `shp→shm_nattch` counter is maintained by a `vm_operations_struct()` called `shm_vm_ops`. It registers `open()` and `close()` callbacks called `shm_open()` and `shm_close()`, respectively. The

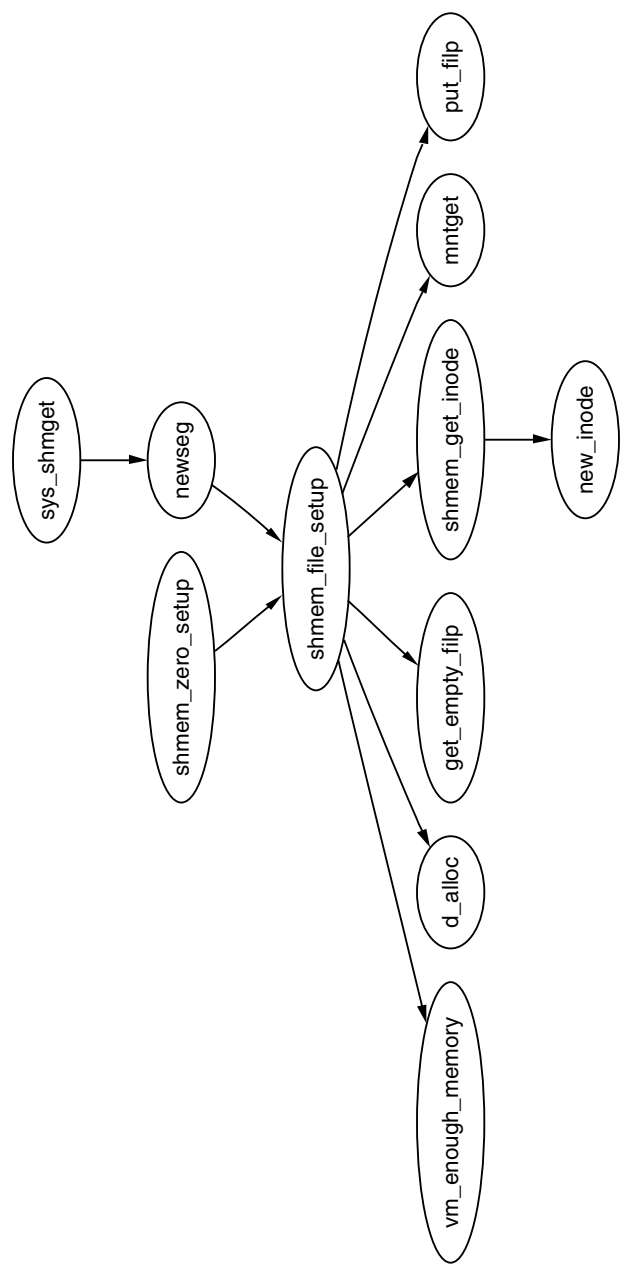


Figure 12.5. Call Graph: shm_mem_zero_setup()

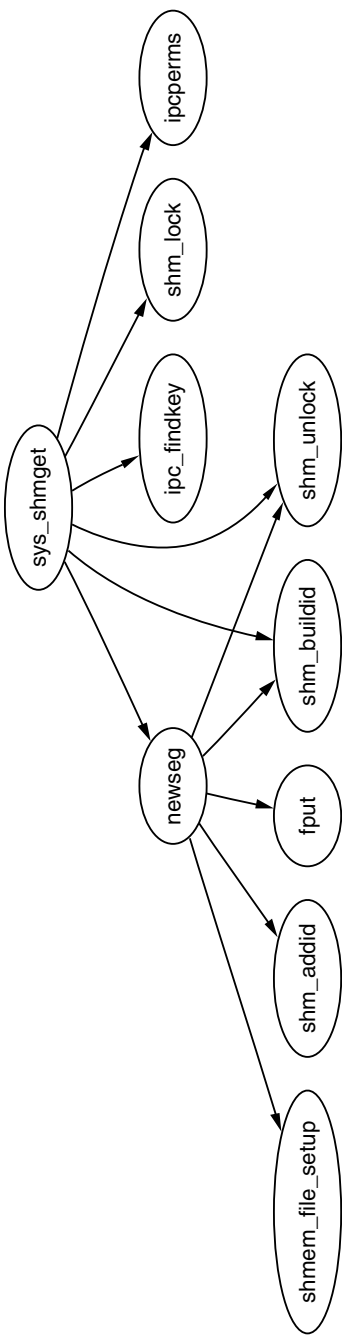


Figure 12.6. Call Graph: `sys_shmget()`

`shm_close()` callback is also responsible for destroyed shared regions if the `SHM_DEST` flag is specified and the `shm_nattch` counter reaches zero.

12.9 What's New in 2.6

The core concept and functionality of the filesystem remains the same, and the changes are either optimizations or extensions to the filesystem's functionality. If the reader understands the 2.4 implementation well, the 2.6 implementation will not present much trouble.¹

A new field has been added to the `shmem_inode_info` called `allocated`. The `allocated` field stores how many data pages are allocated to the file, which had to be calculated on the fly in 2.4 based on `inode→i_blocks`. It both saves a few clock cycles on a common operation as well as makes the code a bit more readable.

The `flags` field now uses the `VM_ACCOUNT` flag as well as the `VM_LOCKED` flag. The `VM_ACCOUNT`, always set, means that the VM will carefully account for the amount of memory used to make sure that allocations will not fail.

Extensions to the file operations are the ability to seek with the system call `_llseek()`, implemented by `generic_file_llseek()`, and to use `sendfile()` with virtual files, implemented by `shmem_file_sendfile()`. An extension has been added to the VMA operations to allow nonlinear mappings, implemented by `shmem_populate()`.

The last major change is that the filesystem is responsible for the allocation and destruction of its own inodes, which are two new callbacks in `struct super_operations`. It is simply implemented by the creation of a slab cache called `shmem_inode_cache`. A constructor function `init_once()` is registered for the slab allocator to use for initializing each new inode.

¹I find that saying "How hard could it possibly be" always helps.

Out of Memory Management

The last aspect of the VM I am going to discuss is the Out Of Memory (OOM) manager. This intentionally is a very short chapter because it has one simple task: check if there is enough available memory to satisfy, verify that the system is truly out of memory and, if so, select a process to kill. This is a controversial part of the VM and it has been suggested that it be removed on many occasions. Regardless of whether it exists in the latest kernel, it still is a useful system to examine because it touches off a number of other subsystems.

13.1 Checking Available Memory

For certain operations, such as expanding the heap with `brk()` or remapping an address space with `mremap()`, the system will check if there is enough available memory to satisfy a request. Note that this is separate to the `out_of_memory()` path that is covered in the next section. This path is used to avoid the system being in a state of OOM if at all possible.

When checking available memory, the number of required pages is passed as a parameter to `vm_enough_memory()`. Unless the system administrator has specified that the system should overcommit memory, the amount of available memory will be checked. To determine how many pages are potentially available, Linux sums up the following bits of data:

Total page cache because page cache is easily reclaimed.

Total free pages because they are already available.

Total free swap pages because userspace pages may be paged out.

Total pages managed by swapper_space However, this double-counts the free swap pages. This is balanced by the fact that slots are sometimes reserved, but not used.

Total pages used by the dentry cache because they are easily reclaimed.

Total pages used by the inode cache because they are easily reclaimed.

If the total number of pages added here is sufficient for the request, `vm_enough_memory()` returns true to the caller. If false is returned, the caller knows that the memory is not available and usually decides to return `-ENOMEM` to userspace.

13.2 Determining OOM Status

When the machine is low on memory, old page frames will be reclaimed (see Chapter 10), but, despite reclaiming pages, it may find that it was unable to free enough pages to satisfy a request even when scanning at highest priority. If it does fail to free page frames, `out_of_memory()` is called to see if the system is out of memory and needs to kill a process. The function's call graph is shown in Figure 13.1.

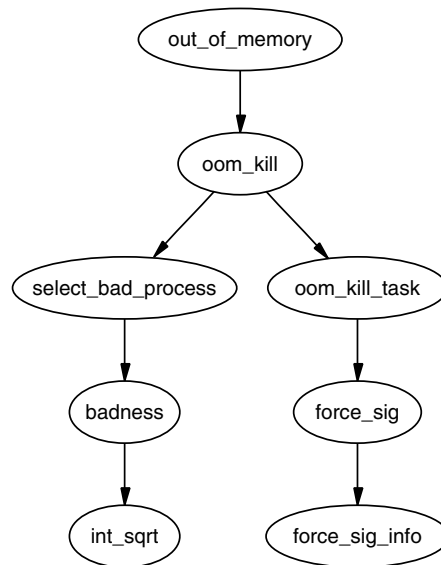


Figure 13.1. Call Graph: `out_of_memory()`

Unfortunately, it is possible that the system is not out of memory and simply needs to wait for I/O to complete or for pages to be swapped to backing storage. This is unfortunate, not because the system has memory, but because the function is being called unnecessarily, which opens the possibility of processes being unnecessarily killed. Before deciding to kill a process, it goes through the following checklist.

- Is there enough swap space left (`nr_swap_pages > 0`)? If yes, it is not OOM.
- Has it been more than 5 seconds since the last failure? If yes, it is not OOM.
- Have we failed within the last second? If no, it is not OOM.
- If there have not been 10 failures at least in the last 5 seconds, it is not OOM.
- Has a process been killed within the last 5 seconds? If yes, it is not OOM.

It is only if the previous tests are passed that `oom_kill()` is called to select a process to kill.

13.3 Selecting a Process

The function `select_bad_process()` is responsible for choosing a process to kill. It decides by stepping through each running task and calculating how suitable it is for killing with the function `badness()`. The badness is calculated as follows. The square roots are integer approximations calculated with `int_sqrt()`:

$$\text{badness_for_task} = \frac{\text{total_vm_for_task}}{\sqrt{\text{cpu_time_in_seconds}} * \sqrt[4]{\text{cpu_time_in_minutes}}}$$

This has been chosen to select a process that is using a large amount of memory, but is not that long lived. Processes that have been running a long time are unlikely to be the cause of memory shortage, so this calculation is likely to select a process that uses a lot of memory, but has not been running long. If the process is a root process or has `CAP_SYS_ADMIN` capabilities, the points are divided by four because it is assumed that root privilege processes are well behaved. Similarly, if it has `CAP_SYS_RAWIO` capabilities (access to raw devices) privileges, the points are further divided by four because it is undesirable to kill a process that has direct access to hardware.

13.4 Killing the Selected Process

After a task is selected, the list is walked again, and each process that shares the same `mm_struct` as the selected process (i.e., they are threads) is sent a signal. If the process has `CAP_SYS_RAWIO` capabilities, a `SIGTERM` is sent to give the process a chance of exiting cleanly. Otherwise, a `SIGKILL` is sent.

13.5 Is That It?

Yes, that is it. OOM management touches a lot of subsystems, but, otherwise, there is not much to it.

13.6 What's New in 2.6

The majority of OOM management remains essentially the same for 2.6 except for the introduction of VM-accounted objects. These are VMAs that are flagged with the `VM_ACCOUNT` flag, first mentioned in Section 4.8. Additional checks will be made to ensure there is memory available when performing operations on VMAs with this flag set. The principal incentive for this complexity is to avoid the need of an OOM killer.

Some regions that always have the `VM_ACCOUNT` flag set are the process stack, the process heap, regions `mmap()`ed with `MAP_SHARED`, private regions that are writable and regions that set up `shmget()`. In other words, most userspace mappings have the `VM_ACCOUNT` flag set.

Linux accounts for the amount of memory that is committed to these VMAs with `vm_acct_memory()`, which increments a variable called `committed_space`. When the

VMA is freed, the committed space is decremented with `vm_unacct_memory()`. This is a fairly simple mechanism, but it allows Linux to remember how much memory it has already committed to userspace when deciding if it should commit more.

The checks are performed by calling `security_vm_enough_memory()`, which introduces another new feature. A feature is available in 2.6 that allows security-related kernel modules to override certain kernel functions. The full list of hooks available is stored in a `struct security_operations` called `security_ops`. There are a number of dummy, or default, functions that may be used, which are all listed in `security/dummy.c`, but the majority do nothing except return. If no security modules are loaded, the `security_operations` struct used is called `dummy_security_ops`, which uses all the default functions.

By default, `security_vm_enough_memory()` calls `dummy_vm_enough_memory()`, which is declared in `security/dummy.c` and is very similar to 2.4's `vm_enough_memory()` function. The new version adds the following pieces of information together to determine available memory:

Total page cache because page cache is easily reclaimed.

Total free pages because they are already available.

Total free swap pages because userspace pages may be paged out.

Slab pages with SLAB_RECLAIM_ACCOUNT set because they are easily reclaimed.

These pages, minus a 3 percent reserve for root processes, is the total amount of memory that is available for the request. If the memory is available, it makes a check to ensure the total amount of committed memory does not exceed the allowed threshold. The allowed threshold is `TotalRam * (OverCommitRatio/100) + TotalSwapPage`, where `OverCommitRatio` is set by the system administrator. If the total amount of committed space is not too high, 1 will be returned so that the allocation can proceed.

The Final Word

Make no mistake, memory management is a large, complex and time-consuming field to research and difficult to apply to practical implementations. Because it is very difficult to model how systems behave in real multiprogrammed systems [CD80], developers often rely on intuition to guide them, and examination of virtual memory algorithms depends on simulations of specific workloads. Simulations are necessary because modeling how scheduling, paging behavior and multiple processes interact presents a considerable challenge. Page replacement policies, a field that has been the focus of considerable amounts of research, is a good example because it is only ever shown to work well for specified workloads. The problem of adjusting algorithms and policies to different workloads is addressed by having administrators tune systems as much as by research and algorithms.

The Linux kernel is also large, complex and fully understood by a relatively small core group of people. Its development is the result of contributions of thousands of programmers with a varying range of specialties, backgrounds and spare time. The first implementations are developed based on the all-important foundation that theory provides. Contributors built upon this framework with changes based on real-world observations.

It has been asserted on the Linux Memory Management mailing list that the VM is poorly documented and difficult to pick up because “the implementation is a nightmare to follow”¹ and the lack of documentation on practical VMs is not just confined to Linux. Matt Dillon, one of the principal developers of the FreeBSD VM² and considered a “VM guru” stated in an interview³ that documentation can be “hard to come by.” One of the principal difficulties with deciphering the implementation is the fact that the developer must have a background in memory management theory to see why implementation decisions were made because a pure understanding of the code is insufficient for any purpose other than microoptimizations.

This book attempted to bridge the gap between memory management theory and the practical implementation in Linux and to tie both fields together in a single place. It tried to describe what life is like in Linux as a memory manager in a

¹<http://mail.nl.linux.org/linux-mm/2002-05/msg00035.html>

²His past involvement with the Linux VM is evident from <http://mail.nl.linux.org/linux-mm/2000-05/msg00419.html>.

³<http://kerneltrap.com/node.php?id=8>

manner that was relatively independent of hardware architecture considerations. I hope after reading this and progressing onto the code commentary that you, the reader, feels a lot more comfortable with tackling the VM subsystem. As a final parting shot, Figure 14.1 broadly illustrates how the subsystems I discussed in detail interact with each other.

On a final personal note, I hope that this book encourages other people to produce similar works for other areas of the kernel. I know I'll buy them!

Introduction

Welcome to the code commentary section of the book. If you are reading this, you are looking for a heavily detailed tour of the code. The commentary presumes you have read the equivalent section in the main part of the book, so, if you just started reading here, you're probably in the wrong place.

Each appendix section corresponds to the order and structure of the book. The order in which the functions are presented is the same order as displayed in the call graphs that are referenced throughout the commentary. At the beginning of each appendix and subsection, there is a mini table of contents to help navigate your way through the commentary. The code coverage is not 100 percent, but all the principal code patterns that are found throughout the VM are here. If the function you are interested in is not commented on, find a function similar to it.

Some of the code has been reformatted slightly for presentation, but the actual code is not changed. It is recommended that you use the companion CD while reading the code commentary. In particular use LXR to browse through the source code so that you get a feel for reading the code with and without the aid of the commentary.

Good Luck!

Describing Physical Memory

Contents

B.1 Initializing Zones	220
B.1.1 Function: <code>setup_memory()</code>	220
B.1.2 Function: <code>zone_sizes_init()</code>	223
B.1.3 Function: <code>free_area_init()</code>	224
B.1.4 Function: <code>free_area_init_node()</code>	224
B.1.5 Function: <code>free_area_init_core()</code>	226
B.1.6 Function: <code>build_zonelists()</code>	232
B.2 Page Operations	234
B.2.1 Locking Pages	234
B.2.1.1 Function: <code>lock_page()</code>	234
B.2.1.2 Function: <code>__lock_page()</code>	234
B.2.1.3 Function: <code>sync_page()</code>	235
B.2.2 Unlocking Pages	236
B.2.2.1 Function: <code>unlock_page()</code>	236
B.2.3 Waiting on Pages	236
B.2.3.1 Function: <code>wait_on_page()</code>	236
B.2.3.2 Function: <code>___wait_on_page()</code>	237

B.1 Initializing Zones

Contents

B.1 Initializing Zones	220
B.1.1 Function: <code>setup_memory()</code>	220
B.1.2 Function: <code>zone_sizes_init()</code>	223
B.1.3 Function: <code>free_area_init()</code>	224
B.1.4 Function: <code>free_area_init_node()</code>	224
B.1.5 Function: <code>free_area_init_core()</code>	226
B.1.6 Function: <code>build_zonelists()</code>	232

B.1.1 Function: `setup_memory()` (*arch/i386/kernel/setup.c*)

The call graph for this function is shown in Figure 2.3. This function gets the necessary information to give to the boot memory allocator to initialize itself. It is broken up into a number of different tasks.

- Find the start and ending PFN for low memory (`min_low_pfn`, `max_low_pfn`), the start and end PFN for high memory (`highstart_pfn`, `highend_pfn`) and the PFN for the last page in the system (`max_pfn`).
- Initialize the `bootmem_data` structure and declare which pages may be used by the boot memory allocator.
- Mark all pages usable by the system as free, and then reserve the pages used by the bitmap representing the pages.
- Reserve pages used by the SMP config or the `initrd` image if one exists.

```

991 static unsigned long __init setup_memory(void)
992 {
993     unsigned long bootmap_size, start_pfn, max_low_pfn;
994
995     /*
996      * partially used pages are not usable - thus
997      * we are rounding upwards:
998      */
999     start_pfn = PFN_UP(__pa(&end));
1000
1001     find_max_pfn();
1002
1003     max_low_pfn = find_max_low_pfn();
1004
1005 #ifdef CONFIG_HIGHMEM
1006     highstart_pfn = highend_pfn = max_pfn;
1007     if (max_pfn > max_low_pfn) {
1008         highstart_pfn = max_low_pfn;
1009     }

```

```

1010     printk(KERN_NOTICE "%ldMB HIGHMEM available.\n",
1011             pages_to_mb(highend_pfn - highstart_pfn));
1012 #endif
1013     printk(KERN_NOTICE "%ldMB LOWMEM available.\n",
1014             pages_to_mb(max_low_pfn));

```

999 `PFN_UP()` takes a physical address, rounds it up to the next page and returns the page frame number. `_end` is the address of the end of the loaded kernel image, so `start_pfn` is now the offset of the first physical page frame that may be used.

1001 `find_max_pfn()` loops through the e820 map searching for the highest available PFN.

1003 `find_max_low_pfn()` finds the highest page frame addressable in `ZONE_NORMAL`.

1005-1011 If high memory is enabled, start with a high memory region of 0. If it turns out memory is available after `max_low_pfn`, put the start of high memory (`highstart_pfn`) there and the end of high memory at `max_pfn`. Print out an informational message on the availability of high memory.

1013-1014 Print out an informational message on the amount of low memory.

```

1018     bootmap_size = init_bootmem(start_pfn, max_low_pfn);
1019
1020     register_bootmem_low_pages(max_low_pfn);
1021
1028     reserve_bootmem(HIGH_MEMORY, (PFN_PHYS(start_pfn) +
1029                                   bootmap_size + PAGE_SIZE-1) - (HIGH_MEMORY));
1030
1035     reserve_bootmem(0, PAGE_SIZE);
1036
1037 #ifdef CONFIG_SMP
1043     reserve_bootmem(PAGE_SIZE, PAGE_SIZE);
1044 #endif
1045 #ifdef CONFIG_ACPI_SLEEP
1046     /*
1047      * Reserve low memory region for sleep support.
1048      */
1049     acpi_reserve_bootmem();
1050 #endif

```

1018 `init_bootmem()` (See Section E.1.1) initializes the `bootmem_data` struct for the `config_page_data` node. It sets where physical memory begins and ends for the node, allocates a bitmap representing the pages and sets all pages as reserved initially.

1020 `register_bootmem_low_pages()` reads the e820 map and calls `free_bootmem()` (See Section E.3.1) for all usable pages in the running system. This marks the pages as reserved during initialization as free.

1028-1029 Reserve the pages that are being used to store the bitmap representing the pages.

1035 Reserves page 0 because it is often a special page used by the BIOS.

1043 Reserves an extra page that is required by the trampoline code. The trampoline code deals with how userspace enters kernel space.

1045-1050 If sleep support is added, reserve memory is required for it. This is only of interest to laptops interested in suspending and is beyond the scope of this book.

```

1051 #ifdef CONFIG_X86_LOCAL_APIC
1052     /*
1053      * Find and reserve possible boot-time SMP configuration:
1054      */
1055     find_smp_config();
1056 #endif
1057 #ifdef CONFIG_BLK_DEV_INITRD
1058     if (LOADER_TYPE && INITRD_START) {
1059         if (INITRD_START + INITRD_SIZE <=
            (max_low_pfn << PAGE_SHIFT)) {
1060             reserve_bootmem(INITRD_START, INITRD_SIZE);
1061             initrd_start =
1062                 INITRD_START? INITRD_START + PAGE_OFFSET : 0;
1063             initrd_end = initrd_start+INITRD_SIZE;
1064         }
1065         else {
1066             printk(KERN_ERR
1067                 "initrd extends beyond end of memory "
1068                 "(0x%08lx > 0x%08lx)\ndisabling initrd\n",
1069                 INITRD_START + INITRD_SIZE,
1070                 max_low_pfn << PAGE_SHIFT);
1071             initrd_start = 0;
1072         }
1073     }
1074 #endif
1075     return max_low_pfn;
1076 }

```

1055 This function reserves memory that stores config information about the SMP setup.

1057-1073 If `initrd` is enabled, the memory containing its image will be reserved. `initrd` provides a tiny filesystem image, which is used to boot the system.

1075 Returns the upper limit of addressable memory in `ZONE_NORMAL`.

B.1.2 Function: `zone_sizes_init()` (*arch/i386/mm/init.c*)

This is the top-level function that is used to initialize each of the zones. The size of the zones in PFNs was discovered during `setup_memory()` (See Section B.1.1). This function populates an array of zone sizes for passing to `free_area_init()`.

```

323 static void __init zone_sizes_init(void)
324 {
325     unsigned long zones_size[MAX_NR_ZONES] = {0, 0, 0};
326     unsigned int max_dma, high, low;
327
328     max_dma = virt_to_phys((char *)MAX_DMA_ADDRESS)>>PAGE_SHIFT;
329     low = max_low_pfn;
330     high = highend_pfn;
331
332     if (low < max_dma)
333         zones_size[ZONE_DMA] = low;
334     else {
335         zones_size[ZONE_DMA] = max_dma;
336         zones_size[ZONE_NORMAL] = low - max_dma;
337 #ifdef CONFIG_HIGHMEM
338         zones_size[ZONE_HIGHMEM] = high - low;
339 #endif
340     }
341     free_area_init(zones_size);
342 }
```

325 Initializes the sizes to 0.

328 Calculates the PFN for the maximum possible DMA address. This doubles as the largest number of pages that may exist in `ZONE_DMA`.

329 `max_low_pfn` is the highest PFN available to `ZONE_NORMAL`.

330 `highend_pfn` is the highest PFN available to `ZONE_HIGHMEM`.

332-333 If the highest PFN in `ZONE_NORMAL` is below `MAX_DMA_ADDRESS`, just set the size of `ZONE_DMA` to it. The other zones remain at 0.

335 Sets the number of pages in `ZONE_DMA`.

336 The size of `ZONE_NORMAL` is `max_low_pfn` minus the number of pages in `ZONE_DMA`.

338 The size of `ZONE_HIGHMEM` is the highest possible PFN minus the highest possible PFN in `ZONE_NORMAL` (`max_low_pfn`).

B.1.3 Function: `free_area_init()` (*mm/page_alloc.c*)

This is the architecture-independent function for setting up a UMA architecture. It simply calls the core function passing the static `contig_page_data` as the node. NUMA architectures will use `free_area_init_node()` instead.

```

838 void __init free_area_init(unsigned long *zones_size)
839 {
840     free_area_init_core(0, &contig_page_data, &mem_map,
                        zones_size, 0, 0, 0);
841 }

```

838 The parameters passed to `free_area_init_core()` are the following:

- **0** is the Node Identifier (NID) for the node, which is 0.
- **contig_page_data** is the static global `pg_data_t`.
- **mem_map** is the global `mem_map` used for tracking `struct pages`. The function `free_area_init_core()` will allocate memory for this array.
- **zones_sizes** is the array of zone sizes filled by `zone_sizes_init()`.
- **0** This zero is the starting physical address.
- **0** The second zero is an array of memory hole sizes that does not apply to UMA architectures.
- **0** The last 0 is a pointer to a local `mem_map` for this node that is used by NUMA architectures.

B.1.4 Function: `free_area_init_node()` (*mm/numa.c*)

This function has two versions. The first is almost identical to `free_area_init()` except that it uses a different starting physical address. This function is also for architectures that have only one node (so they use `contig_page_data`), but their physical address is not at 0.

This version of the function, called after the pagetable initialization, is for initialization of each `pgdat` in the system. The callers have the option of allocating their own local portion of the `mem_map` and passing it in as a parameter if they want to optimize its location for the architecture. If they choose not to, it will be allocated later by `free_area_init_core()`.

```

61 void __init free_area_init_node(int nid,
    pg_data_t *pgdat, struct page *pmap,
62     unsigned long *zones_size, unsigned long zone_start_paddr,
63     unsigned long *zholes_size)
64 {
65     int i, size = 0;
66     struct page *discard;
67
68     if (mem_map == (mem_map_t *)NULL)
69         mem_map = (mem_map_t *)PAGE_OFFSET;

```

```

70
71     free_area_init_core(nid, pgdat, &discard, zones_size,
72                          zone_start_paddr,
73                          zholes_size, pmap);
74     pgdat->node_id = nid;
75     /*
76      * Get space for the valid bitmap.
77      */
78     for (i = 0; i < MAX_NR_ZONES; i++)
79         size += zones_size[i];
80     size = LONG_ALIGN((size + 7) >> 3);
81     pgdat->valid_addr_bitmap =
82         (unsigned long *)alloc_bootmem_node(pgdat, size);
83     memset(pgdat->valid_addr_bitmap, 0, size);
84 }

```

61 The parameters to the function are the following:

- **nid** is the NID of the **pgdat** passed in.
- **pgdat** is the node to be initialized.
- **pmap** is a pointer to the portion of the **mem_map** for this node to use, which is frequently passed as **NULL** and allocated later.
- **zones_size** is an array of zone sizes in this node.
- **zone_start_paddr** is the starting physical address for the node.
- **zholes_size** is an array of hole sizes in each zone.

68-69 If the global **mem_map** has not been set, set it to the beginning of the kernel portion of the linear address space. Remember that, with **NUMA**, **mem_map** is a virtual array with portions filled in by local maps used by each node.

71 Calls **free_area_init_core()**. Note that **discard** is passed in as the third parameter because global **mem_map** does not need to be set for **NUMA**.

73 Records the **pgdat**'s NID.

78-79 Calculates the total size of the NID.

80 Recalculates size as the number of bits required to have one bit for every byte of the size.

81 Allocates a bitmap to represent where valid areas exist in the node. In reality, this is only used by the **Sparc** architecture, so it is unfortunate to waste the memory for every other architecture.

82 Initially, all areas are invalid. Valid regions are marked later in the **mem_init()** functions for the **Sparc**. Other architectures just ignore the bitmap.

B.1.5 Function: `free_area_init_core()` (*mm/page_alloc.c*)

This function is responsible for initializing all zones and allocating their local `lmem_map` within a node. In UMA architectures, this function is called in a way that will initialize the global `mem_map` array. In NUMA architectures, the array is treated as a virtual array that is sparsely populated.

```

684 void __init free_area_init_core(int nid,
    pg_data_t *pgdat, struct page **gmap,
685     unsigned long *zones_size, unsigned long zone_start_paddr,
686     unsigned long *zholes_size, struct page *lmem_map)
687 {
688     unsigned long i, j;
689     unsigned long map_size;
690     unsigned long totalpages, offset, realtotalpages;
691     const unsigned long zone_required_alignment =
        1UL << (MAX_ORDER-1);
692
693     if (zone_start_paddr & ~PAGE_MASK)
694         BUG();
695
696     totalpages = 0;
697     for (i = 0; i < MAX_NR_ZONES; i++) {
698         unsigned long size = zones_size[i];
699         totalpages += size;
700     }
701     realtotalpages = totalpages;
702     if (zholes_size)
703         for (i = 0; i < MAX_NR_ZONES; i++)
704             realtotalpages -= zholes_size[i];
705
706     printk("On node %d totalpages: %lu\n", nid, realtotalpages);

```

This block is mainly responsible for calculating the size of each zone.

691 The zone must be aligned against the maximum-sized block that can be allocated by the buddy allocator for bitwise operations to work.

693-694 It is a bug if the physical address is not page aligned.

696 Initializes the `totalpages` count for this node to 0.

697-700 Calculates the total size of the node by iterating through `zone_sizes`.

701-704 Calculates the real amount of memory by subtracting the size of the holes in `zholes_size`.

706 Prints an informational message for the user on how much memory is available in this node.

```

708     /*
709     * Some architectures (with lots of mem and discontinous memory
710     * maps) have to search for a good mem_map area:
711     * For discontigmem, the conceptual mem map array starts from
712     * PAGE_OFFSET, we need to align the actual array onto a mem map
713     * boundary, so that MAP_NR works.
714     */
715     map_size = (totalpages + 1)*sizeof(struct page);
716     if (lmem_map == (struct page *)0) {
717         lmem_map = (struct page *) alloc_bootmem_node(pgdat, map_size);
718         lmem_map = (struct page *) (PAGE_OFFSET +
719             MAP_ALIGN((unsigned long)lmem_map - PAGE_OFFSET));
720     }
721     *gmap = pgdat->node_mem_map = lmem_map;
722     pgdat->node_size = totalpages;
723     pgdat->node_start_paddr = zone_start_paddr;
724     pgdat->node_start_mapnr = (lmem_map - mem_map);
725     pgdat->nr_zones = 0;
726
727     offset = lmem_map - mem_map;

```

This block allocates the local `lmem_map` if necessary and sets the `gmap`. In UMA architectures, `gmap` is actually `mem_map`, so this is where the memory for it is allocated.

- 715** Calculates the amount of memory required for the array. It is the total number of pages multiplied by the size of a `struct page`.
- 716** If the map has not already been allocated, this allocates it.
- 717** Allocates the memory from the boot memory allocator.
- 718** `MAP_ALIGN()` will align the array on a `struct page`-sized boundary for calculations that locate offsets within the `mem_map` based on the physical address with the `MAP_NR()` macro.
- 721** Sets the `gmap` and `pgdat->node_mem_map` variables to the allocated `lmem_map`. In UMA architectures, this just sets `mem_map`.
- 722** Records the size of the node.
- 723** Records the starting physical address.
- 724** Records what the offset is within `mem_map` that this node occupies.
- 725** Initializes the zone count to 0. This will be set later in the function.
- 727** `offset` is now the offset within `mem_map` that the local portion `lmem_map` begins at.

```

728     for (j = 0; j < MAX_NR_ZONES; j++) {
729         zone_t *zone = pgdat->node_zones + j;
730         unsigned long mask;
731         unsigned long size, realsize;
732
733         zone_table[nid * MAX_NR_ZONES + j] = zone;
734         realsize = size = zones_size[j];
735         if (zholes_size)
736             realsize -= zholes_size[j];
737
738         printk("zone(%lu): %lu pages.\n", j, size);
739         zone->size = size;
740         zone->name = zone_names[j];
741         zone->lock = SPIN_LOCK_UNLOCKED;
742         zone->zone_pgdat = pgdat;
743         zone->free_pages = 0;
744         zone->need_balance = 0;
745         if (!size)
746             continue;

```

This block starts a loop that initializes every `zone_t` within the node. The initialization starts with the setting of the simpler fields that values already exist for.

728 Loops through all zones in the node.

733 Records a pointer to this zone in the `zone_table`. See Section 2.6.

734-736 Calculates the real size of the zone based on the full size in `zones_size` minus the size of the holes in `zholes_size`.

738 Prints an informational message saying how many pages are in this zone.

739 Records the size of the zone.

740 `zone_names` is the string name of the zone for printing purposes.

741-744 Initializes some other fields for the zone such as its parent `pgdat`.

745-746 If the zone has no memory, this continues to the next zone because nothing further is required.

```

752         zone->wait_table_size = wait_table_size(size);
753         zone->wait_table_shift =
754             BITS_PER_LONG - wait_table_bits(zone->wait_table_size);
755         zone->wait_table = (wait_queue_head_t *)
756             alloc_bootmem_node(pgdat, zone->wait_table_size
757                 * sizeof(wait_queue_head_t));
758

```

```

759         for(i = 0; i < zone->wait_table_size; ++i)
760             init_waitqueue_head(zone->wait_table + i);

```

This block initializes the waitqueue for this zone. Processes waiting on pages in the zone use this hashed table to select a queue to wait on. This means that all processes waiting in a zone will not have to be woken when a page is unlocked, just a smaller subset.

752 `wait_table_size()` calculates the size of the table to use based on the number of pages in the zone and the desired ratio between the number of queues and the number of pages. The table will never be larger than 4KiB.

753-754 Calculates the shift for the hashing algorithm.

755 Allocates a table of `wait_queue_head_t` that can hold `zone->wait_table_size` entries.

759-760 Initializes all of the wait queues.

```

762         pgdat->nr_zones = j+1;
763
764         mask = (realsize / zone_balance_ratio[j]);
765         if (mask < zone_balance_min[j])
766             mask = zone_balance_min[j];
767         else if (mask > zone_balance_max[j])
768             mask = zone_balance_max[j];
769         zone->pages_min = mask;
770         zone->pages_low = mask*2;
771         zone->pages_high = mask*3;
772
773         zone->zone_mem_map = mem_map + offset;
774         zone->zone_start_mapnr = offset;
775         zone->zone_start_paddr = zone_start_paddr;
776
777         if ((zone_start_paddr >> PAGE_SHIFT) &
              (zone_required_alignment-1))
778             printk("BUG: wrong zone alignment, it will crash\n");
779

```

This block calculates the watermarks for the zone and records the location of the zone. The watermarks are calculated as ratios of the zone size.

762 First, as a new zone becomes active, this updates the number of zones in this node.

764 Calculates the mask (which will be used as the `pages_min` watermark) as the size of the zone divided by the balance ratio for this zone. The balance ratio is 128 for all zones as declared at the top of `mm/page_alloc.c`.

765-766 The `zone_balance_min` ratios are 20 for all zones, which means that `pages_min` will never be below 20.

767-768 Similarly, the `zone_balance_max` ratios are all 255, so `pages_min` will never be over 255.

769 `pages_min` is set to `mask`.

770 `pages_low` is twice the number of pages as `pages_min`.

771 `pages_high` is three times the number of pages as `pages_min`.

773 Records where the first `struct page` for this zone is located within `mem_map`.

774 Records the index within `mem_map` that this zone begins at.

775 Records the starting physical address.

777-778 Ensures that the zone is correctly aligned for use with the buddy allocator. Otherwise, the bitwise operations used for the buddy allocator will break.

```

780      /*
781      * Initially all pages are reserved - free ones are freed
782      * up by free_all_bootmem() once the early boot process is
783      * done. Non-atomic initialization, single-pass.
784      */
785      for (i = 0; i < size; i++) {
786          struct page *page = mem_map + offset + i;
787          set_page_zone(page, nid * MAX_NR_ZONES + j);
788          set_page_count(page, 0);
789          SetPageReserved(page);
790          INIT_LIST_HEAD(&page->list);
791          if (j != ZONE_HIGHMEM)
792              set_page_address(page, __va(zone_start_paddr));
793          zone_start_paddr += PAGE_SIZE;
794      }
795
```

785-794 Initially, all pages in the zone are marked as reserved because there is no way to know which ones are in use by the boot memory allocator. When the boot memory allocator is retiring in `free_all_bootmem()`, the unused pages will have their `PG_reserved` bit cleared.

786 Gets the page for this offset.

787 The zone the page belongs to is encoded with the page flags. See Section 2.6.

788 Sets the count to 0 because no one is using it.

789 Sets the reserved flag. Later, the boot memory allocator will clear this bit if the page is no longer in use.

790 Initializes the list head for the page.

791-792 Sets the `page→virtual` field if it is available and the page is in low memory.

793 Increments `zone_start_paddr` by a page size because this variable will be used to record the beginning of the next zone.

```

796         offset += size;
797         for (i = 0; ; i++) {
798             unsigned long bitmap_size;
799
800             INIT_LIST_HEAD(&zone->free_area[i].free_list);
801             if (i == MAX_ORDER-1) {
802                 zone->free_area[i].map = NULL;
803                 break;
804             }
805
829             bitmap_size = (size-1) >> (i+4);
830             bitmap_size = LONG_ALIGN(bitmap_size+1);
831             zone->free_area[i].map =
832                 (unsigned long *) alloc_bootmem_node(pgdat,
                                                         bitmap_size);
833         }
834     }
835     build_zonelists(pgdat);
836 }
```

This block initializes the free lists for the zone and allocates the bitmap used by the buddy allocator to record the state of page buddies.

797 This will loop from 0 to `MAX_ORDER-1`.

800 Initializes the linked list for the `free_list` of the current order `i`.

801-804 If this is the last order, this sets the free area map to `NULL` because this is what marks the end of the free lists.

829 Calculates the `bitmap_size` to be the number of bytes required to hold a bitmap where each bit represents a pair of buddies that are 2^i number of pages.

830 Aligns the size to a long with `LONG_ALIGN()` because all bitwise operations are on longs.

831-832 Allocates the memory for the map.

834 Loops back to move to the next zone.

835 Builds the zone fallback lists for this node with `build_zonelists()`.

B.1.6 Function: `build_zonelists()` (*mm/page_alloc.c*)

This function builds the list of fallback zones for each zone in the requested node. This is for when an allocation cannot be satisfied and another zone is consulted. When this consultation is finished, allocations from `ZONE_HIGHMEM` will fall back to `ZONE_NORMAL`. Allocations from `ZONE_NORMAL` will fall back to `ZONE_DMA`, which in turn has nothing to fall back on.

```

589 static inline void build_zonelists(pg_data_t *pgdat)
590 {
591     int i, j, k;
592
593     for (i = 0; i <= GFP_ZONEMASK; i++) {
594         zonelist_t *zonelist;
595         zone_t *zone;
596
597         zonelist = pgdat->node_zonelists + i;
598         memset(zonelist, 0, sizeof(*zonelist));
599
600         j = 0;
601         k = ZONE_NORMAL;
602         if (i & __GFP_HIGHMEM)
603             k = ZONE_HIGHMEM;
604         if (i & __GFP_DMA)
605             k = ZONE_DMA;
606
607         switch (k) {
608             default:
609                 BUG();
610             /*
611              * fallthrough:
612              */
613             case ZONE_HIGHMEM:
614                 zone = pgdat->node_zones + ZONE_HIGHMEM;
615                 if (zone->size) {
616 #ifndef CONFIG_HIGHMEM
617                     BUG();
618 #endif
619                 }
620                 zonelist->zones[j++] = zone;
621             case ZONE_NORMAL:
622                 zone = pgdat->node_zones + ZONE_NORMAL;
623                 if (zone->size)

```

```
624             zonelist->zones[j++] = zone;
625         case ZONE_DMA:
626             zone = pgdat->node_zones + ZONE_DMA;
627             if (zone->size)
628                 zonelist->zones[j++] = zone;
629         }
630     zonelist->zones[j++] = NULL;
631 }
632 }
```

593 Looks through the maximum possible number of zones.

597 Gets the `zonelist` for this zone and zeros it.

600 Starts `j` at 0, which corresponds to `ZONE_DMA`.

601-605 Sets `k` to be the type of zone currently being examined.

614 Gets the `ZONE_HIGHMEM`.

615-620 If the zone has memory, `ZONE_HIGHMEM` is the preferred zone to allocate from for high memory allocations. If `ZONE_HIGHMEM` has no memory, `ZONE_NORMAL` will become the preferred zone when the next case is fallen through to because `j` is not incremented for an empty zone.

621-624 Sets the next preferred zone to allocate from to be `ZONE_NORMAL`. Again, do not use it if the zone has no memory.

626-628 Sets the final fallback zone to be `ZONE_DMA`. The check is still made for `ZONE_DMA` having memory. Like NUMA architecture, not all nodes will have a `ZONE_DMA`.

B.2 Page Operations

Contents

B.2 Page Operations	234
B.2.1 Locking Pages	234
B.2.1.1 Function: <code>lock_page()</code>	234
B.2.1.2 Function: <code>__lock_page()</code>	234
B.2.1.3 Function: <code>sync_page()</code>	235
B.2.2 Unlocking Pages	236
B.2.2.1 Function: <code>unlock_page()</code>	236
B.2.3 Waiting on Pages	236
B.2.3.1 Function: <code>wait_on_page()</code>	236
B.2.3.2 Function: <code>___wait_on_page()</code>	237

B.2.1 Locking Pages

B.2.1.1 Function: `lock_page()` (*mm/filemap.c*)

This function tries to lock a page. If the page cannot be locked, it will cause the process to sleep until the page is available.

```

921 void lock_page(struct page *page)
922 {
923     if (TryLockPage(page))
924         __lock_page(page);
925 }
```

923 `TryLockPage()` is just a wrapper around `test_and_set_bit()` for the `PG_locked` bit in `page→flags`. If the bit was previously clear, the function returns immediately because the page is now locked.

924 Otherwise, `__lock_page()` is called (See Section B.2.1.2) to put the process to sleep.

B.2.1.2 Function: `__lock_page()` (*mm/filemap.c*)

This is called after a `TryLockPage()` failed. It will locate the waitqueue for this page and sleep on it until the lock can be acquired.

```

897 static void __lock_page(struct page *page)
898 {
899     wait_queue_head_t *waitqueue = page_waitqueue(page);
900     struct task_struct *tsk = current;
901     DECLARE_WAITQUEUE(wait, tsk);
902
903     add_wait_queue_exclusive(waitqueue, &wait);
904     for (;;) {
905         set_task_state(tsk, TASK_UNINTERRUPTIBLE);
906         if (PageLocked(page)) {
```

```

907         sync_page(page);
908         schedule();
909     }
910     if (!TryLockPage(page))
911         break;
912 }
913 __set_task_state(tsk, TASK_RUNNING);
914 remove_wait_queue(waitqueue, &wait);
915 }

```

899 `page_waitqueue()` is the implementation of the hash algorithm that determines which wait queue this page belongs to in the table `zone→wait_table`.

900-901 Initializes the waitqueue for this task.

903 Adds this process to the waitqueue returned by `page_waitqueue()`.

904-912 Loops here until the lock is acquired.

905 Sets the process states as being in uninterruptible sleep. When `schedule()` is called, the process will be put to sleep and will not wake again until the queue is explicitly woken up.

906 If the page is still locked, this calls the `sync_page()` function to schedule the page to be synchronized with its backing storage. It calls `schedule()` to sleep until the queue is woken up, such as when the I/O on the page completes.

910-911 Try and lock the page again. If we succeed, exit the loop, otherwise sleep on the queue again.

913-914 The lock is now acquired, so this sets the process state to `TASK_RUNNING` and removes it from the wait queue. The function now returns with the lock acquired.

B.2.1.3 Function: `sync_page()` (*mm/filemap.c*)

This calls the filesystem-specific `sync_page()` to synchronize the page with its backing storage.

```

140 static inline int sync_page(struct page *page)
141 {
142     struct address_space *mapping = page->mapping;
143
144     if (mapping && mapping->a_ops && mapping->a_ops->sync_page)
145         return mapping->a_ops->sync_page(page);
146     return 0;
147 }

```

142 Gets the `address_space` for the page if it exists.

144-145 If a backing exists, and it has an associated `address_space_operations`, which provides a `sync_page()` function, this calls it.

B.2.2 Unlocking Pages

B.2.2.1 Function: `unlock_page()` (*mm/filemap.c*)

This function unlocks a page and wakes up any processes that may be waiting on it.

```

874 void unlock_page(struct page *page)
875 {
876     wait_queue_head_t *waitqueue = page_waitqueue(page);
877     ClearPageLaunder(page);
878     smp_mb__before_clear_bit();
879     if (!test_and_clear_bit(PG_locked, &(page)->flags))
880         BUG();
881     smp_mb__after_clear_bit();
882
883     /*
884      * Although the default semantics of wake_up() are
885      * to wake all, here the specific function is used
886      * to make it even more explicit that a number of
887      * pages are being waited on here.
888      */
889     if (waitqueue_active(waitqueue))
890         wake_up_all(waitqueue);
891 }
```

876 `page_waitqueue()` is the implementation of the hash algorithm, which determines which wait queue this page belongs to in the table `zone→wait_table`.

877 Clears the launder bit because I/O has now completed on the page.

878 This is a memory block operation that must be called before performing bit operations that may be seen by multiple processors.

879-880 Clears the `PG_locked` bit. It is a `BUG()` if the bit was already cleared.

881 Completes the SMP memory block operation.

889-890 If there are processes waiting on the page queue for this page, this wakes them.

B.2.3 Waiting on Pages

B.2.3.1 Function: `wait_on_page()` (*include/linux/pagemap.h*)

```

94 static inline void wait_on_page(struct page * page)
95 {
96     if (PageLocked(page))
97         __wait_on_page(page);
98 }
```

96-97 If the page is currently locked, this calls `___wait_on_page()` to sleep until it is unlocked.

B.2.3.2 Function: `___wait_on_page()` (*mm/filemap.c*)

This function is called after `PageLocked()` has been used to determine that the page is locked. The calling process will probably sleep until the `page` is unlocked.

```

849 void ___wait_on_page(struct page *page)
850 {
851     wait_queue_head_t *waitqueue = page_waitqueue(page);
852     struct task_struct *tsk = current;
853     DECLARE_WAITQUEUE(wait, tsk);
854
855     add_wait_queue(waitqueue, &wait);
856     do {
857         set_task_state(tsk, TASK_UNINTERRUPTIBLE);
858         if (!PageLocked(page))
859             break;
860         sync_page(page);
861         schedule();
862     } while (PageLocked(page));
863     __set_task_state(tsk, TASK_RUNNING);
864     remove_wait_queue(waitqueue, &wait);
865 }
```

851 `page_waitqueue()` is the implementation of the hash algorithm that determines which wait queue this page belongs to in the table `zone→wait_table`.

852-853 Initializes the waitqueue for the current task.

855 Adds this task to the waitqueue returned by `page_waitqueue()`.

857 Sets the process state to be in uninterruptible sleep. When `schedule()` is called, the process will sleep.

858-859 Checks to make sure the page was not unlocked since the last check.

860 Calls `sync_page()` (See Section B.2.1.3) to call the filesystem-specific function to synchronize the page with its backing storage.

861 Calls `schedule()` to go to sleep. The process will be woken when the page is unlocked.

862 Checks if the page is still locked. Remember that multiple pages could be using this wait queue, and there could be processes sleeping that want to lock this page.

863-864 The page has been unlocked. It sets the process to be in the `TASK_RUNNING` state and removes the process from the waitqueue.

Page Table Management

Contents

C.1	Page Table Initialization	240
C.1.1	Function: <code>paging_init()</code>	240
C.1.2	Function: <code>pagetable_init()</code>	241
C.1.3	Function: <code>fixrange_init()</code>	245
C.1.4	Function: <code>kmap_init()</code>	246
C.2	Page Table Walking	248
C.2.1	Function: <code>follow_page()</code>	248

C.1 Page Table Initialization

Contents

C.1 Page Table Initialization	240
C.1.1 Function: <code>paging_init()</code>	240
C.1.2 Function: <code>pagetable_init()</code>	241
C.1.3 Function: <code>fixrange_init()</code>	245
C.1.4 Function: <code>kmap_init()</code>	246

C.1.1 Function: `paging_init()` (*arch/i386/mm/init.c*)

This is the top-level function called from `setup_arch()`. When this function returns, the page tables have been fully set up. Be aware that this is all x86 specific.

```

351 void __init paging_init(void)
352 {
353     pagetable_init();
354
355     load_cr3(swapper_pg_dir);
356
357 #if CONFIG_X86_PAE
362     if (cpu_has_pae)
363         set_in_cr4(X86_CR4_PAE);
364 #endif
365
366     __flush_tlb_all();
367
368 #ifdef CONFIG_HIGHMEM
369     kmap_init();
370 #endif
371     zone_sizes_init();
372 }
```

353 `pagetable_init()` is responsible for setting up a static page table using `swapper_pg_dir` as the PGD.

355 Loads the initialized `swapper_pg_dir` into the CR3 register so that the CPU will be able to use it.

362-363 If PAE is enabled, this sets the appropriate bit in the CR4 register.

366 Flushes all TLBs, including the global kernel ones.

369 `kmap_init()` initializes the region of pagetables reserved for use with `kmap()`.

371 `zone_sizes_init()` (See Section B.1.2) records the size of each of the zones before calling `free_area_init()` (See Section B.1.3) to initialize each zone.

C.1.2 Function: `pagetable_init()` (*arch/i386/mm/init.c*)

This function is responsible for statically initializing a pagetable starting with a statically defined PGD called `swapper_pg_dir`. At the very least, a PTE will be available that points to every page frame in `ZONE_NORMAL`.

```

205 static void __init pagetable_init (void)
206 {
207     unsigned long vaddr, end;
208     pgd_t *pgd, *pgd_base;
209     int i, j, k;
210     pmd_t *pmd;
211     pte_t *pte, *pte_base;
212
213     /*
214      * This can be zero as well - no problem, in that case we exit
215      * the loops anyway due to the PTRS_PER_* conditions.
216      */
217     end = (unsigned long)__va(max_low_pfn*PAGE_SIZE);
218
219     pgd_base = swapper_pg_dir;
220 #if CONFIG_X86_PAE
221     for (i = 0; i < PTRS_PER_PGD; i++)
222         set_pgd(pgd_base + i, __pgd(1 + __pa(empty_zero_page)));
223 #endif
224     i = __pgd_offset(PAGE_OFFSET);
225     pgd = pgd_base + i;

```

This first block initializes the PGD. It does this by pointing each entry to the global zero page. Entries needed to reference available memory in `ZONE_NORMAL` will be allocated later.

217 The variable `end` marks the end of physical memory in `ZONE_NORMAL`.

219 `pgd_base` is set to the beginning of the statically declared PGD.

220-223 If PAE is enabled, it is insufficient to leave each entry simply as 0 (which, in effect, points each entry to the global zero page) because each `pgd_t` is a struct. Instead, `set_pgd` must be called for each `pgd_t` to point the entry to the global zero page.

224 `i` is initialized as the offset within the PGD that corresponds to `PAGE_OFFSET`. In other words, this function will only be initializing the kernel portion of the linear address space. The userspace portion is left alone.

225 `pgd` is initialized to the `pgd_t` corresponding to the beginning of the kernel portion of the linear address space.

```

227     for (; i < PTRS_PER_PGD; pgd++, i++) {
228         vaddr = i*PGDIR_SIZE;
229         if (end && (vaddr >= end))
230             break;
231     #if CONFIG_X86_PAE
232         pmd = (pmd_t *) alloc_bootmem_low_pages(PAGE_SIZE);
233         set_pgd(pgd, __pgd(__pa(pmd) + 0x1));
234     #else
235         pmd = (pmd_t *)pgd;
236     #endif
237         if (pmd != pmd_offset(pgd, 0))
238             BUG();

```

This loop begins setting up valid PMD entries to point to. In the PAE case, pages are allocated with `alloc_bootmem_low_pages()`, and the PGD is set appropriately. Without PAE, there is no middle directory, so it is just folded back onto the PGD to preserve the illusion of a three-level pagetable.

227 `i` is already initialized to the beginning of the kernel portion of the linear address space, so this keeps looping until the last `pgd_t` at `PTRS_PER_PGD` is reached.

228 Calculates the virtual address for this PGD.

229-230 If the end of `ZONE_NORMAL` is reached, this exits the loop because further pagetable entries are not needed.

231-234 If PAE is enabled, this allocates a page for the PMD and inserts the page into the pagetable with `set_pgd()`.

235 If PAE is not available, just set `pmd` to the current `pgd_t`. This is the “folding back” trick for emulating three-level pagetables.

237-238 This is a sanity check to make sure the PMD is valid.

```

239         for (j = 0; j < PTRS_PER_PMD; pmd++, j++) {
240             vaddr = i*PGDIR_SIZE + j*PMD_SIZE;
241             if (end && (vaddr >= end))
242                 break;
243             if (cpu_has_pse) {
244                 unsigned long __pe;
245
246                 set_in_cr4(X86_CR4_PSE);
247                 boot_cpu_data.wp_works_ok = 1;
248                 __pe = _KERNPG_TABLE + _PAGE_PSE + __pa(vaddr);
249                 /* Make it "global" too if supported */
250                 if (cpu_has_pge) {
251                     set_in_cr4(X86_CR4_PGE);

```

```

252             __pe += _PAGE_GLOBAL;
253         }
254         set_pmd(pmd, __pmd(__pe));
255         continue;
256     }
257
258     pte_base = pte =
                (pte_t *) alloc_bootmem_low_pages(PAGE_SIZE);
259

```

This block initializes each entry in the PMD. This loop will only execute if PAE is enabled. Remember that, without PAE, PTRS_PER_PMD is 1.

240 Calculates the virtual address for this PMD.

241-242 If the end of `ZONE_NORMAL` is reached, this finishes.

243-248 If the CPU supports PSE, use large TLB entries. This means that, for kernel pages, a TLB entry will map 4MiB instead of the normal 4KiB, and the third level of PTEs is unnecessary.

258 `__pe` is set as the flags for a kernel pagetable (`_KERNPG_TABLE`), as the flag to indicate that this is an entry mapping 4MiB (`_PAGE_PSE`) and then to the physical address for this virtual address with `__pa()`. This means that 4MiB of physical memory is not being mapped by the pagetables.

250-253 If the CPU supports PGE, then set it for this page table entry. This marks the entry as being global and visible to all processes.

254-255 Because the third level is not required because of PSE, set the PMD now with `set_pmd()` and continue to the next PMD.

258 If not, PSE is not supported, and PTEs are required, so allocate a page for them.

```

260         for (k = 0; k < PTRS_PER_PTE; pte++, k++) {
261             vaddr = i*PGDIR_SIZE + j*PMD_SIZE + k*PAGE_SIZE;
262             if (end && (vaddr >= end))
263                 break;
264             *pte = mk_pte_phys(__pa(vaddr), PAGE_KERNEL);
265         }
266         set_pmd(pmd, __pmd(_KERNPG_TABLE + __pa(pte_base)));
267         if (pte_base != pte_offset(pmd, 0))
268             BUG();
269
270     }
271 }

```

This block initializes the PTEs.

260-265 For each `pte_t`, calculate the virtual address currently being examined and create a PTE that points to the appropriate physical page frame.

266 The PTEs have been initialized, so set the PMD to point to the page containing them.

267-268 Makes sure that the entry was established correctly.

```

273     /*
274     * Fixed mappings, only the page table structure has to be
275     * created - mappings will be set by set_fixmap():
276     */
277     vaddr = __fix_to_virt(__end_of_fixed_addresses - 1) & PMD_MASK;
278     fixrange_init(vaddr, 0, pgd_base);
279
280 #if CONFIG_HIGHMEM
281     /*
282     * Permanent kmaps:
283     */
284     vaddr = PKMAP_BASE;
285     fixrange_init(vaddr, vaddr + PAGE_SIZE*LAST_PKMAP, pgd_base);
286
287     pgd = swapper_pg_dir + __pgd_offset(vaddr);
288     pmd = pmd_offset(pgd, vaddr);
289     pte = pte_offset(pmd, vaddr);
290     pkmap_page_table = pte;
291 #endif
292
293 #if CONFIG_X86_PAE
294     /*
295     * Add low memory identity-mappings - SMP needs it when
296     * starting up on an AP from real-mode. In the non-PAE
297     * case we already have these mappings through head.S.
298     * All user-space mappings are explicitly cleared after
299     * SMP startup.
300     */
301     pgd_base[0] = pgd_base[USER_PTRS_PER_PGD];
302 #endif
303 }
```

At this point, pagetable entries have been set up that reference all parts of `ZONE_NORMAL`. The remaining regions needed are those for fixed mappings and those needed for mapping high memory pages with `kmap()`.

277 The fixed address space is considered to start at `FIXADDR_TOP` and to finish earlier in the address space. `__fix_to_virt()` takes an index as a parameter and returns the index'th pageframe backward (starting from `FIXADDR_TOP`)

within the fixed virtual address space. `__end_of_fixed_addresses` is the last index used by the fixed virtual address space. In other words, this line returns the virtual address of the PMD that corresponds to the beginning of the fixed virtual address space.

278 By passing 0 as the end to `fixrange_init()`, the function will start at `vaddr` and build valid PGDs and PMDs until the end of the virtual address space. PTEs are not needed for these addresses.

280-291 Sets up pagetables for use with `kmap()`.

287-290 Gets the PTE corresponding to the beginning of the region for use with `kmap()`.

301 This sets up a temporary identity mapping between the virtual address 0 and the physical address 0.

C.1.3 Function: `fixrange_init()` (*arch/i386/mm/init.c*)

This function creates valid PGDs and PMDs for fixed virtual address mappings.

```

167 static void __init fixrange_init (unsigned long start,
                                   unsigned long end,
                                   pgd_t *pgd_base)
168 {
169     pgd_t *pgd;
170     pmd_t *pmd;
171     pte_t *pte;
172     int i, j;
173     unsigned long vaddr;
174
175     vaddr = start;
176     i = __pgd_offset(vaddr);
177     j = __pmd_offset(vaddr);
178     pgd = pgd_base + i;
179
180     for ( ; (i < PTRS_PER_PGD) && (vaddr != end); pgd++, i++) {
181 #if CONFIG_X86_PAE
182         if (pgd_none(*pgd)) {
183             pmd = (pmd_t *) alloc_bootmem_low_pages(PAGE_SIZE);
184             set_pgd(pgd, __pgd(__pa(pmd) + 0x1));
185             if (pmd != pmd_offset(pgd, 0))
186                 printk("PAE BUG #02!\n");
187         }
188         pmd = pmd_offset(pgd, vaddr);
189 #else
190         pmd = (pmd_t *)pgd;
191 #endif

```

```

192         for (; (j < PTRS_PER_PMD) && (vaddr != end); pmd++, j++) {
193             if (pmd_none(*pmd)) {
194                 pte = (pte_t *) alloc_bootmem_low_pages(PAGE_SIZE);
195                 set_pmd(pmd, __pmd(_KERNPG_TABLE + __pa(pte)));
196                 if (pte != pte_offset(pmd, 0))
197                     BUG();
198             }
199             vaddr += PMD_SIZE;
200         }
201         j = 0;
202     }
203 }

```

175 Sets the starting virtual address (**vaddr**) to the requested starting address provided as the parameter.

176 Gets the index within the PGD corresponding to **vaddr**.

177 Gets the index within the PMD corresponding to **vaddr**.

178 Gets the starting **pgd_t**.

180 Keeps cycling until **end** is reached. When **pagetable_init()** passes in 0, this loop will continue until the end of the PGD.

182-187 In the case of PAE, this allocates a page for the PMD if one has not already been allocated.

190 Without PAE, there is no PMD, so this treats the **pgd_t** as the **pmd_t**.

192-200 For each entry in the PMD, this allocates a page for the **pte_t** entries and sets it within the pagetables. Note that **vaddr** is incremented in PMD-sized strides.

C.1.4 Function: **kmap_init()** (*arch/i386/mm/init.c*)

This function only exists if **CONFIG_HIGHMEM** is set during compile time. It is responsible for caching where the beginning of the kmap region is, the PTE referencing it and the protection for the page tables. This means the PGD will not have to be checked every time **kmap()** is used.

```

74 #if CONFIG_HIGHMEM
75 pte_t *kmap_pte;
76 pgprot_t kmap_prot;
77
78 #define kmap_get_fixmap_pte(vaddr) \
79     pte_offset(pmd_offset(pgd_offset_k(vaddr), (vaddr)), (vaddr))
80
81 void __init kmap_init(void)

```



```
82 {
83     unsigned long kmap_vstart;
84
85     /* cache the first kmap pte */
86     kmap_vstart = __fix_to_virt(FIX_KMAP_BEGIN);
87     kmap_pte = kmap_get_fixmap_pte(kmap_vstart);
88
89     kmap_prot = PAGE_KERNEL;
90 }
91 #endif /* CONFIG_HIGHMEM */
```

78-79 Because `fixrange_init()` has already set up valid PGDs and PMDs, there is no need to double-check them, so `kmap_get_fixmap_pte()` is responsible for quickly traversing the pagetable.

86 Caches the virtual address for the kmap region in `kmap_vstart`.

87 Caches the PTE for the start of the kmap region in `kmap_pte`.

89 Caches the protection for the pagetable entries with `kmap_prot`.

C.2 Page Table Walking

Contents

C.2 Page Table Walking	248
C.2.1 Function: <code>follow_page()</code>	248

C.2.1 Function: `follow_page()` (*mm/memory.c*)

This function returns the `struct page` used by the PTE at `address` in `mm`'s pagetables.

```

405 static struct page * follow_page(struct mm_struct *mm,
                                   unsigned long address,
                                   int write)
406 {
407     pgd_t *pgd;
408     pmd_t *pmd;
409     pte_t *ptep, pte;
410
411     pgd = pgd_offset(mm, address);
412     if (pgd_none(*pgd) || pgd_bad(*pgd))
413         goto out;
414
415     pmd = pmd_offset(pgd, address);
416     if (pmd_none(*pmd) || pmd_bad(*pmd))
417         goto out;
418
419     ptep = pte_offset(pmd, address);
420     if (!ptep)
421         goto out;
422
423     pte = *ptep;
424     if (pte_present(pte)) {
425         if (!write ||
426             (pte_write(pte) && pte_dirty(pte)))
427             return pte_page(pte);
428     }
429
430 out:
431     return 0;
432 }
```

405 The parameters are the `mm` with the pagetables that are about to be walked, the `address` that has the `struct page` of interest and `write`, which indicates if the page is about to be written to.

411 Gets the PGD for the `address` and makes sure it is present and valid.

415-417 Gets the PMD for the `address` and makes sure it is present and valid.

- 419** Gets the PTE for the `address` and makes sure it exists.
- 424** If the PTE is currently present, then something can be returned.
- 425-426** If the caller has indicated a write is about to take place, this checks to make sure that the PTE has write permissions set and, if so, makes the PTE dirty.
- 427** If the PTE is present and the permissions are fine, this returns the `struct page` mapped by the PTE.
- 431** Returns 0, indicating that the `address` has no associated `struct page`.

Process Address Space

Contents

D.1 Process Memory Descriptors	254
D.1.1 Initializing a Descriptor	254
D.1.2 Copying a Descriptor	254
D.1.2.1 Function: <code>copy_mm()</code>	254
D.1.2.2 Function: <code>mm_init()</code>	257
D.1.3 Allocating a Descriptor	257
D.1.3.1 Function: <code>allocate_mm()</code>	257
D.1.3.2 Function: <code>mm_alloc()</code>	258
D.1.4 Destroying a Descriptor	258
D.1.4.1 Function: <code>mmput()</code>	258
D.1.4.2 Function: <code>mmdrop()</code>	259
D.1.4.3 Function: <code>__mmdrop()</code>	260
D.2 Creating Memory Regions	261
D.2.1 Creating a Memory Region	261
D.2.1.1 Function: <code>do_mmap()</code>	261
D.2.1.2 Function: <code>do_mmap_pgoff()</code>	262
D.2.2 Inserting a Memory Region	270
D.2.2.1 Function: <code>__insert_vm_struct()</code>	270
D.2.2.2 Function: <code>find_vma_prepare()</code>	271
D.2.2.3 Function: <code>vma_link()</code>	272
D.2.2.4 Function: <code>__vma_link()</code>	273
D.2.2.5 Function: <code>__vma_link_list()</code>	273
D.2.2.6 Function: <code>__vma_link_rb()</code>	274
D.2.2.7 Function: <code>__vma_link_file()</code>	274
D.2.3 Merging Contiguous Regions	275
D.2.3.1 Function: <code>vma_merge()</code>	275
D.2.3.2 Function: <code>can_vma_merge()</code>	277
D.2.4 Remapping and Moving a Memory Region	278
D.2.4.1 Function: <code>sys_mremap()</code>	278
D.2.4.2 Function: <code>do_mremap()</code>	278
	251

D.2.4.3	Function: <code>move_vma()</code>	284
D.2.4.4	Function: <code>make_pages_present()</code>	288
D.2.4.5	Function: <code>get_user_pages()</code>	289
D.2.4.6	Function: <code>move_page_tables()</code>	292
D.2.4.7	Function: <code>move_one_page()</code>	293
D.2.4.8	Function: <code>get_one_pte()</code>	294
D.2.4.9	Function: <code>alloc_one_pte()</code>	295
D.2.4.10	Function: <code>copy_one_pte()</code>	295
D.2.5	Deleting a Memory Region	296
D.2.5.1	Function: <code>do_munmap()</code>	296
D.2.5.2	Function: <code>unmap_fixup()</code>	300
D.2.6	Deleting All Memory Regions	304
D.2.6.1	Function: <code>exit_mmap()</code>	304
D.2.6.2	Function: <code>clear_page_tables()</code>	306
D.2.6.3	Function: <code>free_one_pgd()</code>	306
D.2.6.4	Function: <code>free_one_pmd()</code>	307
D.3	Searching Memory Regions	309
D.3.1	Finding a Mapped Memory Region	309
D.3.1.1	Function: <code>find_vma()</code>	309
D.3.1.2	Function: <code>find_vma_prev()</code>	310
D.3.1.3	Function: <code>find_vma_intersection()</code>	312
D.3.2	Finding a Free Memory Region	312
D.3.2.1	Function: <code>get_unmapped_area()</code>	312
D.3.2.2	Function: <code>arch_get_unmapped_area()</code>	313
D.4	Locking and Unlocking Memory Regions	315
D.4.1	Locking a Memory Region	315
D.4.1.1	Function: <code>sys_mlock()</code>	315
D.4.1.2	Function: <code>sys_mlockall()</code>	316
D.4.1.3	Function: <code>do_mlockall()</code>	318
D.4.1.4	Function: <code>do_mlock()</code>	319
D.4.2	Unlocking the Region	321
D.4.2.1	Function: <code>sys_munlock()</code>	321
D.4.2.2	Function: <code>sys_munlockall()</code>	321
D.4.3	Fixing Up Regions After Locking/Unlocking	322
D.4.3.1	Function: <code>mlock_fixup()</code>	322
D.4.3.2	Function: <code>mlock_fixup_all()</code>	323
D.4.3.3	Function: <code>mlock_fixup_start()</code>	324
D.4.3.4	Function: <code>mlock_fixup_end()</code>	325
D.4.3.5	Function: <code>mlock_fixup_middle()</code>	326
D.5	Page Faulting	328
D.5.1	x86 Page Fault Handler	328
D.5.1.1	Function: <code>do_page_fault()</code>	328
D.5.2	Expanding the Stack	337

D.5.2.1	Function: <code>expand_stack()</code>	337
D.5.3	Architecture-Independent Page Fault Handler	338
D.5.3.1	Function: <code>handle_mm_fault()</code>	339
D.5.3.2	Function: <code>handle_pte_fault()</code>	340
D.5.4	Demand Allocation	341
D.5.4.1	Function: <code>do_no_page()</code>	341
D.5.4.2	Function: <code>do_anonymous_page()</code>	344
D.5.5	Demand Paging	346
D.5.5.1	Function: <code>do_swap_page()</code>	346
D.5.5.2	Function: <code>can_share_swap_page()</code>	350
D.5.5.3	Function: <code>exclusive_swap_page()</code>	351
D.5.6	Copy On Write (COW) Pages	352
D.5.6.1	Function: <code>do_wp_page()</code>	352
D.6	Page-Related Disk I/O	355
D.6.1	Generic File Reading	355
D.6.1.1	Function: <code>generic_file_read()</code>	355
D.6.1.2	Function: <code>do_generic_file_read()</code>	358
D.6.1.3	Function: <code>generic_file_readahead()</code>	365
D.6.2	Generic File mmap()	369
D.6.2.1	Function: <code>generic_file_mmap()</code>	369
D.6.3	Generic File Truncation	370
D.6.3.1	Function: <code>vmtruncate()</code>	370
D.6.3.2	Function: <code>vmtruncate_list()</code>	372
D.6.3.3	Function: <code>zap_page_range()</code>	373
D.6.3.4	Function: <code>zap_pmd_range()</code>	375
D.6.3.5	Function: <code>zap_pte_range()</code>	376
D.6.3.6	Function: <code>truncate_inode_pages()</code>	377
D.6.3.7	Function: <code>truncate_list_pages()</code>	378
D.6.3.8	Function: <code>truncate_complete_page()</code>	380
D.6.3.9	Function: <code>do_flushpage()</code>	381
D.6.3.10	Function: <code>truncate_partial_page()</code>	381
D.6.4	Reading Pages for the Page Cache	382
D.6.4.1	Function: <code>filemap_nopage()</code>	382
D.6.4.2	Function: <code>page_cache_read()</code>	387
D.6.5	File Readahead for nopage()	388
D.6.5.1	Function: <code>nopage_sequential_readahead()</code>	388
D.6.5.2	Function: <code>read_cluster_nonblocking()</code>	390
D.6.6	Swap-Related Read-Ahead	391
D.6.6.1	Function: <code>swpin_readahead()</code>	391
D.6.6.2	Function: <code>valid_swaphandles()</code>	392

D.1 Process Memory Descriptors

Contents

D.1 Process Memory Descriptors	254
D.1.1 Initializing a Descriptor	254
D.1.2 Copying a Descriptor	254
D.1.2.1 Function: <code>copy_mm()</code>	254
D.1.2.2 Function: <code>mm_init()</code>	257
D.1.3 Allocating a Descriptor	257
D.1.3.1 Function: <code>allocate_mm()</code>	257
D.1.3.2 Function: <code>mm_alloc()</code>	258
D.1.4 Destroying a Descriptor	258
D.1.4.1 Function: <code>mmput()</code>	258
D.1.4.2 Function: <code>mmdrop()</code>	259
D.1.4.3 Function: <code>__mmdrop()</code>	260

This section covers the functions used to allocate, initialize, copy and destroy memory descriptors.

D.1.1 Initializing a Descriptor

The initial `mm_struct` in the system is called `init_mm` and is statically initialized at compile time using the macro `INIT_MM()`.

```

238 #define INIT_MM(name) \
239 { \
240     mm_rb:          RB_ROOT, \
241     pgd:             swapper_pg_dir, \
242     mm_users:        ATOMIC_INIT(2), \
243     mm_count:        ATOMIC_INIT(1), \
244     mmap_sem:        __RWSEM_INITIALIZER(name.mmap_sem), \
245     page_table_lock: SPIN_LOCK_UNLOCKED, \
246     mmlist:          LIST_HEAD_INIT(name.mmlist), \
247 }
```

After it is established, new `mm_structs` are copies of their parent `mm_struct` and are copied using `copy_mm()` with the process-specific fields initialized with `init_mm()`.

D.1.2 Copying a Descriptor

D.1.2.1 Function: `copy_mm()` (*kernel/fork.c*)

This function makes a copy of the `mm_struct` for the given task. This is only called from `do_fork()` after a new process has been created and needs its own `mm_struct`.

```

315 static int copy_mm(unsigned long clone_flags,
                      struct task_struct * tsk)
316 {
```



```

317     struct mm_struct * mm, *oldmm;
318     int retval;
319
320     tsk->min_flt = tsk->maj_flt = 0;
321     tsk->cmin_flt = tsk->cmaj_flt = 0;
322     tsk->nswap = tsk->cnsnap = 0;
323
324     tsk->mm = NULL;
325     tsk->active_mm = NULL;
326
327     /*
328      * Are we cloning a kernel thread?
329      * We need to steal an active VM for that..
330      */
331     oldmm = current->mm;
332     if (!oldmm)
333         return 0;
334
335     if (clone_flags & CLONE_VM) {
336         atomic_inc(&oldmm->mm_users);
337         mm = oldmm;
338         goto good_mm;
339     }
340

```

This block resets fields that are not inherited by a child `mm_struct` and finds an `mm` to copy from.

315 The parameters are the flags passed for clone and the task that is creating a copy of the `mm_struct`.

320-325 Initializes the `task_struct` fields related to memory management.

332 Borrows the `mm` of the current running process to copy from.

333 A kernel thread has no `mm`, so it can return immediately.

336-341 If the `CLONE_VM` flag is set, the child process is to share the `mm` with the parent process. This is required by users like `pthread`s. The `mm_users` field is incremented so that the `mm` is not destroyed prematurely. The `good_mm` label sets `tsk->mm` and `tsk->active_mm` and returns success.

```

342     retval = -ENOMEM;
343     mm = allocate_mm();
344     if (!mm)
345         goto fail_nomem;
346
347     /* Copy the current MM stuff.. */
348     memcpy(mm, oldmm, sizeof(*mm));

```

```

349     if (!mm_init(mm))
350         goto fail_nomem;
351
352     if (init_new_context(tsk,mm))
353         goto free_pt;
354
355     down_write(&oldmm->mmap_sem);
356     retval = dup_mmap(mm);
357     up_write(&oldmm->mmap_sem);
358

```

343 Allocates a new mm.

348-350 Copies the parent mm and initializes the process-specific mm fields with `init_mm()`.

352-353 Initializes the MMU context for architectures that do not automatically manage their MMU.

355-357 Calls `dup_mmap()`, which is responsible for copying all the VMA's regions in use by the parent process.

```

359     if (retval)
360         goto free_pt;
361
362     /*
363      * child gets a private LDT (if there was an LDT in the parent)
364      */
365     copy_segments(tsk, mm);
366
367 good_mm:
368     tsk->mm = mm;
369     tsk->active_mm = mm;
370     return 0;
371
372 free_pt:
373     mmput(mm);
374 fail_nomem:
375     return retval;
376 }

```

359 `dup_mmap()` returns 0 on success. If it failed, the label `free_pt` will call `mmput()`, which decrements the use count of the mm.

365 Copies the LDT for the new process based on the parent process.

368-370 Sets the new mm, `active_mm`, and return success.

D.1.2.2 Function: `mm_init()` (*kernel/fork.c*)

This function initializes process-specific mm fields.

```

230 static struct mm_struct * mm_init(struct mm_struct * mm)
231 {
232     atomic_set(&mm->mm_users, 1);
233     atomic_set(&mm->mm_count, 1);
234     init_rwsem(&mm->mmap_sem);
235     mm->page_table_lock = SPIN_LOCK_UNLOCKED;
236     mm->pgd = pgd_alloc(mm);
237     mm->def_flags = 0;
238     if (mm->pgd)
239         return mm;
240     free_mm(mm);
241     return NULL;
242 }
```

232 Sets the number of users to 1.

233 Sets the reference count of the mm to 1.

234 Initializes the semaphore protecting the VMA list.

235 Initializes the spinlock protecting write access to it.

236 Allocates a new PGD for the struct.

237 By default, pages used by the process are not locked in memory.

238 If a PGD exists, this returns the initialized struct.

240 If initialization failed, this deletes the `mm_struct` and returns.

D.1.3 Allocating a Descriptor

Two functions are provided that allocate an `mm_struct`. To be slightly confusing, they are essentially the same. `allocate_mm()` will allocate a `mm_struct` from the slab allocator. `mm_alloc()` will allocate the struct and then call the function `mm_init()` to initialize it.

D.1.3.1 Function: `allocate_mm()` (*kernel/fork.c*)

```

227 #define allocate_mm()    (kmem_cache_alloc(mm_cachep, SLAB_KERNEL))
```

227 Allocates an `mm_struct` from the slab allocator.

D.1.3.2 Function: `mm_alloc()` (*kernel/fork.c*)

```

248 struct mm_struct * mm_alloc(void)
249 {
250     struct mm_struct * mm;
251
252     mm = allocate_mm();
253     if (mm) {
254         memset(mm, 0, sizeof(*mm));
255         return mm_init(mm);
256     }
257     return NULL;
258 }

```

252 Allocates an `mm_struct` from the slab allocator.

254 Zeroes out all contents of the struct.

255 Performs basic initialization.

D.1.4 Destroying a Descriptor

A new user to an mm increments the usage count with a simple call:

```
atomic_inc(&mm->mm_users);
```

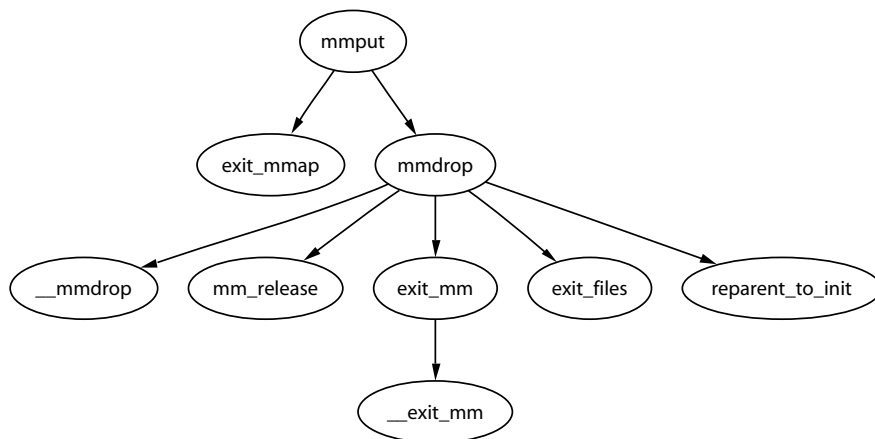
It is decremented with a call to `mmput()`. If the `mm_users` count reaches zero, all the mapped regions are deleted with `exit_mmap()`, and the pagetables are destroyed because there are no longer any users of the userspace portions. The `mm_count` count is decremented with `mmdrop()` because all the users of the pagetables and VMAs are counted as one `mm_struct` user. When `mm_count` reaches zero, the `mm_struct` will be destroyed.

D.1.4.1 Function: `mmput()` (*kernel/fork.c*)

```

276 void mmput(struct mm_struct *mm)
277 {
278     if (atomic_dec_and_lock(&mm->mm_users, &mmlist_lock)) {
279         extern struct mm_struct *swap_mm;
280         if (swap_mm == mm)
281             swap_mm = list_entry(mm->mmlist.next,
                                   struct mm_struct, mmlist);
282         list_del(&mm->mmlist);
283         mmlist_nr--;
284         spin_unlock(&mmlist_lock);
285         exit_mmap(mm);
286         mmdrop(mm);
287     }
288 }

```

Figure D.1. Call Graph: `mmpu()`

278 Atomically decrements the `mm_users` field while holding the `mmlist_lock` lock. It returns with the lock held if the count reaches zero.

279-286 If the usage count reaches zero, the `mm` and associated structures need to be removed.

279-281 The `swap_mm` is the last `mm` that was swapped out by the `vmscan` code. If the current process was the last `mm` swapped, this moves to the next entry in the list.

282 Removes this `mm` from the list.

283-284 Reduces the count of `mms` in the list and releases the `mmlist` lock.

285 Removes all associated mappings.

286 Deletes the `mm`.

D.1.4.2 Function: `mmdrop()` (*include/linux/sched.h*)

```

765 static inline void mmdrop(struct mm_struct * mm)
766 {
767     if (atomic_dec_and_test(&mm->mm_count))
768         __mmdrop(mm);
769 }

```

767 Atomically decrements the reference count. The reference count could be higher if the `mm` was used by lazy `tlb` switching tasks.

768 If the reference count reaches zero, this calls `__mmdrop()`.

D.1.4.3 Function: `__mmdrop()` (*kernel/fork.c*)

```
265 inline void __mmdrop(struct mm_struct *mm)
266 {
267     BUG_ON(mm == &init_mm);
268     pgd_free(mm->pgd);
269     destroy_context(mm);
270     free_mm(mm);
271 }
```

267 Makes sure the `init_mm` is not destroyed.

268 Deletes the PGD entry.

269 Deletes the LDT (Local Descriptor Table).

270 Calls `kmem_cache_free()` for the `mm`, freeing it with the slab allocator.

D.2 Creating Memory Regions

Contents

D.2 Creating Memory Regions	261
D.2.1 Creating a Memory Region	261
D.2.1.1 Function: <code>do_mmap()</code>	261
D.2.1.2 Function: <code>do_mmap_pgoff()</code>	262
D.2.2 Inserting a Memory Region	270
D.2.2.1 Function: <code>__insert_vm_struct()</code>	270
D.2.2.2 Function: <code>find_vma_prepare()</code>	271
D.2.2.3 Function: <code>vma_link()</code>	272
D.2.2.4 Function: <code>__vma_link()</code>	273
D.2.2.5 Function: <code>__vma_link_list()</code>	273
D.2.2.6 Function: <code>__vma_link_rb()</code>	274
D.2.2.7 Function: <code>__vma_link_file()</code>	274
D.2.3 Merging Contiguous Regions	275
D.2.3.1 Function: <code>vma_merge()</code>	275
D.2.3.2 Function: <code>can_vma_merge()</code>	277
D.2.4 Remapping and Moving a Memory Region	278
D.2.4.1 Function: <code>sys_mremap()</code>	278
D.2.4.2 Function: <code>do_mremap()</code>	278
D.2.4.3 Function: <code>move_vma()</code>	284
D.2.4.4 Function: <code>make_pages_present()</code>	288
D.2.4.5 Function: <code>get_user_pages()</code>	289
D.2.4.6 Function: <code>move_page_tables()</code>	292
D.2.4.7 Function: <code>move_one_page()</code>	293
D.2.4.8 Function: <code>get_one_pte()</code>	294
D.2.4.9 Function: <code>alloc_one_pte()</code>	295
D.2.4.10 Function: <code>copy_one_pte()</code>	295
D.2.5 Deleting a Memory Region	296
D.2.5.1 Function: <code>do_munmap()</code>	296
D.2.5.2 Function: <code>unmap_fixup()</code>	300
D.2.6 Deleting All Memory Regions	304
D.2.6.1 Function: <code>exit_mmap()</code>	304
D.2.6.2 Function: <code>clear_page_tables()</code>	306
D.2.6.3 Function: <code>free_one_pgdpd()</code>	306
D.2.6.4 Function: <code>free_one_pmd()</code>	307

This large section deals with the creation, deletion and manipulation of memory regions.

D.2.1 Creating a Memory Region

The main call graph for creating a memory region is shown in Figure 4.3.

D.2.1.1 Function: `do_mmap()` (*include/linux/mm.h*)

This is a very simple wrapper function around `do_mmap_pgoff()`, which performs most of the work.

```
557 static inline unsigned long do_mmap(struct file *file,
```

```

        unsigned long addr,
558        unsigned long len, unsigned long prot,
559        unsigned long flag, unsigned long offset)
560 {
561     unsigned long ret = -EINVAL;
562     if ((offset + PAGE_ALIGN(len)) < offset)
563         goto out;
564     if (!(offset & ~PAGE_MASK))
565         ret = do_mmap_pgoff(file, addr, len, prot, flag,
                           offset >> PAGE_SHIFT);
566 out:
567     return ret;
568 }

```

561 By default, this returns `-EINVAL`.

562-563 Makes sure that the size of the region will not overflow the total size of the address space.

564-565 Page aligns the `offset` and calls `do_mmap_pgoff()` to map the region.

D.2.1.2 Function: `do_mmap_pgoff()` (*mm/mmap.c*)

This function is very large, so it is broken up into a number of sections. Broadly speaking the sections are the following:

- Sanity check the parameters.
- Find a free linear address space large enough for the memory mapping. If a filesystem or device-specific `get_unmapped_area()` function is provided, it will be used. Otherwise, `arch_get_unmapped_area()` is called.
- Calculate the VM flags and check them against the file access permissions.
- If an old area exists where the mapping is to take place, fix it so it is suitable for the new mapping.
- Allocate a `vm_area_struct` from the slab allocator and fill in its entries.
- Link in the new VMA.
- Call the filesystem or device-specific `mmap()` function.
- Update statistics and exit.

```

393 unsigned long do_mmap_pgoff(struct file * file,
        unsigned long addr,
        unsigned long len, unsigned long prot,
394        unsigned long flags, unsigned long pgoff)
395 {
396     struct mm_struct * mm = current->mm;

```



```

397     struct vm_area_struct * vma, * prev;
398     unsigned int vm_flags;
399     int correct_wcount = 0;
400     int error;
401     rb_node_t ** rb_link, * rb_parent;
402
403     if (file && (!file->f_op || !file->f_op->mmap))
404         return -ENODEV;
405
406     if (!len)
407         return addr;
408
409     len = PAGE_ALIGN(len);
410
411     if (len > TASK_SIZE || len == 0)
412         return -EINVAL;
413
414     /* offset overflow? */
415     if ((pgoff + (len >> PAGE_SHIFT)) < pgoff)
416         return -EINVAL;
417
418     /* Too many mappings? */
419     if (mm->map_count > max_map_count)
420         return -ENOMEM;
421

```

393 The parameters that correspond directly to the parameters of the `mmap` system call are the following:

- **file** The struct file to `mmap` if this is a file-backed mapping
- **addr** The requested address to map
- **len** The length in bytes to `mmap`
- **prot** The permissions on the area
- **flags** The flags for the mapping
- **pgoff** The offset within the file to begin the `mmap` at

403-404 If a file or device is mapped, this makes sure a filesystem or device-specific `mmap` function is provided. For most filesystems, this will call `generic_file_mmap()` (See Section D.6.2.1).

406-407 Makes sure a zero length `mmap()` is not requested.

409 Ensures that the mapping is confined to the userspace portion of the address space. On the x86, kernel space begins at `PAGE_OFFSET(3GiB)`.

415-416 Ensures the mapping will not overflow the end of the largest possible file size.

419-420 Only `max_map_count` number of mappings are allowed. By default, this value is `DEFAULT_MAX_MAP_COUNT` or 65,536 mappings.

```

422     /* Obtain the address to map to. we verify (or select) it and
423     * ensure that it represents a valid section of the address space.
424     */
425     addr = get_unmapped_area(file, addr, len, pgoff, flags);
426     if (addr & ~PAGE_MASK)
427         return addr;
428
```

425 After basic sanity checks, this function will call the device- or file-specific `get_unmapped_area()` function. If a device-specific one is unavailable, `arch_get_unmapped_area()` is called. This function is discussed in Section D.3.2.2.

```

429     /* Do simple checking here so the lower-level routines won't
430     * have to. we assume access permissions have been handled by
431     * the open of the memory object, so we don't do any here.
432     */
433     vm_flags = calc_vm_flags(prot, flags) | mm->def_flags
434                | VM_MAYREAD | VM_MAYWRITE | VM_MAYEXEC;
435
436     /* mlock MCL_FUTURE? */
437     if (vm_flags & VM_LOCKED) {
438         unsigned long locked = mm->locked_vm << PAGE_SHIFT;
439         locked += len;
440         if (locked > current->rlim[RLIMIT_MEMLOCK].rlim_cur)
441             return -EAGAIN;
442     }

```

433 `calc_vm_flags()` translates the `prot` and `flags` from userspace and translates them to their VM_ equivalents.

436-440 Checks if it has been requested that all future mappings be locked in memory. If yes, it makes sure the process isn't locking more memory than it is allowed to. If it is, it returns `-EAGAIN`.

```

443     if (file) {
444         switch (flags & MAP_TYPE) {
445             case MAP_SHARED:
446                 if ((prot & PROT_WRITE) &&
447                     !(file->f_mode & FMODE_WRITE))
448                     return -EACCES;
449
450                 /* Make sure we don't allow writing to

```

```

        an append-only file.. */
450     if (IS_APPEND(file->f_dentry->d_inode) &&
        (file->f_mode & FMODE_WRITE))
451         return -EACCES;
452
453     /* make sure there are no mandatory
        locks on the file. */
454     if (locks_verify_locked(file->f_dentry->d_inode))
455         return -EAGAIN;
456
457     vm_flags |= VM_SHARED | VM_MAYSHARE;
458     if (!(file->f_mode & FMODE_WRITE))
459         vm_flags &= ~(VM_MAYWRITE | VM_SHARED);
460
461     /* fall through */
462     case MAP_PRIVATE:
463         if (!(file->f_mode & FMODE_READ))
464             return -EACCES;
465         break;
466
467     default:
468         return -EINVAL;
469 }

```

443-469 If a file is memory mapped, this checks the file's access permissions.

446-447 If write access is requested, this makes sure the file is opened for write.

450-451 Similarly, if the file is opened for append, this makes sure it cannot be written to. The `prot` field is not checked because the `prot` field applies only to the mapping whereas the opened file needs to be checked.

453 If the file is mandatory locked, this returns `-EAGAIN` so the caller will try a second type.

457-459 Fixes up the flags to be consistent with the file flags.

463-464 Makes sure the file can be read before mmappping it.

```

470     } else {
471         vm_flags |= VM_SHARED | VM_MAYSHARE;
472         switch (flags & MAP_TYPE) {
473             default:
474                 return -EINVAL;
475             case MAP_PRIVATE:
476                 vm_flags &= ~(VM_SHARED | VM_MAYSHARE);
477                 /* fall through */
478             case MAP_SHARED:

```

```

479         break;
480     }
481 }

```

471-481 If the file is mapped for anonymous use, this fixes up the flags if the requested mapping is `MAP_PRIVATE` to make sure the flags are consistent.

```

483     /* Clear old maps */
484 munmap_back:
485     vma = find_vma_prepare(mm, addr, &prev, &rb_link, &rb_parent);
486     if (vma && vma->vm_start < addr + len) {
487         if (do_munmap(mm, addr, len))
488             return -ENOMEM;
489         goto munmap_back;
490     }

```

485 `find_vma_prepare()` (See Section D.2.2.2) steps through the RB tree for the VMA corresponding to a given address.

486-488 If a VMA was found and it is part of the new mmaping, this removes the old mapping because the new one will cover both.

```

491
492     /* Check against address space limit. */
493     if ((mm->total_vm << PAGE_SHIFT) + len
494         > current->rlim[RLIMIT_AS].rlim_cur)
495         return -ENOMEM;
496
497     /* Private writable mapping? Check memory availability.. */
498     if ((vm_flags & (VM_SHARED | VM_WRITE)) == VM_WRITE &&
499         !(flags & MAP_NORESERVE) &&
500         !vm_enough_memory(len >> PAGE_SHIFT))
501         return -ENOMEM;
502
503     /* Can we just expand an old anonymous mapping? */
504     if (!file && !(vm_flags & VM_SHARED) && rb_parent)
505         if (vma_merge(mm, prev, rb_parent,
506                     addr, addr + len, vm_flags))
507             goto out;

```

493-495 Ensures the new mapping will not exceed the total VM that a process is allowed to have. It is unclear why this check is not made earlier.

498-501 If the caller does not specifically request that free space is not checked with `MAP_NORESERVE` and it is a private mapping, this ensures enough memory is available to satisfy the mapping under current conditions.

504-506 If two adjacent memory mappings are anonymous and can be treated as one, this expands the old mapping rather than creating a new one.

```

508     /* Determine the object being mapped and call the appropriate
509     * specific mapper. the address has already been validated,
510     * but not unmapped, but the maps are removed from the list.
511     */
512     vma = kmem_cache_alloc(vm_area_cachep, SLAB_KERNEL);
513     if (!vma)
514         return -ENOMEM;
515
516     vma->vm_mm = mm;
517     vma->vm_start = addr;
518     vma->vm_end = addr + len;
519     vma->vm_flags = vm_flags;
520     vma->vm_page_prot = protection_map[vm_flags & 0x0f];
521     vma->vm_ops = NULL;
522     vma->vm_pgoff = pgoff;
523     vma->vm_file = NULL;
524     vma->vm_private_data = NULL;
525     vma->vm_raend = 0;

```

512 Allocates a `vm_area_struct` from the slab allocator.

516-525 Fills in the basic `vm_area_struct` fields.

```

527     if (file) {
528         error = -EINVAL;
529         if (vm_flags & (VM_GROWSDOWN|VM_GROWSUP))
530             goto free_vma;
531         if (vm_flags & VM_DENYWRITE) {
532             error = deny_write_access(file);
533             if (error)
534                 goto free_vma;
535             correct_wcount = 1;
536         }
537         vma->vm_file = file;
538         get_file(file);
539         error = file->f_op->mmap(file, vma);
540         if (error)
541             goto unmap_and_free_vma;

```

527-541 Fills in the file-related fields if this file has been mapped.

529-530 These are both invalid flags for a file mapping, so it frees the `vm_area_struct` and returns.

531-536 This flag is cleared by the system call `mmap()`, but is still cleared for kernel modules that call this function directly. Historically, `-ETXTBUSY` was returned to the calling process if the underlying file was written to.

537 Fills in the `vm_file` field.

538 Increments the file usage count.

539 Calls the filesystem or device-specific `mmap()` function. In many filesystem cases, this will call `generic_file_mmap()` (See Section D.6.2.1).

540-541 If an error is called, this goes to `unmap_and_free_vma` to clean up and return the error.

```

542     } else if (flags & MAP_SHARED) {
543         error = shmem_zero_setup(vma);
544         if (error)
545             goto free_vma;
546     }
547
```

543 If this is an anonymous shared mapping, the region is created and set up by `shmem_zero_setup()` (See Section L.7.1). Anonymous shared pages are backed by a virtual tmpfs filesystem so that they can be synchronized properly with swap. The writeback function is `shmem_writepage()` (See Section L.6.1).

```

548     /* Can addr have changed??
549     *
550     * Answer: Yes, several device drivers can do it in their
551     *       f_op->mmap method. -DaveM
552     */
553     if (addr != vma->vm_start) {
554         /*
555          * It is a bit too late to pretend changing the virtual
556          * area of the mapping, we just corrupted userspace
557          * in the do_munmap, so FIXME (not in 2.4 to avoid
558          * breaking the driver API).
559          */
560         struct vm_area_struct * stale_vma;
561         /* Since addr changed, we rely on the mmap op to prevent
562          * collisions with existing vmas and just use
563          * find_vma_prepare to update the tree pointers.
564          */
565         addr = vma->vm_start;
566         stale_vma = find_vma_prepare(mm, addr, &prev,
567                                     &rb_link, &rb_parent);
568         /*
569          * Make sure the lowlevel driver did its job right.
570          */

```

```

571         if (unlikely(stale_vma && stale_vma->vm_start <
                        vma->vm_end)) {
572             printk(KERN_ERR "buggy mmap operation: [<%p>]\n",
573                    file ? file->f_op->mmap : NULL);
574             BUG();
575         }
576     }
577
578     vma_link(mm, vma, prev, rb_link, rb_parent);
579     if (correct_wcount)
580         atomic_inc(&file->f_dentry->d_inode->i_writecount);
581

```

553-576 If the address has changed, it means the device-specific mmap operation moved the VMA address to somewhere else. The function `find_vma_prepare()` (See Section D.2.2.2) is used to find where the VMA was moved to.

578 Links in the new `vm_area_struct`.

579-580 Updates the file write count.

```

582 out:
583     mm->total_vm += len >> PAGE_SHIFT;
584     if (vm_flags & VM_LOCKED) {
585         mm->locked_vm += len >> PAGE_SHIFT;
586         make_pages_present(addr, addr + len);
587     }
588     return addr;
589
590 unmap_and_free_vma:
591     if (correct_wcount)
592         atomic_inc(&file->f_dentry->d_inode->i_writecount);
593     vma->vm_file = NULL;
594     fput(file);
595
596     /* Undo any partial mapping done by a device driver. */
597     zap_page_range(mm,
                    vma->vm_start,
                    vma->vm_end - vma->vm_start);
598 free_vma:
599     kmem_cache_free(vm_area_cachep, vma);
600     return error;
601 }

```

583-588 Updates statistics for the process `mm_struct` and returns the new address.

590-597 This is reached if the file has been partially mapped before failing. The write statistics are updated, and then all user pages are removed with `zap_page_range()`.

598-600 This `goto` is used if the mapping failed immediately after the `vm_area_struct` is created. It is freed back to the slab allocator before the error is returned.

D.2.2 Inserting a Memory Region

The call graph for `insert_vm_struct()` is shown in Figure 4.5.

D.2.2.1 Function: `__insert_vm_struct()` (*mm/mmap.c*)

This is the top-level function for inserting a new `vma` into an address space. There is a second function like it called simply `insert_vm_struct()` that is not described in detail here because the only difference is the one line of code increasing the `map_count`.

```

1174 void __insert_vm_struct(struct mm_struct * mm,
                           struct vm_area_struct * vma)
1175 {
1176     struct vm_area_struct * __vma, * prev;
1177     rb_node_t ** rb_link, * rb_parent;
1178
1179     __vma = find_vma_prepare(mm, vma->vm_start, &prev,
                             &rb_link, &rb_parent);
1180     if (__vma && __vma->vm_start < vma->vm_end)
1181         BUG();
1182     __vma_link(mm, vma, prev, rb_link, rb_parent);
1183     mm->map_count++;
1184     validate_mm(mm);
1185 }
```

1174 The arguments are the `mm_struct` that represents the linear address space and the `vm_area_struct` that is to be inserted.

1179 `find_vma_prepare()` (See Section D.2.2.2) locates where the new VMA can be inserted. It will be inserted between `prev` and `__vma`, and the required nodes for the red-black tree are also returned.

1180-1181 This is a check to make sure the returned VMA is invalid. It is virtually impossible for this condition to occur without manually inserting bogus VMAs into the address space.

1182 This function does the actual work of linking the VMA struct into the linear linked list and the red-black tree.

1183 Increases the `map_count` to show a new mapping has been added. This line is not present in `insert_vm_struct()`.

1184 `validate_mm()` is a debugging macro for red-black trees. If `DEBUG_MM_RB` is set, the linear list of VMAs and the tree will be traversed to make sure it is

valid. The tree traversal is a recursive function, so it is very important that it is used only if really necessary because a large number of mappings could cause a stack overflow. If it is not set, `validate_mm()` does nothing at all.

D.2.2.2 Function: `find_vma_prepare()` (*mm/mmap.c*)

This is responsible for finding the correct places to insert a VMA at the supplied address. It returns a number of pieces of information through the actual return and the function arguments. The forward VMA to link to is returned with `return`. `pprev` is the previous node, which is required because the list is a singly linked list. `rb_link` and `rb_parent` are the parent and leaf node that the new VMA will be inserted between.

```

246 static struct vm_area_struct * find_vma_prepare(
                struct mm_struct * mm,
                unsigned long addr,
247                struct vm_area_struct ** pprev,
248                rb_node_t *** rb_link,
                rb_node_t ** rb_parent)
249 {
250     struct vm_area_struct * vma;
251     rb_node_t ** __rb_link, * __rb_parent, * rb_prev;
252
253     __rb_link = &mm->mm_rb.rb_node;
254     rb_prev = __rb_parent = NULL;
255     vma = NULL;
256
257     while (*__rb_link) {
258         struct vm_area_struct *vma_tmp;
259
260         __rb_parent = *__rb_link;
261         vma_tmp = rb_entry(__rb_parent,
                struct vm_area_struct, vm_rb);
262
263         if (vma_tmp->vm_end > addr) {
264             vma = vma_tmp;
265             if (vma_tmp->vm_start <= addr)
266                 return vma;
267             __rb_link = &__rb_parent->rb_left;
268         } else {
269             rb_prev = __rb_parent;
270             __rb_link = &__rb_parent->rb_right;
271         }
272     }
273
274     *pprev = NULL;
275     if (rb_prev)

```

```

276     *pprev = rb_entry(rb_prev, struct vm_area_struct, vm_rb);
277     *rb_link = __rb_link;
278     *rb_parent = __rb_parent;
279     return vma;
280 }

```

246 The function arguments are described previously.

253-255 Initializes the search.

263-272 This is a similar tree walk to what was described for `find_vma()`. The only real difference is the nodes last traversed are remembered with the `__rb_link` and `__rb_parent` variables.

275-276 Gets the back linking VMA through the red-black tree.

279 Returns the forward linking VMA.

D.2.2.3 Function: `vma_link()` (*mm/mmap.c*)

This is the top-level function for linking a VMA into the proper lists. It is responsible for acquiring the necessary locks to make a safe insertion.

```

337 static inline void vma_link(struct mm_struct * mm,
                             struct vm_area_struct * vma,
                             struct vm_area_struct * prev,
                             rb_node_t ** rb_link, rb_node_t * rb_parent)
338 {
339 {
340     lock_vma_mappings(vma);
341     spin_lock(&mm->page_table_lock);
342     __vma_link(mm, vma, prev, rb_link, rb_parent);
343     spin_unlock(&mm->page_table_lock);
344     unlock_vma_mappings(vma);
345
346     mm->map_count++;
347     validate_mm(mm);
348 }

```

337 `mm` is the address space that the VMA is to be inserted into. `prev` is the backward-linked VMA for the linear-linked-list of VMAs. `rb_link` and `rb_parent` are the nodes required to make the rb insertion.

340 This function acquires the spinlock that protects the `address_space` representing the file that is memory mapped.

341 Acquires the pagetable lock, which protects the whole `mm_struct`.

342 Inserts the VMA.

343 Frees the lock protecting the `mm_struct`.

345 Unlocks the `address_space` for the file.

346 Increases the number of mappings in this mm.

347 If `DEBUG_MM_RB` is set, the RB trees and linked lists will be checked to make sure they are still valid.

D.2.2.4 Function: `__vma_link()` (*mm/mmap.c*)

This simply calls three helper functions that are responsible for linking the VMA into the three linked lists that link VMAs together.

```
329 static void __vma_link(struct mm_struct * mm,
                        struct vm_area_struct * vma,
                        struct vm_area_struct * prev,
330                        rb_node_t ** rb_link, rb_node_t * rb_parent)
331 {
332     __vma_link_list(mm, vma, prev, rb_parent);
333     __vma_link_rb(mm, vma, rb_link, rb_parent);
334     __vma_link_file(vma);
335 }
```

332 Links the VMA into the linear-linked lists of VMAs in this mm through the `vm_next` field.

333 Links the VMA into the red-black tree of VMAs in this mm that has its root stored in the `vm_rb` field.

334 Links the VMA into the shared mapping VMA links. Memory mapped files are linked together over potentially many mms by this function using the `vm_next_share` and `vm_pprev_share` fields.

D.2.2.5 Function: `__vma_link_list()` (*mm/mmap.c*)

```
282 static inline void __vma_link_list(struct mm_struct * mm,
                                    struct vm_area_struct * vma,
                                    struct vm_area_struct * prev,
283                                    rb_node_t * rb_parent)
284 {
285     if (prev) {
286         vma->vm_next = prev->vm_next;
287         prev->vm_next = vma;
288     } else {
289         mm->mmap = vma;
290         if (rb_parent)
291             vma->vm_next = rb_entry(rb_parent,
                                    struct vm_area_struct,
                                    vm_rb);
```

```

292         else
293             vma->vm_next = NULL;
294     }
295 }

```

285 If `prev` is not null, the VMA is simply inserted into the list.

289 If not, this is the first mapping, and the first element of the list has to be stored in the `mm_struct`.

290 The VMA is stored as the parent node.

D.2.2.6 Function: `__vma_link_rb()` (*mm/mmap.c*)

The principal workings of this function are stored within `<linux/rbtree.h>` and will not be discussed in detail in this book.

```

297 static inline void __vma_link_rb(struct mm_struct * mm,
                                   struct vm_area_struct * vma,
298                                   rb_node_t ** rb_link,
                                   rb_node_t * rb_parent)
299 {
300     rb_link_node(&vma->vm_rb, rb_parent, rb_link);
301     rb_insert_color(&vma->vm_rb, &mm->mm_rb);
302 }

```

D.2.2.7 Function: `__vma_link_file()` (*mm/mmap.c*)

This function links the VMA into a linked list of shared file mappings.

```

304 static inline void __vma_link_file(struct vm_area_struct * vma)
305 {
306     struct file * file;
307
308     file = vma->vm_file;
309     if (file) {
310         struct inode * inode = file->f_dentry->d_inode;
311         struct address_space *mapping = inode->i_mapping;
312         struct vm_area_struct **head;
313
314         if (vma->vm_flags & VM_DENYWRITE)
315             atomic_dec(&inode->i_writecount);
316
317         head = &mapping->i_mmap;
318         if (vma->vm_flags & VM_SHARED)
319             head = &mapping->i_mmap_shared;
320
321         /* insert vma into inode's share list */
322         if((vma->vm_next_share = *head) != NULL)

```

```

323         (*head)->vm_pprev_share = &vma->vm_next_share;
324         *head = vma;
325         vma->vm_pprev_share = head;
326     }
327 }

```

309 Checks to see if this VMA has a shared file mapping. If it does not, this function has nothing more to do.

310-312 Extracts the relevant information about the mapping from the VMA.

314-315 If this mapping is not allowed to write even if the permissions are ok for writing, decrement the `i_writecount` field. A negative value to this field indicates that the file is memory mapped and may not be written to. Efforts to open the file for writing will now fail.

317-319 Checks to make sure this is a shared mapping.

322-325 Inserts the VMA into the shared mapping linked list.

D.2.3 Merging Contiguous Regions

D.2.3.1 Function: `vma_merge()` (*mm/mmap.c*)

This function checks to see if a region pointed to by `prev` may be expanded forward to cover the area from `addr` to `end` instead of allocating a new VMA. If it cannot, the VMA ahead is checked to see whether it can be expanded backward instead.

```

350 static int vma_merge(struct mm_struct * mm,
                       struct vm_area_struct * prev,
351                       rb_node_t * rb_parent,
                       unsigned long addr, unsigned long end,
                       unsigned long vm_flags)
352 {
353     spinlock_t * lock = &mm->page_table_lock;
354     if (!prev) {
355         prev = rb_entry(rb_parent, struct vm_area_struct, vm_rb);
356         goto merge_next;
357     }

```

350 The parameters are as follows:

- **mm** The mm the VMAs belong to
- **prev** The VMA before the address we are interested in
- **rb_parent** The parent RB node as returned by `find_vma_prepare()`
- **addr** The starting address of the region to be merged
- **end** The end of the region to be merged

- **vm_flags** The permission flags of the region to be merged

353 This is the lock to the mm.

354-357 If **prev** is not passed it, it is taken to mean that the VMA being tested for merging is in front of the region from **addr** to **end**. The entry for that VMA is extracted from the **rb_parent**.

```

358     if (prev->vm_end == addr && can_vma_merge(prev, vm_flags)) {
359         struct vm_area_struct * next;
360
361         spin_lock(lock);
362         prev->vm_end = end;
363         next = prev->vm_next;
364         if (next && prev->vm_end == next->vm_start &&
365             can_vma_merge(next, vm_flags)) {
366             prev->vm_end = next->vm_end;
367             __vma_unlink(mm, next, prev);
368             spin_unlock(lock);
369
370             mm->map_count--;
371             kmem_cache_free(vm_area_cachep, next);
372             return 1;
373         }
374         spin_unlock(lock);
375         return 1;
376     }
377     prev = prev->vm_next;
378     if (prev) {
379 merge_next:
380         if (!can_vma_merge(prev, vm_flags))
381             return 0;
382         if (end == prev->vm_start) {
383             spin_lock(lock);
384             prev->vm_start = addr;
385             spin_unlock(lock);
386             return 1;
387         }
388     }
389
390     return 0;
391 }
```

358-375 Checks to see if the region pointed to by **prev** may be expanded to cover the current region.

358 The function `can_vma_merge()` checks the permissions of `prev` with those in `vm_flags` and that the VMA has no file mappings (i.e., it is anonymous). If it is true, the area at `prev` may be expanded.

361 Locks the mm.

362 Expands the end of the VMA region (`vm_end`) to the end of the new mapping (`end`).

363 `next` is now the VMA in front of the newly expanded VMA.

364 Checks if the expanded region can be merged with the VMA in front of it.

365 If it can, this continues to expand the region to cover the next VMA.

366 Because a VMA has been merged, one region is now defunct and may be unlinked.

367 No further adjustments are made to the mm struct, so the lock is released.

369 There is one less mapped region to reduce the `map_count`.

370 Deletes the struct describing the merged VMA.

371 Returns success.

377 If this line is reached, it means the region pointed to by `prev` could not be expanded forward, so a check is made to see if the region ahead can be merged backward instead.

382-388 The same idea as the previous block except instead of adjusted `vm_end` to cover `end`, `vm_start` is expanded to cover `addr`.

D.2.3.2 Function: `can_vma_merge()` (*include/linux/mm.h*)

This trivial function checks to see if the permissions of the supplied VMA match the permissions in `vm_flags`.

```
582 static inline int can_vma_merge(struct vm_area_struct * vma,
                                unsigned long vm_flags)
583 {
584     if (!vma->vm_file && vma->vm_flags == vm_flags)
585         return 1;
586     else
587         return 0;
588 }
```

584 Self-explanatory. It returns true if there is no file/device mapping (i.e., it is anonymous) and if the VMA flags for both regions match.

D.2.4 Remapping and Moving a Memory Region

D.2.4.1 Function: `sys_mremap()` (*mm/mremap.c*)

The call graph for this function is shown in Figure 4.6. This is the system service call to remap a memory region.

```
347 asmlinkage unsigned long sys_mremap(unsigned long addr,  
348     unsigned long old_len, unsigned long new_len,  
349     unsigned long flags, unsigned long new_addr)  
350 {  
351     unsigned long ret;  
352  
353     down_write(&current->mm->mmap_sem);  
354     ret = do_mremap(addr, old_len, new_len, flags, new_addr);  
355     up_write(&current->mm->mmap_sem);  
356     return ret;  
357 }
```

347-349 The parameters are the same as those described in the `mremap()` man page.

353 Acquires the mm semaphore.

354 `do_mremap()` (See Section D.2.4.2) is the top-level function for remapping a region.

355 Releases the mm semaphore.

356 Returns the status of the remapping.

D.2.4.2 Function: `do_mremap()` (*mm/mremap.c*)

This function does most of the actual work required to remap, resize and move a memory region. It is quite long, but can be broken up into distinct parts, which will be dealt with separately here. The tasks are, broadly speaking, the following:

- Check usage flags and page align lengths.
- Handle the condition where `MAP_FIXED` has set and the region has been moved to a new location.
- If a region is shrinking, allow it to happen unconditionally.
- If the region is growing or moving, perform a number of checks in advance to make sure the move is allowed and safe.
- Handle the case where the region has been expanded and cannot be moved.
- Finally, handle the case where the region has to be resized and moved.


```

219 unsigned long do_mremap(unsigned long addr,
220     unsigned long old_len, unsigned long new_len,
221     unsigned long flags, unsigned long new_addr)
222 {
223     struct vm_area_struct *vma;
224     unsigned long ret = -EINVAL;
225
226     if (flags & ~(MREMAP_FIXED | MREMAP_MAYMOVE))
227         goto out;
228
229     if (addr & ~PAGE_MASK)
230         goto out;
231
232     old_len = PAGE_ALIGN(old_len);
233     new_len = PAGE_ALIGN(new_len);
234

```

219 The parameters of the function are the following:

- **addr** is the old starting address.
- **old_len** is the old region length.
- **new_len** is the new region length.
- **flags** is the option flags passed. If **MREMAP_MAYMOVE** is specified, it means that the region is allowed to move if there is not enough linear address space at the current space. If **MREMAP_FIXED** is specified, it means that the whole region is to move to the specified **new_addr** with the new length. The area from **new_addr** to **new_addr+new_len** will be unmapped with **do_munmap()**.
- **new_addr** is the address of the new region if it is moved.

224 At this point, the default return is **-EINVAL** for invalid arguments.

226-227 Makes sure flags other than the two allowed flags are not used.

229-230 The address passed in must be page aligned.

232-233 Page-aligns the passed region lengths.

```

236     if (flags & MREMAP_FIXED) {
237         if (new_addr & ~PAGE_MASK)
238             goto out;
239         if (!(flags & MREMAP_MAYMOVE))
240             goto out;
241
242         if (new_len > TASK_SIZE || new_addr > TASK_SIZE - new_len)
243             goto out;
244

```

```

245      /* Check if the location we're moving into overlaps the
246      * old location at all, and fail if it does.
247      */
248      if ((new_addr <= addr) && (new_addr+new_len) > addr)
249          goto out;
250
251      if ((addr <= new_addr) && (addr+old_len) > new_addr)
252          goto out;
253
254      do_munmap(current->mm, new_addr, new_len);
255  }

```

This block handles the condition where the region location is fixed and must be fully moved. It ensures the area being moved to is safe and definitely unmapped.

236 `MREMAP_FIXED` is the flag that indicates the location is fixed.

237-238 The specified `new_addr` must be page-aligned.

239-240 If `MREMAP_FIXED` is specified, the `MAYMOVE` flag must be used as well.

242-243 Makes sure the resized region does not exceed `TASK_SIZE`.

248-249 Just as the comments indicate, the two regions being used for the move may not overlap.

254 Unmaps the region that is about to be used. It is presumed the caller ensures that the region is not in use for anything important.

```

261      ret = addr;
262      if (old_len >= new_len) {
263          do_munmap(current->mm, addr+new_len, old_len - new_len);
264          if (!(flags & MREMAP_FIXED) || (new_addr == addr))
265              goto out;
266      }

```

261 At this point, the address of the resized region is the return value.

262 If the old length is larger than the new length, the region is shrinking.

263 Unmaps the unused region.

264-265 If the region is not to be moved, either because `MREMAP_FIXED` is not used or the new address matches the old address, goto out, which will return the address.

```

271      ret = -EFAULT;
272      vma = find_vma(current->mm, addr);
273      if (!vma || vma->vm_start > addr)
274          goto out;

```

```

275     /* We can't remap across vm area boundaries */
276     if (old_len > vma->vm_end - addr)
277         goto out;
278     if (vma->vm_flags & VM_DONTEXPAND) {
279         if (new_len > old_len)
280             goto out;
281     }
282     if (vma->vm_flags & VM_LOCKED) {
283         unsigned long locked = current->mm->locked_vm << PAGE_SHIFT;
284         locked += new_len - old_len;
285         ret = -EAGAIN;
286         if (locked > current->rlim[RLIMIT_MEMLOCK].rlim_cur)
287             goto out;
288     }
289     ret = -ENOMEM;
290     if ((current->mm->total_vm << PAGE_SHIFT) + (new_len - old_len)
291         > current->rlim[RLIMIT_AS].rlim_cur)
292         goto out;
293     /* Private writable mapping? Check memory availability.. */
294     if ((vma->vm_flags & (VM_SHARED | VM_WRITE)) == VM_WRITE &&
295         !(flags & MAP_NORESERVE) &&
296         !vm_enough_memory((new_len - old_len) >> PAGE_SHIFT))
297         goto out;

```

This block does a number of checks to make sure it is safe to grow or move the region.

271 At this point, the default action is to return `-EFAULT`, which causes a segmentation fault because the ranges of memory being used are invalid.

272 Finds the VMA responsible for the requested address.

273 If the returned VMA is not responsible for this address, an invalid address was used to return a fault.

276-277 If the `old_len` passed in exceeds the length of the VMA, it means the user is trying to remap multiple regions, which is not allowed.

278-281 If the VMA has been explicitly marked as nonresizable, this raises a fault.

282-283 If the pages for this VMA must be locked in memory, this recalculates the number of locked pages that will be kept in memory. If the number of pages exceeds the ulimit set for this resource, this returns `EAGAIN`, which indicated to the caller that the region is locked and cannot be resized.

289 The default return at this point is to indicate there is not enough memory.

290-292 Ensures that the users will not exceed their allowed allocation of memory.

294-297 Ensures that there is enough memory to satisfy the request after the resizing with `vm_enough_memory()` (See Section M.1.1).

```

302     if (old_len == vma->vm_end - addr &&
303         !((flags & MREMAP_FIXED) && (addr != new_addr)) &&
304         (old_len != new_len || !(flags & MREMAP_MAYMOVE))) {
305         unsigned long max_addr = TASK_SIZE;
306         if (vma->vm_next)
307             max_addr = vma->vm_next->vm_start;
308         /* can we just expand the current mapping? */
309         if (max_addr - addr >= new_len) {
310             int pages = (new_len - old_len) >> PAGE_SHIFT;
311             spin_lock(&vma->vm_mm->page_table_lock);
312             vma->vm_end = addr + new_len;
313             spin_unlock(&vma->vm_mm->page_table_lock);
314             current->mm->total_vm += pages;
315             if (vma->vm_flags & VM_LOCKED) {
316                 current->mm->locked_vm += pages;
317                 make_pages_present(addr + old_len,
318                                   addr + new_len);
319             }
320             ret = addr;
321             goto out;
322         }
323     }

```

This block handles the case where the region is being expanded and cannot be moved.

302 If it is the full region that is being remapped and ...

303 The region is definitely not being moved and ...

304 The region is being expanded and cannot be moved, then ...

305 Sets the maximum address that can be used to `TASK_SIZE`, which is 3GiB on an x86.

306-307 If there is another region, this sets the max address to be the start of the next region.

309-322 Only allows the expansion if the newly sized region does not overlap with the next VMA.

310 Calculates the number of extra pages that will be required.

311 Locks the mm spinlock.

312 Expands the VMA.

313 Frees the mm spinlock.

314 Updates the statistics for the mm.

315-319 If the pages for this region are locked in memory, this makes them present now.

320-321 Returns the address of the resized region.

```

329     ret = -ENOMEM;
330     if (flags & MREMAP_MAYMOVE) {
331         if (!(flags & MREMAP_FIXED)) {
332             unsigned long map_flags = 0;
333             if (vma->vm_flags & VM_SHARED)
334                 map_flags |= MAP_SHARED;
335
336             new_addr = get_unmapped_area(vma->vm_file, 0,
337                                         new_len, vma->vm_pgoff, map_flags);
338             ret = new_addr;
339             if (new_addr & ~PAGE_MASK)
340                 goto out;
341         }
342         ret = move_vma(vma, addr, old_len, new_len, new_addr);
343     }
344     out:
345     return ret;
346 }
```

To expand the region, a new one has to be allocated, and the old one moved to it.

329 The default action is to return saying no memory is available.

330 Checks to make sure the region is allowed to move.

331 If `MREMAP_FIXED` is not specified, it means the new location was not supplied, so one must be found.

333-334 Preserves the `MAP_SHARED` option.

336 Finds an unmapped region of memory large enough for the expansion.

337 The return value is the address of the new region.

338-339 For the returned address to be not page aligned, `get_unmapped_area()` would need to be broken. This could possibly be the case with a buggy device driver implementing `get_unmapped_area()` incorrectly.

341 Calls `move_vma()` to move the region.

343-344 Returns the address if successful and the error code otherwise.

D.2.4.3 Function: `move_vma()` (*mm/mremap.c*)

The call graph for this function is shown in Figure 4.7. This function is responsible for moving all the pagetable entries from one VMA to another region. If necessary, a new VMA will be allocated for the region being moved to. Just like the previous function, it is very long, but may be broken up into the following distinct parts.

- Function preamble finds the VMA preceding the area about to be moved to and the VMA in front of the region to be mapped.
- Handles the case where the new location is between two existing VMAs. It determines if the preceding region can be expanded forward or the next region expanded backward to cover the new mapped region.
- Handles the case where the new location is going to be the last VMA on the list. It determines if the preceding region can be expanded forward.
- If a region could not be expanded, it allocates a new VMA from the slab allocator.
- Calls `move_page_tables()`, fills in the new VMA details if a new one was allocated, and updates statistics before returning.

```

125 static inline unsigned long move_vma(struct vm_area_struct * vma,
126     unsigned long addr, unsigned long old_len, unsigned long
127     new_len, unsigned long new_addr)
128 {
129     struct mm_struct * mm = vma->vm_mm;
130     struct vm_area_struct * new_vma, * next, * prev;
131     int allocated_vma;
132
133     new_vma = NULL;
134     next = find_vma_prev(mm, new_addr, &prev);

```

125-127 The parameters are the following:

- **vma** The VMA that the address being moved belongs to
- **addr** The starting address of the moving region
- **old_len** The old length of the region to move
- **new_len** The new length of the region moved
- **new_addr** The new address to relocate to

134 Finds the VMA preceding the address being moved indicated by `prev` and returns the region after the new mapping as `next`.

```

135     if (next) {
136         if (prev && prev->vm_end == new_addr &&

```

```

137         can_vma_merge(prev, vma->vm_flags) &&
!vma->vm_file && !(vma->vm_flags & VM_SHARED)) {
138         spin_lock(&mm->page_table_lock);
139         prev->vm_end = new_addr + new_len;
140         spin_unlock(&mm->page_table_lock);
141         new_vma = prev;
142         if (next != prev->vm_next)
143             BUG();
144         if (prev->vm_end == next->vm_start &&
can_vma_merge(next, prev->vm_flags)) {
145             spin_lock(&mm->page_table_lock);
146             prev->vm_end = next->vm_end;
147             __vma_unlink(mm, next, prev);
148             spin_unlock(&mm->page_table_lock);
149
150             mm->map_count--;
151             kmem_cache_free(vm_area_cachep, next);
152         }
153     } else if (next->vm_start == new_addr + new_len &&
can_vma_merge(next, vma->vm_flags) &&
154         !vma->vm_file && !(vma->vm_flags & VM_SHARED)) {
155         spin_lock(&mm->page_table_lock);
156         next->vm_start = new_addr;
157         spin_unlock(&mm->page_table_lock);
158         new_vma = next;
159     }
160 } else {

```

In this block, the new location is between two existing VMAs. Checks are made to see if the preceding region can be expanded to cover the new mapping and then if it can be expanded to cover the next VMA as well. If it cannot be expanded, the next region is checked to see if it can be expanded backward.

136-137 If the preceding region touches the address to be mapped to and may be merged, it enters this block, which will attempt to expand regions.

138 Locks the mm.

139 Expands the preceding region to cover the new location.

140 Unlocks the mm.

141 The new VMA is now the preceding VMA, which was just expanded.

142-143 Makes sure the VMA linked list is intact. It would require a device driver with severe brain damage to cause this situation to occur.

144 Checks if the region can be expanded forward to encompass the next region.

145 If it can, this locks the mm.

146 Expands the VMA further to cover the next VMA.

147 There is now an extra VMA, so this unlinks it.

148 Unlocks the mm.

150 There is one less mapping now, so this updates the `map_count`.

151 Frees the memory used by the memory mapping.

153 If the `prev` region could not be expanded forward, this checks if the region pointed to be `next` may be expanded backward to cover the new mapping instead.

155 If it can, this locks the mm.

156 Expands the mapping backward.

157 Unlocks the mm.

158 The VMA representing the new mapping is now `next`.

```

161         prev = find_vma(mm, new_addr-1);
162         if (prev && prev->vm_end == new_addr &&
163             can_vma_merge(prev, vma->vm_flags) && !vma->vm_file &&
164                 !(vma->vm_flags & VM_SHARED)) {
165             spin_lock(&mm->page_table_lock);
166             prev->vm_end = new_addr + new_len;
167             spin_unlock(&mm->page_table_lock);
168             new_vma = prev;
169         }

```

This block is for the case where the newly mapped region is the last VMA (`next` is NULL), so a check is made to see if the preceding region can be expanded.

161 Gets the previously mapped region.

162-163 Checks if the regions may be mapped.

164 Locks the mm.

165 Expands the preceding region to cover the new mapping.

166 Locks the mm.

167 The VMA representing the new mapping is now `prev`.


```

170
171     allocated_vma = 0;
172     if (!new_vma) {
173         new_vma = kmem_cache_alloc(vm_area_cachep, SLAB_KERNEL);
174         if (!new_vma)
175             goto out;
176         allocated_vma = 1;
177     }
178
179     if (!move_page_tables(current->mm, new_addr, addr, old_len)) {
180         unsigned long vm_locked = vma->vm_flags & VM_LOCKED;
181
182         if (allocated_vma) {
183             *new_vma = *vma;
184             new_vma->vm_start = new_addr;
185             new_vma->vm_end = new_addr+new_len;
186             new_vma->vm_pgoff +=
187                 (addr-vma->vm_start) >> PAGE_SHIFT;
188             new_vma->vm_raend = 0;
189             if (new_vma->vm_file)
190                 get_file(new_vma->vm_file);
191             if (new_vma->vm_ops && new_vma->vm_ops->open)
192                 new_vma->vm_ops->open(new_vma);
193             insert_vm_struct(current->mm, new_vma);
194
195             do_munmap(current->mm, addr, old_len);
196
197             current->mm->total_vm += new_len >> PAGE_SHIFT;
198             if (new_vma->vm_flags & VM_LOCKED) {
199                 current->mm->locked_vm += new_len >> PAGE_SHIFT;
200                 make_pages_present(new_vma->vm_start,
201                                     new_vma->vm_end);
202             }
203             return new_addr;
204         }
205         if (allocated_vma)

```

171 Sets a flag indicating if a new VMA was not allocated.

172 If a VMA has not been expanded to cover the new mapping then...

173 Allocates a new VMA from the slab allocator.

174-175 If it could not be allocated, goto out to return failure.

176 Sets the flag indicating that a new VMA was allocated.

```

206         kmem_cache_free(vm_area_cachep, new_vma);
207     out:
208         return -ENOMEM;
209 }

```

179 `move_page_tables()` (See Section D.2.4.6) is responsible for copying all the pagetable entries. It returns 0 on success.

182-193 If a new VMA was allocated, this fills in all the relevant details, including the file/device entries, and inserts it into the various VMA linked lists with `insert_vm_struct()` (See Section D.2.2.1).

194 Unmaps the old region because it is no longer required.

197 Updates the `total_vm` size for this process. The size of the old region is not important because it is handled within `do_munmap()`.

198-202 If the VMA has the `VM_LOCKED` flag, all the pages within the region are made present with `mark_pages_present()`.

203 Returns the address of the new region.

205-206 This is the error path. If a VMA was allocated, it deletes it.

208 Returns an out of memory error.

D.2.4.4 Function: `make_pages_present()` (*mm/memory.c*)

This function makes all pages between `addr` and `end` present. It assumes that the two addresses are within the one VMA.

```

1460 int make_pages_present(unsigned long addr, unsigned long end)
1461 {
1462     int ret, len, write;
1463     struct vm_area_struct * vma;
1464
1465     vma = find_vma(current->mm, addr);
1466     write = (vma->vm_flags & VM_WRITE) != 0;
1467     if (addr >= end)
1468         BUG();
1469     if (end > vma->vm_end)
1470         BUG();
1471     len = (end+PAGE_SIZE-1)/PAGE_SIZE-addr/PAGE_SIZE;
1472     ret = get_user_pages(current, current->mm, addr,
1473         len, write, 0, NULL, NULL);
1474     return ret == len ? 0 : -1;
1475 }

```

1465 Finds the VMA with `find_vma()` (See Section D.3.1.1) that contains the starting address.

1466 Records if write-access is allowed in `write`.

1467-1468 If the starting address is after the end address, then `BUG()` runs.

1469-1470 If the range spans more than one VMA, it is a bug.

1471 Calculates the length of the region to fault in.

1472 Calls `get_user_pages()` to fault in all the pages in the requested region. It returns the number of pages that were faulted in.

1474 Returns true if all the requested pages were successfully faulted in.

D.2.4.5 Function: `get_user_pages()` (*mm/memory.c*)

This function is used to fault in user pages and may be used to fault in pages belonging to another process, which is required by `ptrace()`, for example.

```

454 int get_user_pages(struct task_struct *tsk, struct mm_struct *mm,
                     unsigned long start,
455                     int len, int write, int force, struct page
                     **pages, struct vm_area_struct **vmas)
456 {
457     int i;
458     unsigned int flags;
459
460     /*
461      * Require read or write permissions.
462      * If 'force' is set, we only require the "MAY" flags.
463      */
464     flags = write ? (VM_WRITE | VM_MAYWRITE) : (VM_READ | VM_MAYREAD);
465     flags &= force ? (VM_MAYREAD | VM_MAYWRITE) : (VM_READ | VM_WRITE);
466     i = 0;
467
```

454 The parameters are the following:

- **tsk** is the process that pages are being faulted for.
- **mm** is the `mm_struct` managing the address space being faulted.
- **start** is where to start faulting.
- **len** is the length of the region, in pages, to fault.
- **write** indicates if the pages are being faulted for writing.
- **force** indicates that the pages should be faulted even if the region only has the `VM_MAYREAD` or `VM_MAYWRITE` flags.
- **pages** is an array of struct pages, which may be NULL. If supplied, the array will be filled with `struct page`s that were faulted in.

- **vmass** is similar to the **pages** array. If supplied, it will be filled with VMAs that were affected by the faults.

464 Sets the required flags to **VM_WRITE** and **VM_MAYWRITE** flags if the parameter **write** is set to 1. Otherwise, it uses the read equivalents.

465 If **force** is specified, this only requires the **MAY** flags.

```

468     do {
469         struct vm_area_struct * vma;
470
471         vma = find_extend_vma(mm, start);
472
473         if ( !vma ||
              (pages && vma->vm_flags & VM_IO) ||
              !(flags & vma->vm_flags) )
474             return i ? : -EFAULT;
475
476         spin_lock(&mm->page_table_lock);
477         do {
478             struct page *map;
479             while (!(map = follow_page(mm, start, write))) {
480                 spin_unlock(&mm->page_table_lock);
481                 switch (handle_mm_fault(mm, vma, start, write)) {
482                     case 1:
483                         tsk->min_flt++;
484                         break;
485                     case 2:
486                         tsk->maj_flt++;
487                         break;
488                     case 0:
489                         if (i) return i;
490                         return -EFAULT;
491                     default:
492                         if (i) return i;
493                         return -ENOMEM;
494                 }
495                 spin_lock(&mm->page_table_lock);
496             }
497             if (pages) {
498                 pages[i] = get_page_map(map);
499                 /* FIXME: call the correct function,
500                  * depending on the type of the found page
501                  */
502                 if (!pages[i])
503                     goto bad_page;
504                 page_cache_get(pages[i]);

```

```

505         }
506         if (vmas)
507             vmas[i] = vma;
508             i++;
509             start += PAGE_SIZE;
510             len--;
511     } while(len && start < vma->vm_end);
512     spin_unlock(&mm->page_table_lock);
513 } while(len);
514 out:
515     return i;

```

468-513 This outer loop will move through every VMA affected by the faults.

471 Finds the VMA affected by the current value of **start**. This variable is incremented in **PAGE_SIZE**d strides.

473 If a VMA does not exist for the address, or the caller has requested **struct pages** for a region that is I/O mapped (and therefore not backed by physical memory) or that the VMA does not have the required flags for, this returns **-EFAULT**.

476 Locks the pagetable spinlock.

479-496 **follow_page()** (See Section C.2.1) walks the page tables and returns the **struct page** that represents the frame mapped at **start**. This loop will only be entered if the PTE is not present and will keep looping until the PTE is known to be present with the pagetable spinlock held.

480 Unlocks the page table spinlock because **handle_mm_fault()** is likely to sleep.

481 If the page is not present, this faults it in with **handle_mm_fault()** (See Section D.5.3.1).

482-487 Updates the **task_struct** statistics and indicates if a major or minor fault occurred.

488-490 If the faulting address is invalid, this returns **-EFAULT**.

491-493 If the system is out of memory, this returns **-ENOMEM**.

495 Relocks the page tables. The loop will check to make sure the page is actually present.

597-505 If the caller requested it, this populates the **pages** array with **struct pages** affected by this function. Each struct will have a reference to it taken with **page_cache_get()**.

506-507 Similarly, this records VMAs affected.

508 Increments `i`, which is a counter for the number of pages present in the requested region.

509 Increments `start` in a page-sized stride.

510 Decrements the number of pages that must be faulted in.

511 Keeps moving through the VMAs until the requested pages have been faulted in.

512 Releases the pagetable spinlock.

515 Returns the number of pages known to be present in the region.

```

516
517     /*
518     * We found an invalid page in the VMA. Release all we have
519     * so far and fail.
520     */
521 bad_page:
522     spin_unlock(&mm->page_table_lock);
523     while (i--)
524         page_cache_release(pages[i]);
525     i = -EFAULT;
526     goto out;
527 }
```

521 This will only be reached if a `struct page` is found that represents a non-existent page frame.

523-524 If one is found, it releases references to all pages stored in the `pages` array.

525-526 Returns `-EFAULT`.

D.2.4.6 Function: `move_page_tables()` (*mm/mremap.c*)

The call graph for this function is shown in Figure 4.8. This function is responsible for copying all the pagetable entries from the region pointed to `old_addr` to `new_addr`. It works by literally copying pagetable entries one at a time. When it is finished, it deletes all the entries from the old area. This is not the most efficient way to perform the operation, but it is very easy to error recover.

```

90 static int move_page_tables(struct mm_struct * mm,
91     unsigned long new_addr, unsigned long old_addr,
92     unsigned long len)
93 {
94     unsigned long offset = len;
95     flush_cache_range(mm, old_addr, old_addr + len);
```

```

96
102     while (offset) {
103         offset -= PAGE_SIZE;
104         if (move_one_page(mm, old_addr + offset, new_addr +
105             offset))
106             goto oops_we_failed;
107     }
108     flush_tlb_range(mm, old_addr, old_addr + len);
109     return 0;
110
111 oops_we_failed:
112     flush_cache_range(mm, new_addr, new_addr + len);
113     while ((offset += PAGE_SIZE) < len)
114         move_one_page(mm, new_addr + offset, old_addr + offset);
115     zap_page_range(mm, new_addr, len);
116     return -1;
117 }

```

90 The parameters are the mm for the process, the new location, the old location and the length of the region to move entries for.

95 `flush_cache_range()` will flush all CPU caches for this range. It must be called first because some architectures, notably Sparc's, require that a virtual to physical mapping exist before flushing the TLB.

102-106 Loops through each page in the region and moves the PTE with `move_one_pte()` (See Section D.2.4.7). This translates to a lot of pagetable walking and could be performed much better, but it is a rare operation.

107 Flushes the TLB for the old region.

108 Returns success.

118-120 This block moves all the PTEs back. A `flush_tlb_range()` is not necessary because the region could not have been used yet, so no TLB entries should exist.

121 Zaps any pages that were allocated for the move.

122 Returns failure.

D.2.4.7 Function: `move_one_page()` (*mm/mremap.c*)

This function is responsible for acquiring the spinlock before finding the correct PTE with `get_one_pte()` and copying it with `copy_one_pte()`.

```

77 static int move_one_page(struct mm_struct *mm,
78     unsigned long old_addr, unsigned long new_addr)
79 {

```

```

79     int error = 0;
80     pte_t * src;
81
82     spin_lock(&mm->page_table_lock);
83     src = get_one_pte(mm, old_addr);
84     if (src)
85         error = copy_one_pte(mm, src, alloc_one_pte(mm, new_addr));
86     spin_unlock(&mm->page_table_lock);
87     return error;
88 }

```

82 Acquires the mm lock.

83 Calls `get_one_pte()` (See Section D.2.4.8), which walks the pagetables to get the correct PTE.

84-85 If the PTE exists, this allocates a PTE for the destination and copies the PTEs with `copy_one_pte()` (See Section D.2.4.10).

86 Releases the lock.

87 Returns whatever `copy_one_pte()` returned. It will only return an error if `alloc_one_pte()` (See Section D.2.4.9) failed on line 85.

D.2.4.8 Function: `get_one_pte()` (*mm/mremap.c*)

This is a very simple pagetable walk.

```

18 static inline pte_t *get_one_pte(struct mm_struct *mm,
                                unsigned long addr)
19 {
20     pgd_t * pgd;
21     pmd_t * pmd;
22     pte_t * pte = NULL;
23
24     pgd = pgd_offset(mm, addr);
25     if (pgd_none(*pgd))
26         goto end;
27     if (pgd_bad(*pgd)) {
28         pgd_ERROR(*pgd);
29         pgd_clear(pgd);
30         goto end;
31     }
32
33     pmd = pmd_offset(pgd, addr);
34     if (pmd_none(*pmd))
35         goto end;
36     if (pmd_bad(*pmd)) {

```



```

37         pmd_ERROR(*pmd);
38         pmd_clear(pmd);
39         goto end;
40     }
41
42     pte = pte_offset(pmd, addr);
43     if (pte_none(*pte))
44         pte = NULL;
45 end:
46     return pte;
47 }

```

24 Gets the PGD for this address.

25-26 If no PGD exists, this returns NULL because no PTE will exist either.

27-31 If the PGD is bad, this marks that an error occurred in the region, clears its contents and returns NULL.

33-40 Acquires the correct PMD in the same fashion as for the PGD.

42 Acquires the PTE so it may be returned if it exists.

D.2.4.9 Function: `alloc_one_pte()` (*mm/mremap.c*)

This trivial function allocates what is necessary for one PTE in a region.

```

49 static inline pte_t *alloc_one_pte(struct mm_struct *mm,
                                     unsigned long addr)
50 {
51     pmd_t * pmd;
52     pte_t * pte = NULL;
53
54     pmd = pmd_alloc(mm, pgd_offset(mm, addr), addr);
55     if (pmd)
56         pte = pte_alloc(mm, pmd, addr);
57     return pte;
58 }

```

54 If a PMD entry does not exist, this allocates it.

55-56 If the PMD exists, this allocates a PTE entry. The check to make sure it succeeded is performed later in the function `copy_one_pte()`.

D.2.4.10 Function: `copy_one_pte()` (*mm/mremap.c*)

This copies the contents of one PTE to another.

```

60 static inline int copy_one_pte(struct mm_struct *mm,
                                pte_t * src, pte_t * dst)

```

```

61 {
62     int error = 0;
63     pte_t pte;
64
65     if (!pte_none(*src)) {
66         pte = ptep_get_and_clear(src);
67         if (!dst) {
68             /* No dest? We must put it back. */
69             dst = src;
70             error++;
71         }
72         set_pte(dst, pte);
73     }
74     return error;
75 }

```

65 If the source PTE does not exist, this just returns 0 to say the copy was successful.

66 Gets the PTE and removes it from its old location.

67-71 If the `dst` does not exist, it means the call to `alloc_one_pte()` failed, and the copy operation has failed and must be aborted.

72 Moves the PTE to its new location.

74 Returns an error if one occurred.

D.2.5 Deleting a Memory Region

D.2.5.1 Function: `do_munmap()` (*mm/mmap.c*)

The call graph for this function is shown in Figure 4.10. This function is responsible for unmapping a region. If necessary, the unmapping can span multiple VMAs, and it can partially unmap one if necessary. Hence, the full unmapping operation is divided into two major operations. This function is responsible for finding what VMAs are affected, and `unmap_fixup()` is responsible for fixing up the remaining VMAs.

This function is divided up in a number of small sections that will be dealt with in turn. They are, broadly speaking, the following:

- Function as a preamble, and find the VMA to start working from.
- Take all VMAs affected by the unmapping out of the mm and place them on a linked list headed by the variable `free`.
- Cycle through the list headed by `free`, unmap all the pages in the region to be unmapped and call `unmap_fixup()` to fix up the mappings.
- Validate the mm and free memory associated with the unmapping.

```

924 int do_munmap(struct mm_struct *mm, unsigned long addr,
                size_t len)
925 {
926     struct vm_area_struct *mpnt, *prev, **npp, *free, *extra;
927
928     if ((addr & ~PAGE_MASK) || addr > TASK_SIZE ||
        len > TASK_SIZE-addr)
929         return -EINVAL;
930
931     if ((len = PAGE_ALIGN(len)) == 0)
932         return -EINVAL;
933
939     mpnt = find_vma_prev(mm, addr, &prev);
940     if (!mpnt)
941         return 0;
942     /* we have addr < mpnt->vm_end */
943
944     if (mpnt->vm_start >= addr+len)
945         return 0;
946
948     if ((mpnt->vm_start < addr && mpnt->vm_end > addr+len)
949         && mm->map_count >= max_map_count)
950         return -ENOMEM;
951
956     extra = kmem_cache_alloc(vm_area_cachep, SLAB_KERNEL);
957     if (!extra)
958         return -ENOMEM;

```

924 The parameters are as follows:

- **mm** The mm for the processes performing the unmap operation
- **addr** The starting address of the region to unmap
- **len** The length of the region

928-929 Ensures the address is page aligned and that the area to be unmapped is not in the kernel virtual address space.

931-932 Makes sure the region size to unmap is page aligned.

939 Finds the VMA that contains the starting address and the preceding VMA so it can be easily unlinked later.

940-941 If no **mpnt** was returned, it means the address must be past the last used VMA. Therefore, the address space is unused and just returns.

944-945 If the returned VMA starts past the region you are trying to unmap, the region is unused and just returns.

948-950 The first part of the check sees if the VMA is just being partially unmapped. If it is, another VMA will be created later to deal with a region being broken into, so the `map_count` has to be checked to make sure it is not too large.

956-958 In case a new mapping is required, it is allocated now because later it will be much more difficult to back out in event of an error.

```

960     npp = (prev ? &prev->vm_next : &mm->mmap);
961     free = NULL;
962     spin_lock(&mm->page_table_lock);
963     for ( ; mpnt && mpnt->vm_start < addr+len; mpnt = *npp) {
964         *npp = mpnt->vm_next;
965         mpnt->vm_next = free;
966         free = mpnt;
967         rb_erase(&mpnt->vm_rb, &mm->mm_rb);
968     }
969     mm->mmap_cache = NULL; /* Kill the cache. */
970     spin_unlock(&mm->page_table_lock);

```

This section takes all the VMAs affected by the unmapping and places them on a separate linked list headed by a variable called `free`. This makes the fixup of the regions much easier.

960 `npp` becomes the next VMA in the list during the for loop that follows. To initialize it, it is either the current VMA (`mpnt`), or it becomes the first VMA in the list.

961 `free` is the head of a linked list of VMAs that are affected by the unmapping.

962 Locks the mm.

963 Cycles through the list until the start of the current VMA is past the end of the region to be unmapped.

964 `npp` becomes the next VMA in the list.

965-966 Removes the current VMA from the linear linked list within the mm and places it on a linked list headed by `free`. The current `mpnt` becomes the head of the free linked list.

967 Deletes `mpnt` from the red-black tree.

969 Removes the cached result in case the last looked-up result is one of the regions to be unmapped.

970 Frees the mm.

```

971
972     /* Ok - we have the memory areas we should free on the
973      * 'free' list, so release them, and unmap the page range..
974      * If one of the segments is only being partially unmapped,
975      * it will put new vm_area_struct(s) into the address space.
976      * In that case we have to be careful with VM_DENYWRITE.
977      */
978     while ((mpnt = free) != NULL) {
979         unsigned long st, end, size;
980         struct file *file = NULL;
981
982         free = free->vm_next;
983
984         st = addr < mpnt->vm_start ? mpnt->vm_start : addr;
985         end = addr+len;
986         end = end > mpnt->vm_end ? mpnt->vm_end : end;
987         size = end - st;
988
989         if (mpnt->vm_flags & VM_DENYWRITE &&
990             (st != mpnt->vm_start || end != mpnt->vm_end) &&
991             (file = mpnt->vm_file) != NULL) {
992             atomic_dec(&file->f_dentry->d_inode->i_writecount);
993         }
994         remove_shared_vm_struct(mpnt);
995         mm->map_count--;
996
997         zap_page_range(mm, st, size);
998
999         /*
1000          * Fix the mapping, and free the old area
1001          * if it wasn't reused.
1002          */
1003         extra = unmap_fixup(mm, mpnt, st, size, extra);
1004         if (file)
1005             atomic_inc(&file->f_dentry->d_inode->i_writecount);
1006     }

```

978 Keeps stepping through the list until no VMAs are left.

982 Moves `free` to the next element in the list, leaving `mpnt` as the head about to be removed.

984 `st` is the start of the region to be unmapped. If the `addr` is before the start of the VMA, the starting point is `mpnt->vm_start`. Otherwise, it is the supplied address.

985-986 Calculates the end of the region to map in a similar fashion.

987 Calculates the size of the region to be unmapped in this pass.

989-993 If the `VM_DENYWRITE` flag is specified, a hole will be created by this unmapping, and a file is mapped. Then, the `i_writecounts` are decremented. When this field is negative, it counts how many users there are protecting this file from being opened for writing.

994 Removes the file mapping. If the file is still partially mapped, it will be acquired again during `unmap_fixup()` (See Section D.2.5.2).

995 Reduces the map count.

997 Removes all pages within this region.

1002 Calls `unmap_fixup()` (See Section D.2.5.2) to fix up the regions after this one is deleted.

1003-1004 Increments the writecount to the file because the region has been unmapped. If it was just partially unmapped, this call will simply balance out the decrement at line 987.

```

1006     validate_mm(mm);
1007
1008     /* Release the extra vma struct if it wasn't used */
1009     if (extra)
1010         kmem_cache_free(vm_area_cachep, extra);
1011
1012     free_pgtables(mm, prev, addr, addr+len);
1013
1014     return 0;
1015 }
```

1006 `validate_mm()` is a debugging function. If enabled, it will ensure the VMA tree for this mm is still valid.

1009-1010 If extra VMA was not required, this deletes it.

1012 Frees all the pagetables that were used for the unmapped region.

1014 Returns success.

D.2.5.2 Function: `unmap_fixup()` (*mm/mmap.c*)

This function fixes up the regions after a block has been unmapped. It is passed a list of VMAs that are affected by the unmapping, the region and length to be unmapped and a spare VMA that may be required to fix up the region if a whole is created. This function handles four principle cases: the unmapping of a region, partial unmapping from the start to somewhere in the middle, partial unmapping from somewhere in the middle to the end and creation of a hole in the middle of the region. Each case will be taken in turn.

```

787 static struct vm_area_struct * unmap_fixup(struct mm_struct *mm,
788     struct vm_area_struct *area, unsigned long addr, size_t len,
789     struct vm_area_struct *extra)
790 {
791     struct vm_area_struct *mpnt;
792     unsigned long end = addr + len;
793
794     area->vm_mm->total_vm -= len >> PAGE_SHIFT;
795     if (area->vm_flags & VM_LOCKED)
796         area->vm_mm->locked_vm -= len >> PAGE_SHIFT;
797

```

This block is the function preamble.

787 The parameters to the function are the following:

- **mm** is the mm the unmapped region belongs to.
- **area** is the head of the linked list of VMAs affected by the unmapping.
- **addr** is the starting address of the unmapping.
- **len** is the length of the region to be unmapped.
- **extra** is a spare VMA passed in for when a hole in the middle is created.

792 Calculates the end address of the region being unmapped.

794 Reduces the count of the number of pages used by the process.

795-796 If the pages were locked in memory, this reduces the locked page count.

```

798     /* Unmapping the whole area. */
799     if (addr == area->vm_start && end == area->vm_end) {
800         if (area->vm_ops && area->vm_ops->close)
801             area->vm_ops->close(area);
802         if (area->vm_file)
803             fput(area->vm_file);
804         kmem_cache_free(vm_area_cachep, area);
805         return extra;
806     }

```

The first, and easiest, case is where the full region is being unmapped.

799 The full region is unmapped if the **addr** is the start of the VMA and the end is the end of the VMA. This is interesting because, if the unmapping is spanning regions, it is possible that the end is *beyond* the end of the VMA, but the full of this VMA is still being unmapped.

800-801 If a close operation is supplied by the VMA, this calls it.

802-803 If a file or device is mapped, this calls `fput()`, which decrements the usage count and releases it if the count falls to 0.

804 Frees the memory for the VMA back to the slab allocator.

805 Returns the extra VMA because it was unused.

```

809     if (end == area->vm_end) {
810         /*
811          * here area isn't visible to the semaphore-less readers
812          * so we don't need to update it under the spinlock.
813          */
814         area->vm_end = addr;
815         lock_vma_mappings(area);
816         spin_lock(&mm->page_table_lock);
817     }

```

This block handles the case where the middle of the region to the end is been unmapped.

814 Truncates the VMA back to `addr`. At this point, the pages for the region have already freed, and the pagetable entries will be freed later, so no further work is required.

815 If a file/device is being mapped, the lock protecting shared access to it is taken in the function `lock_vma_mappings()`.

816 Locks the mm. Later in the function, the remaining VMA will be reinserted into the mm.

```

817         else if (addr == area->vm_start) {
818             area->vm_pgoff += (end - area->vm_start) >> PAGE_SHIFT;
819             /* same locking considerations of the above case */
820             area->vm_start = end;
821             lock_vma_mappings(area);
822             spin_lock(&mm->page_table_lock);
823         } else {

```

This block handles the case where the VMA is been unmapped from the start to some part in the middle.

818 Increases the offset within the file/device mapped by the number of pages this unmapping represents.

820 Moves the start of the VMA to the end of the region being unmapped.

821-822 Locks the file/device and mm as previously described.


```

823     } else {
825         /* Add end mapping -- leave beginning for below */
826         mpnt = extra;
827         extra = NULL;
828
829         mpnt->vm_mm = area->vm_mm;
830         mpnt->vm_start = end;
831         mpnt->vm_end = area->vm_end;
832         mpnt->vm_page_prot = area->vm_page_prot;
833         mpnt->vm_flags = area->vm_flags;
834         mpnt->vm_raend = 0;
835         mpnt->vm_ops = area->vm_ops;
836         mpnt->vm_pgoff = area->vm_pgoff +
            ((end - area->vm_start) >> PAGE_SHIFT);
837         mpnt->vm_file = area->vm_file;
838         mpnt->vm_private_data = area->vm_private_data;
839         if (mpnt->vm_file)
840             get_file(mpnt->vm_file);
841         if (mpnt->vm_ops && mpnt->vm_ops->open)
842             mpnt->vm_ops->open(mpnt);
843         area->vm_end = addr;    /* Truncate area */
844
845         /* Because mpnt->vm_file == area->vm_file this locks
846          * things correctly.
847          */
848         lock_vma_mappings(area);
849         spin_lock(&mm->page_table_lock);
850         __insert_vm_struct(mm, mpnt);
851     }

```

This block handles the case where a hole is being created by a partial unmapping. In this case, the extra VMA is required to create a new mapping from the end of the unmapped region to the end of the old VMA.

826-827 Takes the extra VMA and makes VMA NULL so that the calling function will know it is in use and cannot be freed.

828-838 Copies in all the VMA information.

839 If a file/device is mapped, this gets a reference to it with `get_file()`.

841-842 If an open function is provided, this calls it.

843 Truncates the VMA so that it ends at the start of the region to be unmapped.

848-849 Locks the files and mm as with the two previous cases.

850 Inserts the extra VMA into the mm.

```

852
853     __insert_vm_struct(mm, area);
854     spin_unlock(&mm->page_table_lock);
855     unlock_vma_mappings(area);
856     return extra;
857 }

```

853 Reinserts the VMA into the mm.

854 Unlocks the pagetables.

855 Unlocks the spinlock to the shared mapping.

856 Returns the extra VMA if it was not used and NULL if it was.

D.2.6 Deleting All Memory Regions

D.2.6.1 Function: `exit_mmap()` (*mm/mmap.c*)

This function simply steps through all VMAs associated with the supplied mm and unmaps them.

```

1127 void exit_mmap(struct mm_struct * mm)
1128 {
1129     struct vm_area_struct * mpnt;
1130
1131     release_segments(mm);
1132     spin_lock(&mm->page_table_lock);
1133     mpnt = mm->mmap;
1134     mm->mmap = mm->mmap_cache = NULL;
1135     mm->mm_rb = RB_ROOT;
1136     mm->rss = 0;
1137     spin_unlock(&mm->page_table_lock);
1138     mm->total_vm = 0;
1139     mm->locked_vm = 0;
1140
1141     flush_cache_mm(mm);
1142     while (mpnt) {
1143         struct vm_area_struct * next = mpnt->vm_next;
1144         unsigned long start = mpnt->vm_start;
1145         unsigned long end = mpnt->vm_end;
1146         unsigned long size = end - start;
1147
1148         if (mpnt->vm_ops) {
1149             if (mpnt->vm_ops->close)
1150                 mpnt->vm_ops->close(mpnt);
1151         }
1152         mm->map_count--;
1153         remove_shared_vm_struct(mpnt);

```

```

1154     zap_page_range(mm, start, size);
1155     if (mpnt->vm_file)
1156         fput(mpnt->vm_file);
1157     kmem_cache_free(vm_area_cachep, mpnt);
1158     mpnt = next;
1159 }
1160 flush_tlb_mm(mm);
1161
1162 /* This is just debugging */
1163 if (mm->map_count)
1164     BUG();
1165
1166 clear_page_tables(mm, FIRST_USER_PGD_NR, USER_PTRS_PER_PGD);
1167 }

```

1131 `release_segments()` will release memory segments associated with the process on its Local Descriptor Table (LDT) if the architecture supports segments and the process was using them. Some applications, notably WINE, use this feature.

1132 Locks the mm.

1133 `mpnt` becomes the first VMA on the list.

1134 Clears VMA-related information from the mm so that it may be unlocked.

1137 Unlocks the mm.

1138-1139 Clears the mm statistics.

1141 Flushes the CPU for the address range.

1142-1159 Steps through every VMA that was associated with the mm.

1143 Records what the next VMA to clear will be so that this one may be deleted.

1144-1146 Records the start, end and size of the region to be deleted.

1148-1151 If there is a close operation associated with this VMA, this calls it.

1152 Reduces the map count.

1153 Removes the file/device mapping from the shared mappings list.

1154 Frees all pages associated with this region.

1155-1156 If a file/device was mapped in this region, this frees it.

1157 Frees the VMA struct.

1158 Moves to the next VMA.

1160 Flushes the TLB for this whole mm because it is about to be unmapped.

1163-1164 If the `map_count` is positive, it means the map count was not accounted for properly, so this calls `BUG()` to mark it.

1166 Clears the pagetables associated with this region with `clear_page_tables()` (See Section D.2.6.2).

D.2.6.2 Function: `clear_page_tables()` (*mm/memory.c*)

This is the top-level function used to unmap all PTEs and free pages within a region. It is used when pagetables need to be torn down, such as when the process exits or a region is unmapped.

```

146 void clear_page_tables(struct mm_struct *mm,
                          unsigned long first, int nr)
147 {
148     pgd_t * page_dir = mm->pgd;
149
150     spin_lock(&mm->page_table_lock);
151     page_dir += first;
152     do {
153         free_one_pgd(page_dir);
154         page_dir++;
155     } while (--nr);
156     spin_unlock(&mm->page_table_lock);
157
158     /* keep the pagetable cache within bounds */
159     check_pgt_cache();
160 }
```

148 Gets the PGD for the mm being unmapped.

150 Locks the pagetables.

151-155 Steps through all PGDs in the requested range. For each PGD found, this calls `free_one_pgd()` (See Section D.2.6.3).

156 Unlocks the pagetables.

159 Checks the cache of available PGD structures. If there are too many PGDs in the PGD quicklist, some of them will be reclaimed.

D.2.6.3 Function: `free_one_pgd()` (*mm/memory.c*)

This function tears down one PGD. For each PMD in this PGD, `free_one_pmd()` will be called.

```

109 static inline void free_one_pgd(pgd_t * dir)
110 {
```

```

111     int j;
112     pmd_t * pmd;
113
114     if (pgd_none(*dir))
115         return;
116     if (pgd_bad(*dir)) {
117         pgd_ERROR(*dir);
118         pgd_clear(dir);
119         return;
120     }
121     pmd = pmd_offset(dir, 0);
122     pgd_clear(dir);
123     for (j = 0; j < PTRS_PER_PMD ; j++) {
124         prefetchw(pmd+j+(PREFETCH_STRIDE/16));
125         free_one_pmd(pmd+j);
126     }
127     pmd_free(pmd);
128 }

```

114-115 If no PGD exists here, this returns.

116-120 If the PGD is bad, this flags the error and returns.

1121 Gets the first PMD in the PGD.

122 Clears the PGD entry.

123-126 For each PMD in this PGD, this calls `free_one_pmd()` (See Section D.2.6.4).

127 Frees the PMD page to the PMD quicklist. Later, `check_pgt_cache()` will be called, and, if the cache has too many PMD pages in it, they will be reclaimed.

D.2.6.4 Function: `free_one_pmd()` (*mm/memory.c*)

```

93 static inline void free_one_pmd(pmd_t * dir)
94 {
95     pte_t * pte;
96
97     if (pmd_none(*dir))
98         return;
99     if (pmd_bad(*dir)) {
100         pmd_ERROR(*dir);
101         pmd_clear(dir);
102         return;
103     }
104     pte = pte_offset(dir, 0);
105     pmd_clear(dir);

```

```
106     pte_free(pte);  
107 }
```

97-98 If no PMD exists here, this returns.

99-103 If the PMD is bad, this flags the error and returns.

104 Gets the first PTE in the PMD.

105 Clears the PMD from the pagetable.

106 Frees the PTE page to the PTE quicklist cache with `pte_free()`. Later, `check_pgt_cache()` will be called, and, if the cache has too many PTE pages in it, they will be reclaimed.

D.3 Searching Memory Regions

Contents

D.3 Searching Memory Regions	309
D.3.1 Finding a Mapped Memory Region	309
D.3.1.1 Function: <code>find_vma()</code>	309
D.3.1.2 Function: <code>find_vma_prev()</code>	310
D.3.1.3 Function: <code>find_vma_intersection()</code>	312
D.3.2 Finding a Free Memory Region	312
D.3.2.1 Function: <code>get_unmapped_area()</code>	312
D.3.2.2 Function: <code>arch_get_unmapped_area()</code>	313

The functions in this section deal with searching the virtual address space for mapped and free regions.

D.3.1 Finding a Mapped Memory Region

D.3.1.1 Function: `find_vma()` (*mm/mmap.c*)

```

661 struct vm_area_struct * find_vma(struct mm_struct * mm,
                                   unsigned long addr)
662 {
663     struct vm_area_struct *vma = NULL;
664
665     if (mm) {
666         /* Check the cache first. */
667         /* (Cache hit rate is typically around 35%.) */
668         vma = mm->mmap_cache;
669         if (!(vma && vma->vm_end > addr &&
              vma->vm_start <= addr)) {
670             rb_node_t * rb_node;
671
672             rb_node = mm->mm_rb.rb_node;
673             vma = NULL;
674
675             while (rb_node) {
676                 struct vm_area_struct * vma_tmp;
677
678                 vma_tmp = rb_entry(rb_node,
                                   struct vm_area_struct, vm_rb);
679
680                 if (vma_tmp->vm_end > addr) {
681                     vma = vma_tmp;
682                     if (vma_tmp->vm_start <= addr)
683                         break;
684                     rb_node = rb_node->rb_left;
685                 } else
686                     rb_node = rb_node->rb_right;

```

```

687         }
688         if (vma)
689             mm->mmap_cache = vma;
690     }
691 }
692 return vma;
693 }

```

661 The two parameters are the top-level `mm_struct` that is to be searched and the address the caller is interested in.

663 Defaults to returning `NULL` for address not found.

665 Makes sure the caller does not try to search a bogus `mm`.

668 `mmap_cache` has the result of the last call to `find_vma()`. This has a chance of not having to search at all through the red-black tree.

669 If it is a valid VMA that is being examined, this checks to see if the address being searched is contained within it. If it is, the VMA was the `mmap_cache` one, so it can be returned. Otherwise, the tree is searched.

670-674 Starts at the root of the tree.

675-687 This block is the tree walk.

678 The macro, as the name suggests, returns the VMA that this tree node points to.

680 Checks if the next node is traversed by the left or right leaf.

682 If the current VMA is what is required, this exits the while loop.

689 If the VMA is valid, this sets the `mmap_cache` for the next call to `find_vma()`.

692 Returns the VMA that contains the address or, as a side effect of the tree walk, returns the VMA that is closest to the requested address.

D.3.1.2 Function: `find_vma_prev()` (*mm/mmap.c*)

```

696 struct vm_area_struct * find_vma_prev(struct mm_struct * mm,
697                                     unsigned long addr,
698                                     struct vm_area_struct **pprev)
699 {
700     if (mm) {
701         /* Go through the RB tree quickly. */
702         struct vm_area_struct * vma;
703         rb_node_t * rb_node, * rb_last_right, * rb_prev;
704         rb_node = mm->mm_rb.rb_node;

```



```

705     rb_last_right = rb_prev = NULL;
706     vma = NULL;
707
708     while (rb_node) {
709         struct vm_area_struct * vma_tmp;
710
711         vma_tmp = rb_entry(rb_node,
                           struct vm_area_struct, vm_rb);
712
713         if (vma_tmp->vm_end > addr) {
714             vma = vma_tmp;
715             rb_prev = rb_last_right;
716             if (vma_tmp->vm_start <= addr)
717                 break;
718             rb_node = rb_node->rb_left;
719         } else {
720             rb_last_right = rb_node;
721             rb_node = rb_node->rb_right;
722         }
723     }
724     if (vma) {
725         if (vma->vm_rb.rb_left) {
726             rb_prev = vma->vm_rb.rb_left;
727             while (rb_prev->rb_right)
728                 rb_prev = rb_prev->rb_right;
729         }
730         *pprev = NULL;
731         if (rb_prev)
732             *pprev = rb_entry(rb_prev, struct
                              vm_area_struct, vm_rb);
733         if ((rb_prev ? (*pprev)->vm_next : mm->mmap) !=
vma)
734             BUG();
735         return vma;
736     }
737 }
738 *pprev = NULL;
739 return NULL;
740 }

```

696-723 This is essentially the same as the `find_vma()` function already described. The only difference is that the last right node accessed is remembered because this will represent the VMA previous to the requested VMA.

725-729 If the returned VMA has a left node, it means that it has to be traversed. It first takes the left leaf and then follows each right leaf until the bottom of the tree is found.

731-732 Extracts the VMA from the red-black tree node.

733-734 A debugging check. If this is the previous node, its next field should point to the VMA being returned. If it is not, it is a bug.

D.3.1.3 Function: `find_vma_intersection()` (*include/linux/mm.h*)

```
673 static inline struct vm_area_struct * find_vma_intersection(
        struct mm_struct * mm,
        unsigned long start_addr, unsigned long end_addr)
674 {
675     struct vm_area_struct * vma = find_vma(mm, start_addr);
676
677     if (vma && end_addr <= vma->vm_start)
678         vma = NULL;
679     return vma;
680 }
```

675 Returns the VMA closest to the starting address.

677 If a VMA is returned and the end address is still less than the beginning of the returned VMA, the VMA does not intersect.

679 Returns the VMA if it does intersect.

D.3.2 Finding a Free Memory Region

D.3.2.1 Function: `get_unmapped_area()` (*mm/mmap.c*)

The call graph for this function is shown in Figure 4.4.

```
644 unsigned long get_unmapped_area(struct file *file,
        unsigned long addr,
        unsigned long len,
        unsigned long pgoff,
        unsigned long flags)
645 {
646     if (flags & MAP_FIXED) {
647         if (addr > TASK_SIZE - len)
648             return -ENOMEM;
649         if (addr & ~PAGE_MASK)
650             return -EINVAL;
651         return addr;
652     }
653
654     if (file && file->f_op && file->f_op->get_unmapped_area)
655         return file->f_op->get_unmapped_area(file, addr,
            len, pgoff, flags);
656 }
```

```

657     return arch_get_unmapped_area(file, addr, len, pgoff, flags);
658 }

```

644 The parameters passed are the following:

- **file** The file or device being mapped
- **addr** The requested address to map to
- **len** The length of the mapping
- **pgoff** The offset within the file being mapped
- **flags** Protection flags

646-652 A sanity check. If it is required that the mapping be placed at the specified address, this makes sure it will not overflow the address space and that it is page aligned.

654 If the `struct file` provides a `get_unmapped_area()` function, this uses it.

657 Uses `arch_get_unmapped_area()` (See Section D.3.2.2) as an anonymous version of the `get_unmapped_area()` function.

D.3.2.2 Function: `arch_get_unmapped_area()` (*mm/mmap.c*)

Architectures have the option of specifying this function for themselves by defining `HAVE_ARCH_UNMAPPED_AREA`. If the architectures do not supply one, this version is used.

```

614 #ifndef HAVE_ARCH_UNMAPPED_AREA
615 static inline unsigned long arch_get_unmapped_area(
        struct file *filp,
        unsigned long addr, unsigned long len,
        unsigned long pgoff, unsigned long flags)
616 {
617     struct vm_area_struct *vma;
618
619     if (len > TASK_SIZE)
620         return -ENOMEM;
621
622     if (addr) {
623         addr = PAGE_ALIGN(addr);
624         vma = find_vma(current->mm, addr);
625         if (TASK_SIZE - len >= addr &&
626             (!vma || addr + len <= vma->vm_start))
627             return addr;
628     }
629     addr = PAGE_ALIGN(TASK_UNMAPPED_BASE);
630
631     for (vma = find_vma(current->mm, addr); ; vma = vma->vm_next) {

```

Process Address Space

```

632      /* At this point: (!vma || addr < vma->vm_end). */
633      if (TASK_SIZE - len < addr)
634          return -ENOMEM;
635      if (!vma || addr + len <= vma->vm_start)
636          return addr;
637      addr = vma->vm_end;
638  }
639 }
640 #else
641 extern unsigned long arch_get_unmapped_area(struct file *,
        unsigned long, unsigned long,
        unsigned long, unsigned long);
642 #endif

```

614 If this is not defined, it means that the architecture does not provide its own `arch_get_unmapped_area()`, so this one is used instead.

615 The parameters are the same as those for `get_unmapped_area()` (See Section D.3.2.1).

619-620 A sanity check to make sure the required map length is not too long.

622-628 If an address is provided, this uses it for the mapping.

623 Makes sure the address is page aligned.

624 `find_vma()` (See Section D.3.1.1) will return the region closest to the requested address.

625-627 Makes sure the mapping will not overlap with another region. If it does not, it returns it because it is safe to use. Otherwise, it gets ignored.

629 `TASK_UNMAPPED_BASE` is the starting point for searching for a free region to use.

631-638 Starting from `TASK_UNMAPPED_BASE`, this linearly searches the VMAs until a large enough region between them is found to store the new mapping. This is essentially a first fit search.

641 If an external function is provided, it still needs to be declared here.

D.4 Locking and Unlocking Memory Regions

Contents

D.4 Locking and Unlocking Memory Regions	315
D.4.1 Locking a Memory Region	315
D.4.1.1 Function: <code>sys_mlock()</code>	315
D.4.1.2 Function: <code>sys_mlockall()</code>	316
D.4.1.3 Function: <code>do_mlockall()</code>	318
D.4.1.4 Function: <code>do_mlock()</code>	319
D.4.2 Unlocking the Region	321
D.4.2.1 Function: <code>sys_munlock()</code>	321
D.4.2.2 Function: <code>sys_munlockall()</code>	321
D.4.3 Fixing Up Regions After Locking/Unlocking	322
D.4.3.1 Function: <code>mlock_fixup()</code>	322
D.4.3.2 Function: <code>mlock_fixup_all()</code>	323
D.4.3.3 Function: <code>mlock_fixup_start()</code>	324
D.4.3.4 Function: <code>mlock_fixup_end()</code>	325
D.4.3.5 Function: <code>mlock_fixup_middle()</code>	326

This section contains the functions related to locking and unlocking a region. The main complexity in them is how the regions need to be fixed up after the operation takes place.

D.4.1 Locking a Memory Region

D.4.1.1 Function: `sys_mlock()` (*mm/mlock.c*)

The call graph for this function is shown in Figure 4.9. This is the system call `mlock()` for locking a region of memory into physical memory. This function simply checks to make sure that process and user limits are not exceeded and that the region to lock is page aligned.

```

195 asmlinkage long sys_mlock(unsigned long start, size_t len)
196 {
197     unsigned long locked;
198     unsigned long lock_limit;
199     int error = -ENOMEM;
200
201     down_write(&current->mm->mmap_sem);
202     len = PAGE_ALIGN(len + (start & ~PAGE_MASK));
203     start &= PAGE_MASK;
204
205     locked = len >> PAGE_SHIFT;
206     locked += current->mm->locked_vm;
207
208     lock_limit = current->rlim[RLIMIT_MEMLOCK].rlim_cur;
209     lock_limit >>= PAGE_SHIFT;
210
211     /* check against resource limits */

```

Process Address
Space

```

212     if (locked > lock_limit)
213         goto out;
214
215     /* we may lock at most half of physical memory... */
216     /* (this check is pretty bogus, but doesn't hurt) */
217     if (locked > num_physpages/2)
218         goto out;
219
220     error = do_mlock(start, len, 1);
221 out:
222     up_write(&current->mm->mmap_sem);
223     return error;
224 }

```

201 Takes the semaphore. We are likely to sleep during this, so a spinlock cannot be used.

202 Rounds the length up to the page boundary.

203 Rounds the start address down to the page boundary.

205 Calculates how many pages will be locked.

206 Calculates how many pages will be locked in total by this process.

208-209 Calculates what the limit is to the number of locked pages.

212-213 Does not allow the process to lock more than it should.

217-218 Does not allow the process to map more than half of physical memory.

220 Calls `do_mlock()` (See Section D.4.1.4), which starts the real work by finding the VMA closest to the area to lock before calling `mlock_fixup()` (See Section D.4.3.1).

222 Frees the semaphore.

223 Returns the error or success code from `do_mlock()`.

D.4.1.2 Function: `sys_mlockall()` (*mm/mlock.c*)

This is the system call `mlockall()`, which attempts to lock all pages in the calling process in memory. If `MCL_CURRENT` is specified, all current pages will be locked. If `MCL_FUTURE` is specified, all future mappings will be locked. The flags may be or-ed together. This function makes sure that the flags and process limits are ok before calling `do_mlockall()`.

```

266 asmlinkage long sys_mlockall(int flags)
267 {
268     unsigned long lock_limit;

```

```

269     int ret = -EINVAL;
270
271     down_write(&current->mm->mmap_sem);
272     if (!flags || (flags & ~(MCL_CURRENT | MCL_FUTURE)))
273         goto out;
274
275     lock_limit = current->rlim[RLIMIT_MEMLOCK].rlim_cur;
276     lock_limit >>= PAGE_SHIFT;
277
278     ret = -ENOMEM;
279     if (current->mm->total_vm > lock_limit)
280         goto out;
281
282     /* we may lock at most half of physical memory... */
283     /* (this check is pretty bogus, but doesn't hurt) */
284     if (current->mm->total_vm > num_physpages/2)
285         goto out;
286
287     ret = do_mlockall(flags);
288 out:
289     up_write(&current->mm->mmap_sem);
290     return ret;
291 }

```

269 By default, this returns `-EINVAL` to indicate invalid parameters.

271 Acquires the current `mm_struct` semaphore.

272-273 Makes sure that some valid flag has been specified. If not, it uses `goto out` to unlock the semaphore and returns `-EINVAL`.

275-276 Checks the process limits to see how many pages may be locked.

278 From here on, the default error is `-ENOMEM`.

279-280 If the size of the locking would exceed set limits, then it uses `goto out`.

284-285 Do not allow this process to lock more than half of physical memory. This is a bogus check because four processes locking a quarter of physical memory each will bypass this. It is acceptable though because only root processes are allowed to lock memory and are unlikely to make this type of mistake.

287 Calls the core function `do_mlockall()` (See Section D.4.1.3).

289-290 Unlocks the semaphore and returns.

D.4.1.3 Function: do_mlockall() (*mm/mlock.c*)

```

238 static int do_mlockall(int flags)
239 {
240     int error;
241     unsigned int def_flags;
242     struct vm_area_struct * vma;
243
244     if (!capable(CAP_IPC_LOCK))
245         return -EPERM;
246
247     def_flags = 0;
248     if (flags & MCL_FUTURE)
249         def_flags = VM_LOCKED;
250     current->mm->def_flags = def_flags;
251
252     error = 0;
253     for (vma = current->mm->mmap; vma ; vma = vma->vm_next) {
254         unsigned int newflags;
255
256         newflags = vma->vm_flags | VM_LOCKED;
257         if (!(flags & MCL_CURRENT))
258             newflags &= ~VM_LOCKED;
259         error = mlock_fixup(vma, vma->vm_start, vma->vm_end,
                             newflags);
260         if (error)
261             break;
262     }
263     return error;
264 }

```

244-245 The calling process must be either root or have `CAP_IPC_LOCK` capabilities.

248-250 The `MCL_FUTURE` flag says that all future pages should be locked, so, if set, the `def_flags` for VMAs should be `VM_LOCKED`.

253-262 Cycles through all VMAs.

256 Sets the `VM_LOCKED` flag in the current VMA flags.

257-258 If the `MCL_CURRENT` flag has not been set requesting that all current pages be locked, then this clears the `VM_LOCKED` flag. The logic is arranged like this so that the unlock code can use this same function, just with no flags.

259 Calls `mlock_fixup()` (See Section D.4.3.1), which will adjust the regions to match the locking as necessary.

260-261 If a nonzero value is returned at any point, this stops locking. It is interesting to note that VMAs already locked will not be unlocked.

263 Returns the success or error value.

D.4.1.4 Function: `do_mlock()` (*mm/mlock.c*)

This function is responsible for starting the work needed to either lock or unlock a region, depending on the value of the `on` parameter. It is broken up into two sections. The first makes sure the region is page aligned (despite the fact the only two callers of this function do the same thing) before finding the VMA that is to be adjusted. The second part then sets the appropriate flags before calling `mlock_fixup()` for each VMA that is affected by this locking.

```

148 static int do_mlock(unsigned long start, size_t len, int on)
149 {
150     unsigned long nstart, end, tmp;
151     struct vm_area_struct * vma, * next;
152     int error;
153
154     if (on && !capable(CAP_IPC_LOCK))
155         return -EPERM;
156     len = PAGE_ALIGN(len);
157     end = start + len;
158     if (end < start)
159         return -EINVAL;
160     if (end == start)
161         return 0;
162     vma = find_vma(current->mm, start);
163     if (!vma || vma->vm_start > start)
164         return -ENOMEM;

```

This block page aligns the request and finds the VMA.

154 Only root processes can lock pages.

156 Page aligns the length. This is redundant because the length is page aligned in the parent functions.

157-159 Calculates the end of the locking and makes sure it is a valid region. It returns `-EINVAL` if it is not.

160-161 If locking a region of size 0, this just returns.

162 Finds the VMA that will be affected by this locking.

163-164 If the VMA for this address range does not exist, it returns `-ENOMEM`.

```

166     for (nstart = start ; ; ) {
167         unsigned int newflags;
168
169
170

```

```

171     newflags = vma->vm_flags | VM_LOCKED;
172     if (!on)
173         newflags &= ~VM_LOCKED;
174
175     if (vma->vm_end >= end) {
176         error = mlock_fixup(vma, nstart, end, newflags);
177         break;
178     }
179
180     tmp = vma->vm_end;
181     next = vma->vm_next;
182     error = mlock_fixup(vma, nstart, tmp, newflags);
183     if (error)
184         break;
185     nstart = tmp;
186     vma = next;
187     if (!vma || vma->vm_start != nstart) {
188         error = -ENOMEM;
189         break;
190     }
191 }
192 return error;
193 }

```

This block walks through the VMAs affected by this locking and calls `mlock_fixup()` for each of them.

166-192 Cycles through as many VMAs as necessary to lock the pages.

171 Sets the `VM_LOCKED` flag on the VMA.

172-173 If this is an unlock, it removes the flag.

175-177 If this VMA is the last VMA to be affected by the unlocking, this calls `mlock_fixup()` with the end address for the locking and exits.

180-190 This is whole VMA that needs to be locked. To lock it, the end of this VMA is passed as a parameter to `mlock_fixup()` (See Section D.4.3.1) instead of the end of the actual locking.

180 `tmp` is the end of the mapping on this VMA.

181 `next` is the next VMA that will be affected by the locking.

182 Calls `mlock_fixup()` (See Section D.4.3.1) for this VMA.

183-184 If an error occurs, this backs out. Note that the VMAs already locked are not fixed up right.

185 The next start address is the start of the next VMA.

186 Moves to the next VMA.

187-190 If there is no VMA, this returns `-ENOMEM`. The next condition, though, would require the regions to be extremely broken as a result of a broken implementation of `mlock_fixup()` or have VMAs that overlap.

192 Returns the error or success value.

D.4.2 Unlocking the Region

D.4.2.1 Function: `sys_munlock()` (*mm/mlock.c*)

This page aligns the request before calling `do_mlock()`, which begins the real work of fixing up the regions.

```
226 asmlinkage long sys_munlock(unsigned long start, size_t len)
227 {
228     int ret;
229
230     down_write(&current->mm->mmap_sem);
231     len = PAGE_ALIGN(len + (start & ~PAGE_MASK));
232     start &= PAGE_MASK;
233     ret = do_mlock(start, len, 0);
234     up_write(&current->mm->mmap_sem);
235     return ret;
236 }
```

230 Acquires the semaphore protecting the `mm_struct`.

231 Rounds the length of the region up to the nearest page boundary.

232 Rounds the start of the region down to the nearest page boundary.

233 Calls `do_mlock()` (See Section D.4.1.4) with 0 as the third parameter to unlock the region.

234 Releases the semaphore.

235 Returns the success or failure code.

D.4.2.2 Function: `sys_munlockall()` (*mm/mlock.c*)

This is a trivial function. If the flags to `mlockall()` are 0, it gets translated as none of the current pages must be present and no future mappings should be locked either, which means the `VM_LOCKED` flag will be removed on all VMAs.

```
293 asmlinkage long sys_munlockall(void)
294 {
295     int ret;
296
297     down_write(&current->mm->mmap_sem);
```

```

298     ret = do_mlockall(0);
299     up_write(&current->mm->mmap_sem);
300     return ret;
301 }

```

297 Acquires the semaphore protecting the `mm_struct`.

298 Calls `do_mlockall()` (See Section D.4.1.3) with 0 as flags, which will remove the `VM_LOCKED` from all VMAs.

299 Releases the semaphore.

300 Returns the error or success code.

D.4.3 Fixing Up Regions After Locking/Unlocking

D.4.3.1 Function: `mlock_fixup()` (*mm/mlock.c*)

This function identifies four separate types of locking that must be addressed. The first is where the full VMA is to be locked, and it calls `mlock_fixup_all()`. The second is where only the beginning portion of the VMA is affected, which is handled by `mlock_fixup_start()`. The third is the locking of a region at the end, which is handled by `mlock_fixup_end()`, and the last is locking a region in the middle of the VMA with `mlock_fixup_middle()`.

```

117 static int mlock_fixup(struct vm_area_struct * vma,
118     unsigned long start, unsigned long end, unsigned int newflags)
119 {
120     int pages, retval;
121
122     if (newflags == vma->vm_flags)
123         return 0;
124
125     if (start == vma->vm_start) {
126         if (end == vma->vm_end)
127             retval = mlock_fixup_all(vma, newflags);
128         else
129             retval = mlock_fixup_start(vma, end, newflags);
130     } else {
131         if (end == vma->vm_end)
132             retval = mlock_fixup_end(vma, start, newflags);
133         else
134             retval = mlock_fixup_middle(vma, start,
135                                     end, newflags);
136     }
137     if (!retval) {
138         /* keep track of amount of locked VM */
139         pages = (end - start) >> PAGE_SHIFT;
140         if (newflags & VM_LOCKED) {

```

```

140         pages = -pages;
141         make_pages_present(start, end);
142     }
143     vma->vm_mm->locked_vm -= pages;
144 }
145 return retval;
146 }

```

122-123 If no change is to be made, this just returns.

125 If the start of the locking is at the start of the VMA, it means that either the full region is to be locked or only a portion at the beginning.

126-127 If the full VMA is being locked, this calls `mlock_fixup_all()` (See Section D.4.3.2).

128-129 If part of the VMA is being locked with the start of the VMA matching the start of the locking, this calls `mlock_fixup_start()` (See Section D.4.3.3).

130 Means that either a region at the end is to be locked or a region in the middle.

131-132 If the end of the locking matches the end of the VMA, this calls `mlock_fixup_end()` (See Section D.4.3.4).

133-134 If a region in the middle of the VMA is to be locked, this calls `mlock_fixup_middle()` (See Section D.4.3.5).

136-144 For this, the fixup functions return 0 on success. If the fixup of the regions succeed and the regions are now marked as locked, this calls `make_pages_present()`, which makes some basic checks before calling `get_user_pages()`, which faults in all the pages in the same way that the page fault handler does.

D.4.3.2 Function: `mlock_fixup_all()` (*mm/mlock.c*)

```

15 static inline int mlock_fixup_all(struct vm_area_struct * vma,
16                                   int newflags)
17 {
18     spin_lock(&vma->vm_mm->page_table_lock);
19     vma->vm_flags = newflags;
20     spin_unlock(&vma->vm_mm->page_table_lock);
21     return 0;
22 }

```

17-19 Trivial. It locks the VMA with the spinlock, sets the new flags, releases the lock and returns success.

D.4.3.3 Function: `mlock_fixup_start()` (*mm/mlock.c*)

This is slightly more complicated. A new VMA is required to represent the affected region. The start of the old VMA is moved forward.

```

23 static inline int mlock_fixup_start(struct vm_area_struct * vma,
24     unsigned long end, int newflags)
25 {
26     struct vm_area_struct * n;
27
28     n = kmem_cache_alloc(vm_area_cachep, SLAB_KERNEL);
29     if (!n)
30         return -EAGAIN;
31     *n = *vma;
32     n->vm_end = end;
33     n->vm_flags = newflags;
34     n->vm_raend = 0;
35     if (n->vm_file)
36         get_file(n->vm_file);
37     if (n->vm_ops && n->vm_ops->open)
38         n->vm_ops->open(n);
39     vma->vm_pgoff += (end - vma->vm_start) >> PAGE_SHIFT;
40     lock_vma_mappings(vma);
41     spin_lock(&vma->vm_mm->page_table_lock);
42     vma->vm_start = end;
43     __insert_vm_struct(current->mm, n);
44     spin_unlock(&vma->vm_mm->page_table_lock);
45     unlock_vma_mappings(vma);
46     return 0;
47 }
```

28 Allocates a VMA from the slab allocator for the affected region.

31-34 Copies in the necessary information.

35-36 If the VMA has a file or device mapping, `get_file()` will increment the reference count.

37-38 If an `open()` function is provided, this calls it.

39 Updates the offset within the file or device mapping for the old VMA to be the end of the locked region.

40 `lock_vma_mappings()` will lock any files if this VMA is a shared region.

41-44 Locks the parent `mm_struct`, updates its start to be the end of the affected region, inserts the new VMA into the processes linked lists (See Section D.2.2.1) and releases the lock.

45 Unlocks the file mappings with `unlock_vma_mappings()`.

46 Returns success.

D.4.3.4 Function: `mlock_fixup_end()` (*mm/mlock.c*)

This function is essentially the same as `mlock_fixup_start()` except the affected region is at the end of the VMA.

```

49 static inline int mlock_fixup_end(struct vm_area_struct * vma,
50     unsigned long start, int newflags)
51 {
52     struct vm_area_struct * n;
53
54     n = kmem_cache_alloc(vm_area_cachep, SLAB_KERNEL);
55     if (!n)
56         return -EAGAIN;
57     *n = *vma;
58     n->vm_start = start;
59     n->vm_pgoff += (n->vm_start - vma->vm_start) >> PAGE_SHIFT;
60     n->vm_flags = newflags;
61     n->vm_raend = 0;
62     if (n->vm_file)
63         get_file(n->vm_file);
64     if (n->vm_ops && n->vm_ops->open)
65         n->vm_ops->open(n);
66     lock_vma_mappings(vma);
67     spin_lock(&vma->vm_mm->page_table_lock);
68     vma->vm_end = start;
69     __insert_vm_struct(current->mm, n);
70     spin_unlock(&vma->vm_mm->page_table_lock);
71     unlock_vma_mappings(vma);
72     return 0;
73 }

```

54 Allocates a VMA from the slab allocator for the affected region.

57-61 Copies in the necessary information and updates the offset within the file or device mapping.

62-63 If the VMA has a file or device mapping, `get_file()` will increment the reference count.

64-65 If an `open()` function is provided, this calls it.

66 `lock_vma_mappings()` will lock any files if this VMA is a shared region.

67-70 Locks the parent `mm_struct`, updates its start to be the end of the affected region, inserts the new VMA into the processes linked lists (See Section D.2.2.1) and releases the lock.

71 Unlocks the file mappings with `unlock_vma_mappings()`.

72 Returns success.

D.4.3.5 Function: `mlock_fixup_middle()` (*mm/mlock.c*)

This is similar to the previous two fixup functions except that two new regions are required to fix up the mapping.

```

75 static inline int mlock_fixup_middle(struct vm_area_struct * vma,
76     unsigned long start, unsigned long end, int newflags)
77 {
78     struct vm_area_struct * left, * right;
79
80     left = kmem_cache_alloc(vm_area_cachep, SLAB_KERNEL);
81     if (!left)
82         return -EAGAIN;
83     right = kmem_cache_alloc(vm_area_cachep, SLAB_KERNEL);
84     if (!right) {
85         kmem_cache_free(vm_area_cachep, left);
86         return -EAGAIN;
87     }
88     *left = *vma;
89     *right = *vma;
90     left->vm_end = start;
91     right->vm_start = end;
92     right->vm_pgoff += (right->vm_start - left->vm_start) >>
        PAGE_SHIFT;
93     vma->vm_flags = newflags;
94     left->vm_raend = 0;
95     right->vm_raend = 0;
96     if (vma->vm_file)
97         atomic_add(2, &vma->vm_file->f_count);
98
99     if (vma->vm_ops && vma->vm_ops->open) {
100         vma->vm_ops->open(left);
101         vma->vm_ops->open(right);
102     }
103     vma->vm_raend = 0;
104     vma->vm_pgoff += (start - vma->vm_start) >> PAGE_SHIFT;
105     lock_vma_mappings(vma);
106     spin_lock(&vma->vm_mm->page_table_lock);
107     vma->vm_start = start;
108     vma->vm_end = end;
109     vma->vm_flags = newflags;
110     __insert_vm_struct(current->mm, left);
111     __insert_vm_struct(current->mm, right);

```



```
112     spin_unlock(&vma->vm_mm->page_table_lock);
113     unlock_vma_mappings(vma);
114     return 0;
115 }
```

80-87 Allocates the two new VMAs from the slab allocator.

88-89 Copies in the information from the old VMA into the new VMAs.

90 The end of the left region is the start of the region to be affected.

91 The start of the right region is the end of the affected region.

92 Updates the file offset.

93 The old VMA is now the affected region, so this updates its flags.

94-95 Makes the readahead window 0 to ensure pages not belonging to their regions are not accidentally read ahead.

96-97 Increments the reference count to the file/device mapping if there is one.

99-102 Calls the `open()` function for the two new mappings.

103-104 Cancels the readahead window and updates the offset within the file to be the beginning of the locked region.

105 Locks the shared file/device mappings.

106-112 Locks the parent `mm_struct`, updates the VMA and inserts the two new regions into the process before releasing the lock again.

113 Unlocks the shared mappings.

114 Returns success.

D.5 Page Faulting

Contents

D.5 Page Faulting	328
D.5.1 x86 Page Fault Handler	328
D.5.1.1 Function: <code>do_page_fault()</code>	328
D.5.2 Expanding the Stack	337
D.5.2.1 Function: <code>expand_stack()</code>	337
D.5.3 Architecture-Independent Page Fault Handler	338
D.5.3.1 Function: <code>handle_mm_fault()</code>	339
D.5.3.2 Function: <code>handle_pte_fault()</code>	340
D.5.4 Demand Allocation	341
D.5.4.1 Function: <code>do_no_page()</code>	341
D.5.4.2 Function: <code>do_anonymous_page()</code>	344
D.5.5 Demand Paging	346
D.5.5.1 Function: <code>do_swap_page()</code>	346
D.5.5.2 Function: <code>can_share_swap_page()</code>	350
D.5.5.3 Function: <code>exclusive_swap_page()</code>	351
D.5.6 Copy On Write (COW) Pages	352
D.5.6.1 Function: <code>do_wp_page()</code>	352

This section deals with the page fault handler. It begins with the architecture-specific function for the x86 and then moves to the architecture-independent layer. The architecture-specific functions all have the same responsibilities.

D.5.1 x86 Page Fault Handler

D.5.1.1 Function: `do_page_fault()` (*arch/i386/mm/fault.c*)

The call graph for this function is shown in Figure 4.11. This function is the x86 architecture-dependent function for the handling of page fault exception handlers. Each architecture registers its own, but all of them have similar responsibilities.

```

140 asmlinkage void do_page_fault(struct pt_regs *regs,
                                unsigned long error_code)
141 {
142     struct task_struct *tsk;
143     struct mm_struct *mm;
144     struct vm_area_struct * vma;
145     unsigned long address;
146     unsigned long page;
147     unsigned long fixup;
148     int write;
149     siginfo_t info;
150
151     /* get the address */
152     __asm__("movl %%cr2,%0":"=r" (address));
153
154     /* It's safe to allow irq's after cr2 has been saved */
155     if (regs->eflags & X86_EFLAGS_IF)

```

```

156         local_irq_enable();
157
158     tsk = current;
159

```

This is the function preamble. It gets the fault address and enables interrupts.

140 The parameters are the following:

- **regs** is a struct containing what all the registers have at fault time.
- **error_code** indicates what sort of fault occurred.

152 As the comment indicates, the cr2 register holds the fault address.

155-156 If the fault is from within an interrupt, this enables it.

158 Sets the current task.

```

173     if (address >= TASK_SIZE && !(error_code & 5))
174         goto vmalloc_fault;
175
176     mm = tsk->mm;
177     info.si_code = SEGV_MAPERR;
178
183     if (in_interrupt() || !mm)
184         goto no_context;
185

```

This block checks for exceptional faults, kernel faults, fault in interrupt and fault with no memory context.

173 If the fault address is over **TASK_SIZE**, it is within the kernel address space. If the error code is 5, it means the error happened while in kernel mode and is not a protection error, so this handles a vmalloc fault.

176 Records the working mm.

183 If this is an interrupt or there is no memory context (such as with a kernel thread), there is no way to safely handle the fault, so goto **no_context**.

```

186     down_read(&mm->mmap_sem);
187
188     vma = find_vma(mm, address);
189     if (!vma)
190         goto bad_area;
191     if (vma->vm_start <= address)
192         goto good_area;
193     if (!(vma->vm_flags & VM_GROWSDOWN))
194         goto bad_area;

```

Process Address
Space

```

195     if (error_code & 4) {
196         /*
197          * accessing the stack below %esp is always a bug.
198          * The "+ 32" is there due to some instructions (like
199          * pusha) doing post-decrement on the stack and that
200          * doesn't show up until later..
201          */
202         if (address + 32 < regs->esp)
203             goto bad_area;
204     }
205     if (expand_stack(vma, address))
206         goto bad_area;

```

If the fault is in userspace, this block finds the VMA for the faulting address and determines if it is a good area, a bad area or if the fault occurred near a region that can be expanded, such as the stack.

186 Takes the long-lived mm semaphore.

188 Finds the VMA that is responsible or is closest to the faulting address.

189-190 If a VMA does not exist at all, goto `bad_area`.

191-192 If the start of the region is before the address, it means this VMA is the correct VMA for the fault, so goto `good_area`, which will check the permissions.

193-194 For the region that is closest, this checks if it can grown down (`VM_GROWSDOWN`). If it does, it means the stack can probably be expanded. If not, goto `bad_area`.

195-204 Checks to make sure it is not an access below the stack. If the `error_code` is 4, it means it is running in userspace.

205-206 The stack is the only region with `VM_GROWSDOWN` set, so, if we reach here, the stack is expanded with `expand_stack()` (See Section D.5.2.1). If it fails, goto `bad_area`.

```

211 good_area:
212     info.si_code = SEGV_ACCERR;
213     write = 0;
214     switch (error_code & 3) {
215         default: /* 3: write, present */
216 #ifdef TEST_VERIFY_AREA
217         if (regs->cs == KERNEL_CS)
218             printk("WP fault at %08lx\n", regs->eip);
219 #endif
220         /* fall through */
221         case 2: /* write, not present */

```

```

222         if (!(vma->vm_flags & VM_WRITE))
223             goto bad_area;
224         write++;
225         break;
226     case 1:      /* read, present */
227         goto bad_area;
228     case 0:      /* read, not present */
229         if (!(vma->vm_flags & (VM_READ | VM_EXEC)))
230             goto bad_area;
231     }

```

This block is where the first part of a fault in a good area is handled. The permissions need to be checked in case this is a protection fault.

212 By default, this returns an error.

214 Checks the error code against bits 0 and 1 of the error code. Bit 0 at 0 means the page was not present. At 1, it means a protection fault, like a write to a read-only area. Bit 1 is 0 if it was a read fault and 1 if it was a write fault.

215 If it is 3, both bits are 1, so it is a write protection fault.

221 Bit 1 is a 1, so it is a write fault.

222-223 If the region cannot be written to, it is a bad write to goto `bad_area`. If the region can be written to, this is a page that is marked Copy On Write (COW).

224 Flags that a write has occurred.

226-227 This is a read, and the page is present. There is no reason for the fault, so it must be some other type of exception like a divide by zero, or goto `bad_area` where it is handled.

228-230 A read occurred on a missing page. This makes sure it is ok to read or exec this page. If not, goto `bad_area`. The check for exec is made because the x86 cannot exec protect a page and instead uses the read protect flag. This is why both have to be checked.

```

233 survive:
239     switch (handle_mm_fault(mm, vma, address, write)) {
240     case 1:
241         tsk->min_flt++;
242         break;
243     case 2:
244         tsk->maj_flt++;
245         break;
246     case 0:
247         goto do_sigbus;

```

```

248     default:
249         goto out_of_memory;
250     }
251
252     /*
253     * Did it hit the DOS screen memory VA from vm86 mode?
254     */
255     if (regs->eflags & VM_MASK) {
256         unsigned long bit = (address - 0xA0000) >> PAGE_SHIFT;
257         if (bit < 32)
258             tsk->thread.screen_bitmap |= 1 << bit;
259     }
260     up_read(&mm->mmap_sem);
261     return;

```

At this point, an attempt is going to be made to handle the fault gracefully with `handle_mm_fault()`.

239 Calls `handle_mm_fault()` with the relevant information about the fault. This is the architecture-independent part of the handler.

240-242 A return of 1 means it was a minor fault. Updates statistics.

243-245 A return of 2 means it was a major fault. Update statistics

246-247 A return of 0 means some I/O error happened during the fault, so it goes to the `do_sigbus` handler.

248-249 Any other return means memory could not be allocated for the fault, so we are out of memory. In reality, this does not happen because another function `out_of_memory()` is invoked in `mm/oom_kill.c` before this could happen, which is a function that is a lot more graceful about who it kills.

260 Releases the lock to the mm.

261 Returns because the fault has been successfully handled.

```

267 bad_area:
268     up_read(&mm->mmap_sem);
269
270     /* User mode accesses just cause a SIGSEGV */
271     if (error_code & 4) {
272         tsk->thread.cr2 = address;
273         tsk->thread.error_code = error_code;
274         tsk->thread.trap_no = 14;
275         info.si_signo = SIGSEGV;
276         info.si_errno = 0;
277         /* info.si_code has been set above */
278         info.si_addr = (void *)address;

```

```

279         force_sig_info(SIGSEGV, &info, tsk);
280         return;
281     }
282
283     /*
284      * Pentium F0 0F C7 C8 bug workaround.
285      */
286     if (boot_cpu_data.f00f_bug) {
287         unsigned long nr;
288
289         nr = (address - idt) >> 3;
290
291         if (nr == 6) {
292             do_invalid_op(regs, 0);
293             return;
294         }
295     }

```

This is the bad area handler, such as using memory with no `vm_area_struct` managing it. If the fault is not by a user process or the f00f bug, the `no_context` label is fallen through to.

271 An error code of 4 implies userspace, so it is a simple case of sending a `SIGSEGV` to kill the process.

272-274 Sets thread information about what happened, which can be read by a debugger later.

275 Records that a `SIGSEGV` signal was sent.

276 Clears `errno`, as the `SIGSEGV` is sufficient to explain the error.

278 Records the address.

279 Sends the `SIGSEGV` signal. The process will exit and dump all the relevant information.

280 Returns because the fault has been successfully handled.

286-295 A bug in the first Pentiums was called the f00f bug, which caused the processor to constantly page fault. It was used as a local DoS attack on a running Linux system. This bug was trapped within a few hours, and a patch was released. Now it results in a harmless termination of the process rather than a rebooting system.

```

296
297 no_context:
298     /* Are we prepared to handle this kernel fault? */
299     if ((fixup = search_exception_table(regs->eip)) != 0) {

```

```

300         regs->eip = fixup;
301         return;
302     }

```

299-302 Searches the exception table with `search_exception_table()` to see if this exception be handled, and, if so, it calls the proper exception handler after returning. This is really important during `copy_from_user()` and `copy_to_user()` when an exception handler is installed to trap reads and writes to invalid regions in userspace without having to make expensive checks. It means that a small fixup block of code can be called rather than falling through to the next block, which causes an oops.

```

304 /*
305  * Oops. The kernel tried to access some bad page. We'll have to
306  * terminate things with extreme prejudice.
307  */
308
309     bust_spinlocks(1);
310
311     if (address < PAGE_SIZE)
312         printk(KERN_ALERT "Unable to handle kernel NULL pointer
313                     dereference");
314     else
315         printk(KERN_ALERT "Unable to handle kernel paging
316                     request");
317     printk(" at virtual address %08lx\n", address);
318     printk(" printing eip:\n");
319     printk("%08lx\n", regs->eip);
320     asm("movl %%cr3,%0":"=r" (page));
321     page = ((unsigned long *) __va(page))[address >> 22];
322     printk(KERN_ALERT "*pde = %08lx\n", page);
323     if (page & 1) {
324         page &= PAGE_MASK;
325         address &= 0x003ff000;
326         page = ((unsigned long *)
327                 __va(page))[address >> PAGE_SHIFT];
328         printk(KERN_ALERT "*pte = %08lx\n", page);
329     }
330     die("Oops", regs, error_code);
331     bust_spinlocks(0);
332     do_exit(SIGKILL);

```

This is the `no_context` handler. Some bad exception occurred, which is going to end up in the process being terminated in all likelihood. Otherwise, the kernel faulted when it definitely should have, and an oops report is generated.

309-329 Otherwise, the kernel faulted when it really should not have, and it is a kernel bug. This block generates an oops report.

309 Forcibly frees spinlocks, which might prevent a message getting to the console.

311-312 If the address is `< PAGE_SIZE`, it means that a null pointer was used. Linux deliberately has page 0 unassigned to trap this type of fault, which is a common programming error.

313-314 Otherwise, it is just some bad kernel error, such as a driver trying to access userspace incorrectly.

315-320 Prints out information about the fault.

321-326 Prints out information about the page being faulted.

327 Dies and generates an oops report, which can be used later to get a stack trace so that a developer can see more accurately where and how the fault occurred.

329 Forcibly kills the faulting process.

```
335 out_of_memory:
336     if (tsk->pid == 1) {
337         yield();
338         goto survive;
339     }
340     up_read(&mm->mmap_sem);
341     printk("VM: killing process %s\n", tsk->comm);
342     if (error_code & 4)
343         do_exit(SIGKILL);
344     goto no_context;
```

This block is the out of memory handler. It usually ends with the faulting process getting killed unless it is **init**.

336-339 If the process is **init**, just yield and goto survive, which will try to handle the fault gracefully. **init** should never be killed.

340 Frees the mm semaphore.

341 Prints out a helpful “You are Dead” message.

342 If it is from userspace, this just kills the process.

344 If it is in kernel space, go to the **no_context** handler, which, in this case, will probably result in a kernel oops.

```
345
346 do_sigbus:
347     up_read(&mm->mmap_sem);
348
353     tsk->thread.cr2 = address;
```

```

354     tsk->thread.error_code = error_code;
355     tsk->thread.trap_no = 14;
356     info.si_signo = SIGBUS;
357     info.si_errno = 0;
358     info.si_code = BUS_ADRERR;
359     info.si_addr = (void *)address;
360     force_sig_info(SIGBUS, &info, tsk);
361
362     /* Kernel mode? Handle exceptions or die */
363     if (!(error_code & 4))
364         goto no_context;
365     return;

```

347 Frees the mm lock.

353-359 Fills in information to show a SIGBUS occurred at the faulting address so that a debugger can trap it later.

360 Sends the signal.

363-364 If in kernel mode, this tries and handles the exception during `no_context`.

365 If it is in userspace, this just returns, and the process will die in due course.

```

367 vmalloc_fault:
368     {
369         int offset = __pgd_offset(address);
370         pgd_t *pgd, *pgd_k;
371         pmd_t *pmd, *pmd_k;
372         pte_t *pte_k;
373
374         asm("movl %%cr3,%0":"=r" (pgd));
375         pgd = offset + (pgd_t *)__va(pgd);
376         pgd_k = init_mm.pgd + offset;
377
378         if (!pgd_present(*pgd_k))
379             goto no_context;
380         set_pgd(pgd, *pgd_k);
381
382         pmd = pmd_offset(pgd, address);
383         pmd_k = pmd_offset(pgd_k, address);
384         if (!pmd_present(*pmd_k))
385             goto no_context;
386         set_pmd(pmd, *pmd_k);
387
388         pte_k = pte_offset(pmd_k, address);
389         if (!pte_present(*pte_k))
390             goto no_context;

```

```

398         return;
399     }
400 }

```

This is the vmalloc fault handler. When pages are mapped in the vmalloc space, only the reference pagetable is updated. As each process references this area, a fault will be trapped, and the process pagetables will be synchronized with the reference pagetable here.

376 Gets the offset within a PGD.

381 Copies the address of the PGD for the process from the cr3 register to pgd.

382 Calculates the pgd pointer from the process PGD.

383 Calculates for the kernel reference PGD.

385-386 If the pgd entry is invalid for the kernel page table, goto `no_context`.

386 Sets the pagetable entry in the process pagetable with a copy from the kernel reference pagetable.

389-393 This is the same idea for the PMD. Copies the pagetable entry from the kernel reference pagetable to the process pagetables.

395 Checks the PTE.

396-397 If it is not present, it means the page was not valid even in the kernel reference pagetable, so goto `no_context` to handle what is probably a kernel bug or a reference to a random part of unused kernel space.

398 Returns knowing the process pagetables have been updated and are in sync with the kernel pagetables.

D.5.2 Expanding the Stack

D.5.2.1 Function: `expand_stack()` (*include/linux/mm.h*)

This function is called by the architecture-dependent page fault handler. The VMA supplied is guaranteed to be one that can grow to cover the address.

```

640 static inline int expand_stack(struct vm_area_struct * vma,
                                unsigned long address)
641 {
642     unsigned long grow;
643
644     /*
645      * vma->vm_start/vm_end cannot change under us because
646      * the caller is required
647      * to hold the mmap_sem in write mode. We need to get the
648      * spinlock only before relocating the vma range ourself.

```

Process Address
Space

```

648     */
649     address &= PAGE_MASK;
650     spin_lock(&vma->vm_mm->page_table_lock);
651     grow = (vma->vm_start - address) >> PAGE_SHIFT;
652     if (vma->vm_end - address >
        current->rlim[RLIMIT_STACK].rlim_cur ||
653     ((vma->vm_mm->total_vm + grow) << PAGE_SHIFT) >
        current->rlim[RLIMIT_AS].rlim_cur) {
654         spin_unlock(&vma->vm_mm->page_table_lock);
655         return -ENOMEM;
656     }
657     vma->vm_start = address;
658     vma->vm_pgoff -= grow;
659     vma->vm_mm->total_vm += grow;
660     if (vma->vm_flags & VM_LOCKED)
661         vma->vm_mm->locked_vm += grow;
662     spin_unlock(&vma->vm_mm->page_table_lock);
663     return 0;
664 }

```

649 Rounds the address down to the nearest page boundary.

650 Locks the pagetables spinlock.

651 Calculates how many pages the stack needs to grow by.

652 Checks to make sure that the size of the stack does not exceed the process limits.

653 Checks to make sure that the size of the address space will not exceed process limits after the stack is grown.

654-655 If either of the limits are reached, this returns `-ENOMEM`, which will cause the faulting process to segfault.

657-658 Grows the VMA down.

659 Updates the amount of address space used by the process.

660-661 If the region is locked, this updates the number of locked pages used by the process.

662-663 Unlocks the process pagetables and returns success.

D.5.3 Architecture-Independent Page Fault Handler

This is the top-level pair of functions for the architecture-independent page fault handler.

D.5.3.1 Function: `handle_mm_fault()` (*mm/memory.c*)

The call graph for this function is shown in Figure 4.13. This function allocates the PMD and PTE necessary for this new PTE that is about to be allocated. It takes the necessary locks to protect the pagetables before calling `handle_pte_fault()` to fault in the page itself.

```

1364 int handle_mm_fault(struct mm_struct *mm,
1365                     struct vm_area_struct * vma,
1366                     unsigned long address, int write_access)
1367 {
1368     pgd_t *pgd;
1369     pmd_t *pmd;
1370
1371     current->state = TASK_RUNNING;
1372     pgd = pgd_offset(mm, address);
1373
1374     /*
1375      * We need the page table lock to synchronize with kswapd
1376      * and the SMP-safe atomic PTE updates.
1377      */
1378     spin_lock(&mm->page_table_lock);
1379     pmd = pmd_alloc(mm, pgd, address);
1380
1381     if (pmd) {
1382         pte_t *pte = pte_alloc(mm, pmd, address);
1383         if (pte)
1384             return handle_pte_fault(mm, vma, address,
1385                                     write_access, pte);
1386     }
1387     spin_unlock(&mm->page_table_lock);
1388     return -1;
1389 }
```

1364 The parameters of the function are the following:

- **mm** is the `mm_struct` for the faulting process.
- **vma** is the `vm_area_struct` managing the region the fault occurred in.
- **address** is the faulting address.
- **write_access** is 1 if the fault is a write fault.

1370 Sets the current state of the process.

1371 Gets the pgd entry from the top-level pagetable.

1377 Locks the `mm_struct` because the pagetables will change.

1378 `pmd_alloc()` will allocate a `pmd_t` if one does not already exist.

1380 If the pmd has been successfully allocated, then...

1381 Allocates a PTE for this address if one does not already exist.

1382-1383 Handles the page fault with `handle_pte_fault()` (See Section D.5.3.2) and returns the status code.

1385 Failure path and unlocks the `mm_struct`.

1386 Returns -1, which will be interpreted as an out of memory condition. This is correct because this line is only reached if a PMD or PTE could not be allocated.

D.5.3.2 Function: `handle_pte_fault()` (*mm/memory.c*)

This function decides what type of fault this is and which function should handle it. `do_no_page()` is called if this is the first time a page is to be allocated. `do_swap_page()` handles the case where the page was swapped out to disk with the exception of pages swapped out from tmpfs. `do_wp_page()` breaks COW pages. If none of them are appropriate, the PTE entry is simply updated. If it was written to, it is marked dirty, and it is marked accessed to show it is a young page.

```

1331 static inline int handle_pte_fault(struct mm_struct *mm,
1332     struct vm_area_struct * vma, unsigned long address,
1333     int write_access, pte_t * pte)
1334 {
1335     pte_t entry;
1336
1337     entry = *pte;
1338     if (!pte_present(entry)) {
1339         /*
1340          * If it truly wasn't present, we know that kswapd
1341          * and the PTE updates will not touch it later. So
1342          * drop the lock.
1343          */
1344         if (pte_none(entry))
1345             return do_no_page(mm, vma, address,
1346                             write_access, pte);
1347         return do_swap_page(mm, vma, address, pte, entry,
1348                             write_access);
1349     }
1350     if (write_access) {
1351         if (!pte_write(entry))
1352             return do_wp_page(mm, vma, address, pte, entry);
1353         entry = pte_mkdirty(entry);
1354     }

```

```

1355     entry = pte_mkyoung(entry);
1356     establish_pte(vma, address, pte, entry);
1357     spin_unlock(&mm->page_table_lock);
1358     return 1;
1359 }

```

1331 The parameters of the function are the same as those for `handle_mm_fault()` except that the PTE for the fault is included.

1337 Records the PTE.

1338 Handles the case where the PTE is not present.

1344 If the PTE has never been filled, this handles the allocation of the PTE with `do_no_page()` (See Section D.5.4.1).

1346 If the page has been swapped out to backing storage, this handles it with `do_swap_page()` (See Section D.5.5.1).

1349-1354 Handles the case where the page is been written to.

1350-1351 If the PTE is marked write-only, it is a COW page, so handle it with `do_wp_page()` (See Section D.5.6.1).

1353 Otherwise, this just simply marks the page as dirty.

1355 Marks the page as accessed.

1356 `establish_pte()` copies the PTE and then updates the TLB and MMU cache. This does not copy in a new PTE, but some architectures require the TLB and MMU update.

1357 Unlocks the `mm_struct` and returns that a minor fault occurred.

D.5.4 Demand Allocation

D.5.4.1 Function: `do_no_page()` (*mm/memory.c*)

The call graph for this function is shown in Figure 4.14. This function is called the first time a page is referenced so that it may be allocated and filled with data if necessary. If it is an anonymous page, which is determined by the lack of a `vm_ops` available to the VMA or the lack of a `nopage()` function, `do_anonymous_page()` is called. Otherwise, the supplied `nopage()` function is called to allocate a page, and it is inserted into the pagetables here. The function has the following tasks:

- Check if `do_anonymous_page()` should be used, and, if so, call it and return the page it allocates. If not, call the supplied `nopage()` function and ensure it allocates a page successfully.
- Break COW early if appropriate.

- Add the page to the pagetable entries and call the appropriate architecture-dependent hooks.

```

1245 static int do_no_page(struct mm_struct * mm,
1246     struct vm_area_struct * vma,
1247     unsigned long address, int write_access, pte_t *page_table)
1248 {
1249     struct page * new_page;
1250     pte_t entry;
1251     if (!vma->vm_ops || !vma->vm_ops->nopage)
1252         return do_anonymous_page(mm, vma, page_table,
1253             write_access, address);
1254     spin_unlock(&mm->page_table_lock);
1255     new_page = vma->vm_ops->nopage(vma, address & PAGE_MASK, 0);
1256     if (new_page == NULL) /* no page was available -- SIGBUS */
1257         return 0;
1258     if (new_page == NOPAGE_OOM)
1259         return -1;
1260

```

1245 The parameters supplied are the same as those for `handle_pte_fault()`.

1251-1252 If no `vm_ops` is supplied or no `nopage()` function is supplied, then this calls `do_anonymous_page()` (See Section D.5.4.2) to allocate a page and return it.

1253 Otherwise, this frees the pagetable lock because the `nopage()` function cannot be called with spinlocks held.

1255 Calls the supplied `nopage` function. In the case of filesystems, this is frequently `filemap_nopage()` (See Section D.6.4.1), but will be different for each device driver.

1257-1258 If `NULL` is returned, it means some error occurred in the `nopage` function, such as an I/O error while reading from disk. In this case, 0 is returned which results in a `SIGBUS` being sent to the faulting process.

1259-1260 If `NOPAGE_OOM` is returned, the physical page allocator failed to allocate a page, and -1 is returned, which will forcibly kill the process.

```

1265     if (write_access && !(vma->vm_flags & VM_SHARED)) {
1266         struct page * page = alloc_page(GFP_HIGHUSER);
1267         if (!page) {
1268             page_cache_release(new_page);
1269             return -1;
1270         }

```



```

1271     copy_user_highpage(page, new_page, address);
1272     page_cache_release(new_page);
1273     lru_cache_add(page);
1274     new_page = page;
1275 }

```

This block breaks COW early in this block if appropriate. COW is broken if the fault is a write fault and the region is not shared with `VM_SHARED`. If COW was not broken in this case, a second fault would occur immediately upon return.

1265 Checks if COW should be broken early.

1266 If so, this allocates a new page for the process.

1267-1270 If the page could not be allocated, this reduces the reference count to the page returned by the `nopage()` function and returns -1 for out of memory.

1271 Otherwise, it copies the contents.

1272 Reduces the reference count to the returned page, which may still be in use by another process.

1273 Adds the new page to the LRU lists so that it may be reclaimed by `kswapd` later.

```

1277     spin_lock(&mm->page_table_lock);
1288     /* Only go through if we didn't race with anybody else... */
1289     if (pte_none(*page_table)) {
1290         ++mm->rss;
1291         flush_page_to_ram(new_page);
1292         flush_icache_page(vma, new_page);
1293         entry = mk_pte(new_page, vma->vm_page_prot);
1294         if (write_access)
1295             entry = pte_mkwwrite(pte_mkdirty(entry));
1296         set_pte(page_table, entry);
1297     } else {
1298         /* One of our sibling threads was faster, back out. */
1299         page_cache_release(new_page);
1300         spin_unlock(&mm->page_table_lock);
1301         return 1;
1302     }
1303
1304     /* no need to invalidate: a not-present page shouldn't
     * be cached
     */
1305     update_mmu_cache(vma, address, entry);
1306     spin_unlock(&mm->page_table_lock);
1307     return 2;      /* Major fault */
1308 }

```

- 1277** Locks the pagetables again because the allocations have finished and the pagetables are about to be updated.
- 1289** Checks if there is still no PTE in the entry we are about to use. If two faults hit here at the same time, it is possible another processor has already completed the page fault and that this one should be backed out.
- 1290-1297** If there is no PTE entered, this completes the fault.
- 1290** Increases the RSS count because the process is now using another page. A check really should be made here to make sure it isn't the global zero page because the RSS count could be misleading.
- 1291** As the page is about to be mapped to the process space, it is possible for some architectures that write to the page in kernel space will not be visible to the process. `flush_page_to_ram()` ensures the CPU cache will be coherent.
- 1292** `flush_icache_page()` is similar in principle except it ensures the icache and dcaches are coherent.
- 1293** Creates a `pte_t` with the appropriate permissions.
- 1294-1295** If this is a write, then this makes sure the PTE has write permissions.
- 1296** Places the new PTE in the process pagetables.
- 1297-1302** If the PTE is already filled, the page acquired from the `nopage()` function must be released.
- 1299** Decrements the reference count to the page. If it drops to 0, it will be freed.
- 1300-1301** Releases the `mm_struct` lock and returns 1 to signal this is a minor page fault because no major work had to be done for this fault because it was all done by the winner of the race.
- 1305** Updates the MMU cache for architectures that require it.
- 1306-1307** Releases the `mm_struct` lock and returns 2 to signal this is a major page fault.

D.5.4.2 Function: `do_anonymous_page()` (*mm/memory.c*)

This function allocates a new page for a process accessing a page for the first time. If it is a read access, a systemwide page containing only zeros is mapped into the process. If it is write, a zero-filled page is allocated and placed within the pagetables.

```
1190 static int do_anonymous_page(struct mm_struct * mm,
                                struct vm_area_struct * vma,
                                pte_t *page_table, int write_access,
                                unsigned long addr)
```

```

1191 {
1192     pte_t entry;
1193
1194     /* Read-only mapping of ZERO_PAGE. */
1195     entry = pte_wrprotect(mk_pte(ZERO_PAGE(addr),
                                   vma->vm_page_prot));
1196
1197     /* ..except if it's a write access */
1198     if (write_access) {
1199         struct page *page;
1200
1201         /* Allocate our own private page. */
1202         spin_unlock(&mm->page_table_lock);
1203
1204         page = alloc_page(GFP_HIGHUSER);
1205         if (!page)
1206             goto no_mem;
1207         clear_user_highpage(page, addr);
1208
1209         spin_lock(&mm->page_table_lock);
1210         if (!pte_none(*page_table)) {
1211             page_cache_release(page);
1212             spin_unlock(&mm->page_table_lock);
1213             return 1;
1214         }
1215         mm->rss++;
1216         flush_page_to_ram(page);
1217         entry = pte_mkwrite(
1218             pte_mkdirty(mk_pte(page, vma->vm_page_prot)));
1219         lru_cache_add(page);
1220         mark_page_accessed(page);
1221     }
1222     set_pte(page_table, entry);
1223
1224     /* No need to invalidate - it was non-present before */
1225     update_mmu_cache(vma, addr, entry);
1226     spin_unlock(&mm->page_table_lock);
1227     return 1;      /* Minor fault */
1228
1229 no_mem:
1230     return -1;
1231 }

```

Process Address
Space

1190 The parameters are the same as those passed to `handle_pte_fault()` (See Section D.5.3.2).

- 1195** For read accesses, this simply maps the systemwide `empty_zero_page`, which the `ZERO_PAGE()` macro returns with the given permissions. The page is write protected so that a write to the page will result in a page fault.
- 1198-1220** If this is a write fault, it allocates a new page and zero-fills it.
- 1202** Unlocks the `mm_struct` so the allocation of a new page could sleep.
- 1204** Allocates a new page.
- 1205** If a page could not be allocated, this returns -1 to handle the OOM situation.
- 1207** Zero-fills the page.
- 1209** Reacquires the lock because the pagetables are to be updated.
- 1215** Updates the RSS for the process. Note that the RSS is not updated if it is the global zero page being mapped as is the case with the read-only fault at line 1195.
- 1216** Ensures the cache is coherent.
- 1217** Marks the PTE writable and dirty because it has been written to.
- 1218** Adds the page to the LRU list so that it may be reclaimed by the swapper later.
- 1219** Marks the page accessed, which ensures the page is marked hot and on the top of the active list.
- 1222** Fixes the PTE in the pagetables for this process.
- 1225** Updates the MMU cache if the architecture needs it.
- 1226** Frees the pagetable lock.
- 1227** Returns as a minor fault. Even though it is possible the page allocator spent time writing out pages, data did not have to be read from disk to fill this page.

D.5.5 Demand Paging

D.5.5.1 Function: `do_swap_page()` (*mm/memory.c*)

The call graph for this function is shown in Figure 4.15. This function handles the case where a page has been swapped out. A swapped-out page may exist in the swap cache if it is shared between a number of processes or recently swapped in during readahead. This function is broken up into three parts:

- Search for the page in swap cache.
- If it does not exist, call `swapin_readahead()` to read in the page.

- Insert the page into the process pagetables.

```

1117 static int do_swap_page(struct mm_struct * mm,
1118     struct vm_area_struct * vma, unsigned long address,
1119     pte_t * page_table, pte_t orig_pte, int write_access)
1120 {
1121     struct page *page;
1122     swp_entry_t entry = pte_to_swp_entry(orig_pte);
1123     pte_t pte;
1124     int ret = 1;
1125
1126     spin_unlock(&mm->page_table_lock);
1127     page = lookup_swap_cache(entry);

```

This block is a function preamble. It checks for the page in the swap cache.

1117-1119 The parameters are the same as those supplied to `handle_pte_fault()` (See Section D.5.3.2).

1122 Gets the swap entry information from the PTE.

1126 Frees the `mm_struct` spinlock.

1127 Looks up the page in the swap cache.

```

1128     if (!page) {
1129         swpin_readahead(entry);
1130         page = read_swap_cache_async(entry);
1131         if (!page) {
1132             int retval;
1133             spin_lock(&mm->page_table_lock);
1134             retval = pte_same(*page_table, orig_pte) ? -1 : 1;
1135             spin_unlock(&mm->page_table_lock);
1136             return retval;
1137         }
1138
1139         /* Had to read the page from swap area: Major fault */
1140         ret = 2;
1141     }
1142
1143
1144
1145 }

```

If the page did not exist in the swap cache, then this block reads it from backing storage with `swpin_readahead()`, which reads in the requested pages and a number of pages after it. After it completes, `read_swap_cache_async()` should be able to return the page.

1128-1145 This block is executed if the page was not in the swap cache.

1129 `swpin_readahead()` (See Section D.6.6.1) reads in the requested page and a number of pages after it. The number of pages read in is determined by the

`page_cluster` variable in `mm/swap.c`, which is initialized to 2 on machines with less than 16MiB of memory and 3 otherwise. $2^{\text{page_cluster}}$ pages are read in after the requested page unless a bad or empty page entry is encountered.

1130 `read_swap_cache_async()` (See Section K.3.1.1) will look up the requested page and read it from disk if necessary.

1131-1141 If the page does not exist, there was another fault that swapped in this page and removed it from the cache while spinlocks were dropped.

1137 Locks the `mm_struct`.

1138 Compares the two PTEs. If they do not match, -1 is returned to signal an I/O error. If not, 1 is returned to mark a minor page fault because a disk access was not required for this particular page.

1139-1140 Frees the `mm_struct` and returns the status.

1144 The disk had to be accessed to mark that this is a major page fault.

```

1147     mark_page_accessed(page);
1148
1149     lock_page(page);
1150
1151     /*
1152      * Back out if somebody else faulted in this pte while we
1153      * released the page table lock.
1154      */
1155     spin_lock(&mm->page_table_lock);
1156     if (!pte_same(*page_table, orig_pte)) {
1157         spin_unlock(&mm->page_table_lock);
1158         unlock_page(page);
1159         page_cache_release(page);
1160         return 1;
1161     }
1162
1163     /* The page isn't present yet, go ahead with the fault. */
1164
1165     swap_free(entry);
1166     if (vm_swap_full())
1167         remove_exclusive_swap_page(page);
1168
1169     mm->rss++;
1170     pte = mk_pte(page, vma->vm_page_prot);
1171     if (write_access && can_share_swap_page(page))
1172         pte = pte_mkdirty(pte_mkwrite(pte));
1173     unlock_page(page);
1174
```

```
1175     flush_page_to_ram(page);
1176     flush_icache_page(vma, page);
1177     set_pte(page_table, pte);
1178
1179     /* No need to invalidate - it was non-present before */
1180     update_mmu_cache(vma, address, pte);
1181     spin_unlock(&mm->page_table_lock);
1182     return ret;
1183 }
```

This block places the page in the process pagetables.

1147 `mark_page_accessed()` (See Section J.2.3.1) will mark the page as active so that it will be moved to the top of the active LRU list.

1149 Locks the page, which has the side effect of waiting for the I/O swapping in the page to complete.

1155-1161 If someone else faulted in the page before we could, the reference to the page is dropped, the lock is freed and this returns that this was a minor fault.

1165 The function `swap_free()` (See Section K.2.2.1) reduces the reference to a swap entry. If it drops to 0, it is actually freed.

1166-1167 Page slots in swap space are reserved for the same page after they have been swapped out to avoid having to search for a free slot each time. If the swap space is full, though, the reservation is broken, and the slot freed up for another page.

1169 The page is now going to be used, so this increments the `mm_structs` RSS count.

1170 Makes a PTE for this page.

1171 If the page is being written to and is not shared between more than one process, this marks it dirty so that it will be kept in sync with the backing storage and swap cache for other processes.

1173 Unlocks the page.

1175 As the page is about to be mapped to the process space, it is possible for some architectures that write to the page in kernel space that it will not be visible to the process. `flush_page_to_ram()` ensures the cache will be coherent.

1176 `flush_icache_page()` is similar in principle except it ensures the icache and dcaches are coherent.

1177 Sets the PTE in the process pagetables.

1180 Updates the MMU cache if the architecture requires it.

1181-1182 Unlocks the `mm_struct` and returns whether it was a minor or major page fault.

D.5.5.2 Function: `can_share_swap_page()` (*mm/swapfile.c*)

This function determines if the swap cache entry for this page may be used or not. It may be used if there is no other references to it. Most of the work is performed by `exclusive_swap_page()`, but this function first makes a few basic checks to avoid having to acquire too many locks.

```

259 int can_share_swap_page(struct page *page)
260 {
261     int retval = 0;
262
263     if (!PageLocked(page))
264         BUG();
265     switch (page_count(page)) {
266     case 3:
267         if (!page->buffers)
268             break;
269         /* Fallthrough */
270     case 2:
271         if (!PageSwapCache(page))
272             break;
273         retval = exclusive_swap_page(page);
274         break;
275     case 1:
276         if (PageReserved(page))
277             break;
278         retval = 1;
279     }
280     return retval;
281 }
```

263-264 This function is called from the fault path, and the page must be locked.

265 Switch is based on the number of references.

266-268 If the count is 3, but there are no buffers associated with it, there is more than one process using the page. Buffers may be associated for just one process if the page is backed by a swap file instead of a partition.

270-273 If the count is only two, but it is not a member of the swap cache, then it has no slot that may be shared, so it returns false. Otherwise, it performs a full check with `exclusive_swap_page()` (See Section D.5.5.3).

276-277 If the page is reserved, it is the global `ZERO_PAGE`, so it cannot be shared. Otherwise, this page is definitely the only one.

D.5.5.3 Function: `exclusive_swap_page()` (*mm/swapfile.c*)

This function checks if the process is the only user of a locked swap page.

```

229 static int exclusive_swap_page(struct page *page)
230 {
231     int retval = 0;
232     struct swap_info_struct * p;
233     swp_entry_t entry;
234
235     entry.val = page->index;
236     p = swap_info_get(entry);
237     if (p) {
238         /* Is the only swap cache user the cache itself? */
239         if (p->swap_map[SWP_OFFSET(entry)] == 1) {
240             /* Recheck the page count with the pagecache
241              * lock held.. */
242             spin_lock(&pagecache_lock);
243             if (page_count(page) - !!page->buffers == 2)
244                 retval = 1;
245             spin_unlock(&pagecache_lock);
246         }
247         swap_info_put(p);
248     }
249     return retval;

```

231 By default, this returns false.

235 The `swp_entry_t` for the page is stored in `page->index` as explained in Section 2.5.

236 Gets the `swap_info_struct` with `swap_info_get()` (See Section K.2.3.1).

237-247 If a slot exists, this checks if we are the exclusive user and returns true if we are.

239 Checks if the slot is only being used by the cache itself. If it is, the page count needs to be checked again with the `pagecache_lock` held.

242-243 `!!page->buffers` will evaluate to 1 if there buffers are present, so this block effectively checks if the process is the only user of the page. If it is, `retval` is set to 1 so that true will be returned.

246 Drops the reference to the slot that was taken with `swap_info_get()` (See Section K.2.3.1).

D.5.6 Copy On Write (COW) Pages

D.5.6.1 Function: `do_wp_page()` (*mm/memory.c*)

The call graph for this function is shown in Figure 4.16. This function handles the case where a user tries to write to a private page shared among processes, such as what happens after `fork()`. Basically what happens is a page is allocated, the contents are copied to the new page and the shared count is decremented in the old page.

```

948 static int do_wp_page(struct mm_struct *mm,
                        struct vm_area_struct * vma,
949                        unsigned long address, pte_t *page_table, pte_t pte)
950 {
951     struct page *old_page, *new_page;
952
953     old_page = pte_page(pte);
954     if (!VALID_PAGE(old_page))
955         goto bad_wp_page;
956
```

948-950 The parameters are the same as those supplied to `handle_pte_fault()`.

953-955 Gets a reference to the current page in the PTE and makes sure it is valid.

```

957     if (!TryLockPage(old_page)) {
958         int reuse = can_share_swap_page(old_page);
959         unlock_page(old_page);
960         if (reuse) {
961             flush_cache_page(vma, address);
962             establish_pte(vma, address, page_table,
                           pte_mkyoung(pte_mkdirty(pte_mkwrite(pte))));
963             spin_unlock(&mm->page_table_lock);
964             return 1;          /* Minor fault */
965         }
966     }
```

957 First tries to lock the page. If 0 is returned, it means the page was previously unlocked.

958 If we managed to lock it, this calls `can_share_swap_page()` (See Section D.5.5.2) to see if we are the exclusive user of the swap slot for this page. If we are, it means that we are the last process to break COW and that we can simply use this page rather than allocating a new one.

960-965 If we are the only users of the swap slot, it means we are the only user of this page and are the last process to break COW. Therefore, the PTE is simply re-established, and we return a minor fault.

```

968     /*
969      * Ok, we need to copy. Oh, well..
970      */
971     page_cache_get(old_page);
972     spin_unlock(&mm->page_table_lock);
973
974     new_page = alloc_page(GFP_HIGHUSER);
975     if (!new_page)
976         goto no_mem;
977     copy_cow_page(old_page, new_page, address);
978

```

971 We need to copy this page, so it first gets a reference to the old page so that it doesn't disappear before we are finished with it.

972 Unlocks the spinlock as we are about to call `alloc_page()` (See Section F.2.1), which may sleep.

974-976 Allocates a page and makes sure one was returned.

977 No prizes for guessing what this function does. If the page being broken is the global zero page, `clear_user_highpage()` will be used to zero out the contents of the page. Otherwise, `copy_user_highpage()` copies the actual contents.

```

982     spin_lock(&mm->page_table_lock);
983     if (pte_same(*page_table, pte)) {
984         if (PageReserved(old_page))
985             ++mm->rss;
986         break_cow(vma, new_page, address, page_table);
987         lru_cache_add(new_page);
988
989         /* Free the old page.. */
990         new_page = old_page;
991     }
992     spin_unlock(&mm->page_table_lock);
993     page_cache_release(new_page);
994     page_cache_release(old_page);
995     return 1;          /* Minor fault */

```

982 The pagetable lock was released for `alloc_page()` (See Section F.2.1), so this reacquires it.

983 Makes sure the PTE has not changed in the meantime, which could have happened if another fault occurred while the spinlock was released.

984-985 The RSS is only updated if `PageReserved()` is true, which will only happen if the page being faulted is the global `ZERO_PAGE`, which is not accounted

for in the RSS. If this was a normal page, the process would be using the same number of physical frames after the fault as it was before, but, against the zero page, it will be using a new frame, so `rss++` reflects the use of a new page.

986 `break_cow()` is responsible for calling the architecture hooks to ensure the CPU cache and TLBs are up to date and then establishes the new page into the PTE. It first calls `flush_page_to_ram()`, which must be called when a `struct page` is about to be placed in userspace. Next is `flush_cache_page()`, which flushes the page from the CPU cache. Last is `establish_pte()`, which establishes the new page into the PTE.

987 Adds the page to the LRU lists.

992 Releases the spinlock.

993-994 Drops the references to the pages.

995 Returns a minor fault.

```

996
997 bad_wp_page:
998     spin_unlock(&mm->page_table_lock);
999     printk("do_wp_page: bogus page at address %08lx (page 0x%lx)\n",
            address, (unsigned long)old_page);
1000     return -1;
1001 no_mem:
1002     page_cache_release(old_page);
1003     return -1;
1004 }
```

997-1000 This is a false COW break, which will only happen with a buggy kernel. It prints out an informational message and returns.

1001-1003 The page allocation failed, so this releases the reference to the old page and returns -1.

D.6 Page-Related Disk I/O

Contents

D.6 Page-Related Disk I/O	355
D.6.1 Generic File Reading	355
D.6.1.1 Function: <code>generic_file_read()</code>	355
D.6.1.2 Function: <code>do_generic_file_read()</code>	358
D.6.1.3 Function: <code>generic_file_readahead()</code>	365
D.6.2 Generic File <code>mmap()</code>	369
D.6.2.1 Function: <code>generic_file_mmap()</code>	369
D.6.3 Generic File Truncation	370
D.6.3.1 Function: <code>vmtruncate()</code>	370
D.6.3.2 Function: <code>vmtruncate_list()</code>	372
D.6.3.3 Function: <code>zap_page_range()</code>	373
D.6.3.4 Function: <code>zap_pmd_range()</code>	375
D.6.3.5 Function: <code>zap_pte_range()</code>	376
D.6.3.6 Function: <code>truncate_inode_pages()</code>	377
D.6.3.7 Function: <code>truncate_list_pages()</code>	378
D.6.3.8 Function: <code>truncate_complete_page()</code>	380
D.6.3.9 Function: <code>do_flushpage()</code>	381
D.6.3.10 Function: <code>truncate_partial_page()</code>	381
D.6.4 Reading Pages for the Page Cache	382
D.6.4.1 Function: <code>filemap_nopage()</code>	382
D.6.4.2 Function: <code>page_cache_read()</code>	387
D.6.5 File Readahead for <code>nopage()</code>	388
D.6.5.1 Function: <code>nopage_sequential_readahead()</code>	388
D.6.5.2 Function: <code>read_cluster_nonblocking()</code>	390
D.6.6 Swap Related Readahead	391
D.6.6.1 Function: <code>swpin_readahead()</code>	391
D.6.6.2 Function: <code>valid_swaphandles()</code>	392

D.6.1 Generic File Reading

This is more the domain of the I/O manager than the VM, but, because it performs the operations through the page cache, we will cover it briefly. The operation of `generic_file_write()` is essentially the same, although it is not covered by this book. However, if you understand how the read takes place, the write function will pose no problem to you.

D.6.1.1 Function: `generic_file_read()` (*mm/filemap.c*)

This is the generic file read function used by any filesystem that reads pages through the page cache. For normal I/O, it is responsible for building a `read_descriptor_t` for use with `do_generic_file_read()` and `file_read_actor()`. For direct I/O, this function is basically a wrapper around `generic_file_direct_IO()`.

```
1695 ssize_t generic_file_read(struct file * filp,
                             char * buf, size_t count,
                             loff_t *ppos)
```

```

1696 {
1697     ssize_t retval;
1698
1699     if ((ssize_t) count < 0)
1700         return -EINVAL;
1701
1702     if (filp->f_flags & O_DIRECT)
1703         goto o_direct;
1704
1705     retval = -EFAULT;
1706     if (access_ok(VERIFY_WRITE, buf, count)) {
1707         retval = 0;
1708
1709         if (count) {
1710             read_descriptor_t desc;
1711
1712             desc.written = 0;
1713             desc.count = count;
1714             desc.buf = buf;
1715             desc.error = 0;
1716             do_generic_file_read(filp, ppos, &desc,
                                   file_read_actor);
1717
1718             retval = desc.written;
1719             if (!retval)
1720                 retval = desc.error;
1721         }
1722     }
1723 out:
1724     return retval;

```

This block is concerned with normal file I/O.

1702-1703 If this is direct I/O, it jumps to the `o_direct` label.

1706 If the access permissions to write to a userspace page are ok, then this proceeds.

1709 If the `count` is 0, there is no I/O to perform.

1712-1715 Populates a `read_descriptor_t` structure, which will be used by `file_read_actor()` (See Section L.3.2.3).

1716 Performs the file read.

1718 Extracts the number of bytes written from the read descriptor struct.

1719-1720 If an error occurred, this extracts what the error was.

1724 Returns either the number of bytes read or the error that occurred.

```

1725
1726 o_direct:
1727     {
1728         loff_t pos = *ppos, size;
1729         struct address_space *mapping =
1730             filp->f_dentry->d_inode->i_mapping;
1731         struct inode *inode = mapping->host;
1732         retval = 0;
1733         if (!count)
1734             goto out; /* skip atime */
1735         down_read(&inode->i_alloc_sem);
1736         down(&inode->i_sem);
1737         size = inode->i_size;
1738         if (pos < size) {
1739             retval = generic_file_direct_IO(READ, filp, buf,
1740                                     count, pos);
1741             if (retval > 0)
1742                 *ppos = pos + retval;
1743         }
1744         UPDATE_ATIME(filp->f_dentry->d_inode);
1745         goto out;
1746     }

```

This block is concerned with direct I/O. It is largely responsible for extracting the parameters required for `generic_file_direct_IO()`.

1729 Gets the `address_space` used by this `struct file`.

1733-1734 If no I/O has been requested, this jumps out to avoid updating the inodes' access time.

1737 Gets the size of the file.

1738-1739 If the current position is before the end of the file, the read is safe, so this calls `generic_file_direct_IO()`.

1740-1741 If the read was successful, this updates the current position in the file for the reader.

1743 Updates the access time.

1744 Goto out, which just returns `retval`.

D.6.1.2 Function: do_generic_file_read() (*mm/filemap.c*)

This is the core part of the generic file read operation. It is responsible for allocating a page if it doesn't already exist in the page cache. If it does, it must make sure the page is up to date, and it is responsible for making sure that the appropriate readahead window is set.

```

1349 void do_generic_file_read(struct file * filp,
                             loff_t *ppos,
                             read_descriptor_t * desc,
                             read_actor_t actor)
1350 {
1351     struct address_space *mapping =
                             filp->f_dentry->d_inode->i_mapping;
1352     struct inode *inode = mapping->host;
1353     unsigned long index, offset;
1354     struct page *cached_page;
1355     int reada_ok;
1356     int error;
1357     int max_readahead = get_max_readahead(inode);
1358
1359     cached_page = NULL;
1360     index = *ppos >> PAGE_CACHE_SHIFT;
1361     offset = *ppos & ~PAGE_CACHE_MASK;
1362
1357 Gets the maximum readahead window size for this block device.
1360 Calculates the page index, which holds the current file position pointer.
1361 Calculates the offset within the page that holds the current file position
    pointer.
1363 /*
1364  * If the current position is outside the previous read-ahead
1365  * window, we reset the current read-ahead context and set read
1366  * ahead max to zero (will be set to just needed value later),
1367  * otherwise, we assume that the file accesses are sequential
1368  * enough to continue read-ahead.
1369  */
1370     if (index > filp->f_raend ||
         index + filp->f_rawin < filp->f_raend) {
1371         reada_ok = 0;
1372         filp->f_raend = 0;
1373         filp->f_ralen = 0;
1374         filp->f_ramax = 0;
1375         filp->f_rawin = 0;
1376     } else {

```



```

1377         reada_ok = 1;
1378     }
1379 /*
1380  * Adjust the current value of read-ahead max.
1381  * If the read operation stay in the first half page, force no
1382  * readahead. Otherwise try to increase read ahead max just
1383  * enough to do the read request.
1384  * Then, at least MIN_READAHEAD if read ahead is ok,
1385  * and at most MAX_READAHEAD in all cases.
1386  */
1386     if (!index && offset + desc->count <= (PAGE_CACHE_SIZE >> 1)) {
1387         filp->f_ramax = 0;
1388     } else {
1389         unsigned long needed;
1390
1391         needed = ((offset + desc->count) >> PAGE_CACHE_SHIFT) + 1;
1392
1393         if (filp->f_ramax < needed)
1394             filp->f_ramax = needed;
1395
1396         if (reada_ok && filp->f_ramax < vm_min_readahead)
1397             filp->f_ramax = vm_min_readahead;
1398         if (filp->f_ramax > max_readahead)
1399             filp->f_ramax = max_readahead;
1400     }

```

1370-1378 As the comment suggests, the readahead window gets reset if the current file position is outside the current readahead window. It gets reset to 0 here and adjusted by `generic_file_readahead()` (See Section D.6.1.3) as necessary.

1386-1400 As the comment states, the readahead window gets adjusted slightly if we are in the second half of the current page.

```

1402     for (;;) {
1403         struct page *page, **hash;
1404         unsigned long end_index, nr, ret;
1405
1406         end_index = inode->i_size >> PAGE_CACHE_SHIFT;
1407
1408         if (index > end_index)
1409             break;
1410         nr = PAGE_CACHE_SIZE;
1411         if (index == end_index) {
1412             nr = inode->i_size & ~PAGE_CACHE_MASK;
1413             if (nr <= offset)
1414                 break;

```

```

1415     }
1416
1417     nr = nr - offset;
1418
1419     /*
1420     * Try to find the data in the page cache..
1421     */
1422     hash = page_hash(mapping, index);
1423
1424     spin_lock(&pagecache_lock);
1425     page = __find_page_nolock(mapping, index, *hash);
1426     if (!page)
1427         goto no_cached_page;

```

1402 This loop goes through each of the pages necessary to satisfy the read request.

1406 Calculates where the end of the file is in pages.

1408-1409 If the current index is beyond the end, then this breaks out because we are trying to read beyond the end of the file.

1410-1417 Calculates `nr` to be the number of bytes remaining to be read in the current page. The block takes into account that this might be the last page used by the file and where the current file position is within the page.

1422-1425 Searches for the page in the page cache.

1426-1427 If the page is not in the page cache, goto `no_cached_page` where it will be allocated.

```

1428 found_page:
1429     page_cache_get(page);
1430     spin_unlock(&pagecache_lock);
1431
1432     if (!Page_Uptodate(page))
1433         goto page_not_up_to_date;
1434     generic_file_readahead(reada_ok, filp, inode, page);

```

In this block, the page was found in the page cache.

1429 Takes a reference to the page in the page cache so it does not get freed prematurely.

1432-1433 If the page is not up to date, goto `page_not_up_to_date` to update the page with information on the disk.

1434 Performs file readahead with `generic_file_readahead()` (See Section D.6.1.3).

```

1435 page_ok:
1436     /* If users can be writing to this page using arbitrary
1437        * virtual addresses, take care about potential aliasing
1438        * before reading the page on the kernel side.
1439        */
1440     if (mapping->i_mmap_shared != NULL)
1441         flush_dcache_page(page);
1442
1443     /*
1444        * Mark the page accessed if we read the
1445        * beginning or we just did an lseek.
1446        */
1447     if (!offset || !filp->f_reada)
1448         mark_page_accessed(page);
1449
1450     /*
1451        * Ok, we have the page, and it's up-to-date, so
1452        * now we can copy it to user space...
1453        *
1454        * The actor routine returns how many bytes were actually
1455        * used.. NOTE! This may not be the same as how much of a
1456        * user buffer we filled up (we may be padding etc), so we
1457        * can only update "pos" here (the actor routine has to
1458        * update the user buffer pointers and the remaining count).
1459        */
1460     ret = actor(desc, page, offset, nr);
1461     offset += ret;
1462     index += offset >> PAGE_CACHE_SHIFT;
1463     offset &= ~PAGE_CACHE_MASK;
1464
1465     page_cache_release(page);
1466     if (ret == nr && desc->count)
1467         continue;
1468     break;

```

In this block, the page is present in the page cache and ready to be read by the file read actor function.

1440-1441 Because other users could be writing this page, call `flush_dcache_page()` to make sure the changes are visible.

1447-1448 Because the page has just been accessed, call `mark_page_accessed()` (See Section J.2.3.1) to move it to the `active_list`.

1460 Calls the actor function. In this case, the actor function is `file_read_actor()` (See Section L.3.2.3), which is responsible for copying the bytes from the page to userspace.

1461 Updates the current offset within the file.

1462 Moves to the next page if necessary.

1463 Updates the offset within the page we are currently reading. Remember that we could have just crossed into the next page in the file.

1465 Releases our reference to this page.

1466-1468 If there is still data to be read, this loops again to read the next page. Otherwise, it breaks because the read operation is complete.

```

1470 /*
1471  * Ok, the page was not immediately readable, so let's try to
1472  * read ahead while we're at it..
1473 */
1473 page_not_up_to_date:
1474     generic_file_readahead(reada_ok, filp, inode, page);
1475
1476     if (Page_Uptodate(page))
1477         goto page_ok;
1478
1479     /* Get exclusive access to the page ... */
1480     lock_page(page);
1481
1482     /* Did it get unhashed before we got the lock? */
1483     if (!page->mapping) {
1484         UnlockPage(page);
1485         page_cache_release(page);
1486         continue;
1487     }
1488
1489     /* Did somebody else fill it already? */
1490     if (Page_Uptodate(page)) {
1491         UnlockPage(page);
1492         goto page_ok;
1493     }

```

In this block, the page being read was not up to date with information on the disk. `generic_file_readahead()` is called to update the current page and readahead because I/O is required anyway.

1474 Calls `generic_file_readahead()` (See Section D.6.1.3) to sync the current page and readahead if necessary.

1476-1477 If the page is now up to date, `goto page_ok` to start copying the bytes to userspace.

1480 Otherwise, something happened with readahead, so this locks the page for exclusive access.

1483-1487 If the page was somehow removed from the page cache while spinlocks were not held, then this releases the reference to the page and starts all over again. The second time around, the page will get allocated and inserted into the page cache all over again.

1490-1493 If someone updated the page while we did not have a lock on the page, then unlock it again and goto `page_ok` to copy the bytes to userspace.

```

1495 readpage:
1496     /* ... and start the actual read. The read will
1497        * unlock the page. */
1498     error = mapping->a_ops->readpage(filp, page);
1499     if (!error) {
1500         if (Page_Uptodate(page))
1501             goto page_ok;
1502
1503         /* Again, try some read-ahead while waiting for
1504            * the page to finish.. */
1505         generic_file_readahead(reada_ok, filp, inode, page);
1506         wait_on_page(page);
1507         if (Page_Uptodate(page))
1508             goto page_ok;
1509         error = -EIO;
1510     }
1511     /* UHHUH! A synchronous read error occurred. Report it */
1512     desc->error = error;
1513     page_cache_release(page);
1514     break;

```

At this block, readahead failed to synchronously read the page with the `address_space` supplied `readpage()` function.

1497 Calls the `address_space` filesystem-specific `readpage()` function. In many cases, this will ultimately call the function `block_read_full_page()` declared in `fs/buffer.c()`.

1499-1501 If no error occurred and the page is now up to date, goto `page_ok` to begin copying the bytes to userspace.

1504 Otherwise, it schedules some readahead to occur because we are forced to wait on I/O anyway.

1505-1507 Waits for I/O on the requested page to complete. If it finished successfully, then goto `page_ok`.

1508 Otherwise, an error occurred, so this sets `-EIO` to be returned to userspace.

1512-1514 An I/O error occurred, so this records it and releases the reference to the current page. This error will be picked up from the `read_descriptor_t` struct by `generic_file_read()` (See Section D.6.1.1).

```

1516 no_cached_page:
1517     /*
1518      * Ok, it wasn't cached, so we need to create a new
1519      * page..
1520      *
1521      * We get here with the page cache lock held.
1522      */
1523     if (!cached_page) {
1524         spin_unlock(&pagecache_lock);
1525         cached_page = page_cache_alloc(mapping);
1526         if (!cached_page) {
1527             desc->error = -ENOMEM;
1528             break;
1529         }
1530
1531         /*
1532          * Somebody may have added the page while we
1533          * dropped the page cache lock. Check for that.
1534          */
1535         spin_lock(&pagecache_lock);
1536         page = __find_page_nolock(mapping, index, *hash);
1537         if (page)
1538             goto found_page;
1539     }
1540
1541     /*
1542      * Ok, add the new page to the hash-queues...
1543      */
1544     page = cached_page;
1545     __add_to_page_cache(page, mapping, index, hash);
1546     spin_unlock(&pagecache_lock);
1547     lru_cache_add(page);
1548     cached_page = NULL;
1549
1550     goto readpage;
1551 }
```

In this block, the page does not exist in the page cache, so it allocates one and adds it.

1523-1539 If a cache page has not already been allocated, then allocate one and

make sure that someone else did not insert one into the page cache while we were sleeping.

1524 Releases `pagecache_lock` because `page_cache_alloc()` may sleep.

1525-1529 Allocates a page and sets `-ENOMEM` to be returned if the allocation failed.

1535-1536 Acquires `pagecache_lock` again and searches the page cache to make sure another process has not inserted it while the lock was dropped.

1537 If another process added a suitable page to the cache already, this jumps to `found_page` because the one we just allocated is no longer necessary.

1544-1545 Otherwise, this adds the page we just allocated to the page cache.

1547 Adds the page to the LRU lists.

1548 Sets `cached_page` to `NULL` because it is now in use.

1550 Goto `readpage` to schedule the page to be read from disk.

```

1552
1553     *ppos = ((loff_t) index << PAGE_CACHE_SHIFT) + offset;
1554     filp->f_reada = 1;
1555     if (cached_page)
1556         page_cache_release(cached_page);
1557     UPDATE_ATIME(inode);
1558 }
```

1553 Updates our position within the file.

1555-1556 If a page was allocated for addition to the page cache and then found to be unneeded, it is released it here.

1557 Updates the access time to the file.

D.6.1.3 Function: `generic_file_readahead()` (*mm/filemap.c*)

This function performs generic file readahead. Readahead is one of the few areas that is very heavily commented upon in the code. It is highly recommended that you read the comments in `mm/filemap.c` marked with “Read-ahead context.”

```

1222 static void generic_file_readahead(int reada_ok,
1223     struct file * filp, struct inode * inode,
1224     struct page * page)
1225 {
1226     unsigned long end_index;
1227     unsigned long index = page->index;
1228     unsigned long max_ahead, ahead;
1229     unsigned long raend;
```

```

1230     int max_readahead = get_max_readahead(inode);
1231
1232     end_index = inode->i_size >> PAGE_CACHE_SHIFT;
1233
1234     raend = filp->f_raend;
1235     max_ahead = 0;

```

1227 Gets the index to start from based on the supplied page.

1230 Gets the maximum-sized readahead for this block device.

1232 Gets the index, in pages, of the end of the file.

1234 Gets the end of the readahead window from the struct file.

```

1236
1237 /*
1238  * The current page is locked.
1239  * If the current position is inside the previous read IO request,
1240  * do not try to reread previously read ahead pages.
1241  * Otherwise decide or not to read ahead some pages synchronously.
1242  * If we are not going to read ahead, set the read ahead context
1243  * for this page only.
1244  */
1245     if (PageLocked(page)) {
1246         if (!filp->f_ralen ||
            index >= raend ||
            index + filp->f_rawin < raend) {
1247             raend = index;
1248             if (raend < end_index)
1249                 max_ahead = filp->f_ramax;
1250             filp->f_rawin = 0;
1251             filp->f_ralen = 1;
1252             if (!max_ahead) {
1253                 filp->f_raend = index + filp->f_ralen;
1254                 filp->f_rawin += filp->f_ralen;
1255             }
1256         }
1257     }

```

This block has encountered a page that is locked, so it must decide whether to temporarily disable readahead.

1245 If the current page is locked for I/O, then check if the current page is within the last readahead window. If it is, there is no point trying to readahead again. If it is not or readahead has not been performed previously, update the readahead context.

1246 The first check is if readahead has been performed previously. The second is to see if the current locked page is after where the the previous readahead finished. The third check is if the current locked page is within the current readahead window.

1247 Updates the end of the readahead window.

1248-1249 If the end of the readahead window is not after the end of the file, this sets `max_ahead` to be the maximum amount of readahead that should be used with this `struct file(filp→f_ramax)`.

1250-1255 Sets readahead to only occur with the current page, effectively disabling readahead.

```

1258 /*
1259  * The current page is not locked.
1260  * If we were reading ahead and,
1261  * if the current max read ahead size is not zero and,
1262  * if the current position is inside the last read-ahead IO
1263  * request, it is the moment to try to read ahead asynchronously.
1264  * We will later force unplug device in order to force
1265  * asynchronous read IO.
1266  */
1267     else if (reada_ok && filp->f_ramax && raend >= 1 &&
1268             index <= raend && index + filp->f_ralen >= raend) {
1269  /*
1270  * Add ONE page to max_ahead in order to try to have about the
1271  * same IO maxsize as synchronous read-ahead
1272  * (MAX_READAHEAD + 1)*PAGE_CACHE_SIZE.
1273  * Compute the position of the last page we have tried to read
1274  * in order to begin to read ahead just at the next page.
1275  */
1276         raend -= 1;
1277         if (raend < end_index)
1278             max_ahead = filp->f_ramax + 1;
1279
1280         if (max_ahead) {
1281             filp->f_rawin = filp->f_ralen;
1282             filp->f_ralen = 0;
1283             reada_ok      = 2;
1284         }
1285     }

```

This is one of the rare cases where the in-code commentary makes the code as clear as it possibly could be. Basically, it is saying that if the current page is not locked for I/O, then it extends the readahead window slightly and remembers that readahead is currently going well.

```

1284 /*
1285  * Try to read ahead pages.
1286  * We hope that ll_rw_blk() plug/unplug, coalescence, requests
1287  * sort and the scheduler, will work enough for us to avoid too
1288  * bad actuals IO requests.
1289  */
1289     ahead = 0;
1290     while (ahead < max_ahead) {
1291         ahead ++;
1292         if ((raend + ahead) >= end_index)
1293             break;
1294         if (page_cache_read(filp, raend + ahead) < 0)
1295             break;
1296     }

```

This block performs the actual readahead by calling `page_cache_read()` for each of the pages in the readahead window. Note here how `ahead` is incremented for each page that is readahead.

```

1297 /*
1298  * If we tried to read ahead some pages,
1299  * If we tried to read ahead asynchronously,
1300  * Try to force unplug of the device in order to start an
1301  * asynchronous read IO request.
1302  * Update the read-ahead context.
1303  * Store the length of the current read-ahead window.
1304  * Double the current max read ahead size.
1305  * That heuristic avoid to do some large IO for files that are
1306  * not really accessed sequentially.
1307  */
1308     if (ahead) {
1309         filp->f_ralen += ahead;
1310         filp->f_rawin += filp->f_ralen;
1311         filp->f_raend = raend + ahead + 1;
1312
1313         filp->f_ramax += filp->f_ramax;
1314
1315         if (filp->f_ramax > max_readahead)
1316             filp->f_ramax = max_readahead;
1317
1318 #ifdef PROFILE_READAHEAD
1319         profile_readahead((reada_ok == 2), filp);
1320 #endif
1321     }
1322
1323     return;
1324 }

```

If readahead was successful, then this updates the readahead fields in the `struct file` to mark the progress. This is basically growing the readahead context, but can be reset by `do_generic_file_readahead()` if it is found that the readahead is ineffective.

1309 Updates the `f_raLEN` with the number of pages that were readahead in this pass.

1310 Updates the size of the readahead window.

1311 Marks the end of the readahead.

1313 Doubles the current maximum-sized readahead.

1315-1316 Do not let the maximum-sized readahead get larger than the maximum readahead defined for this block device.

D.6.2 Generic File `mmap()`

D.6.2.1 Function: `generic_file_mmap()` (*mm/filemap.c*)

This is the generic `mmap()` function used by many `struct files` as their `struct file_operations`. It is mainly responsible for ensuring the appropriate `address_space` functions exist and for setting what VMA operations to use.

```

2249 int generic_file_mmap(struct file * file,
                          struct vm_area_struct * vma)
2250 {
2251     struct address_space *mapping =
                          file->f_dentry->d_inode->i_mapping;
2252     struct inode *inode = mapping->host;
2253
2254     if ((vma->vm_flags & VM_SHARED) &&
        (vma->vm_flags & VM_MAYWRITE)) {
2255         if (!mapping->a_ops->writepage)
2256             return -EINVAL;
2257     }
2258     if (!mapping->a_ops->readpage)
2259         return -ENOEXEC;
2260     UPDATE_ATIME(inode);
2261     vma->vm_ops = &generic_file_vm_ops;
2262     return 0;
2263 }
```

2251 Gets the `address_space` that is managing the file being mapped.

2252 Gets the `struct inode` for this `address_space`.

2254-2257 If the VMA is to be shared and writable, this makes sure an `a_ops->writepage()` function exists. It returns `-EINVAL` if it does not.

2258-2259 Makes sure an `a_ops→readpage()` function exists.

2260 Updates the access time for the inode.

2261 Uses `generic_file_vm_ops` for the file operations. The generic VM operations structure, defined in `mm/filemap.c`, only supplies `filemap_nopage()` (See Section D.6.4.1) as its `nopage()` function. No other callback is defined.

D.6.3 Generic File Truncation

This section covers the path where a file is being truncated. The actual system call `truncate()` is implemented by `sys_truncate()` in `fs/open.c`. By the time the top-level function in the VM is called (`vmtruncate()`), the dentry information for the file has been updated, and the inode's semaphore has been acquired.

D.6.3.1 Function: `vmtruncate()` (*mm/memory.c*)

This is the top-level VM function responsible for truncating a file. When it completes, all pagetable entries mapping pages that have been truncated have been unmapped and reclaimed if possible.

```

1042 int vmtruncate(struct inode * inode, loff_t offset)
1043 {
1044     unsigned long pgoff;
1045     struct address_space *mapping = inode->i_mapping;
1046     unsigned long limit;
1047
1048     if (inode->i_size < offset)
1049         goto do_expand;
1050     inode->i_size = offset;
1051     spin_lock(&mapping->i_shared_lock);
1052     if (!mapping->i_mmap && !mapping->i_mmap_shared)
1053         goto out_unlock;
1054
1055     pgoff = (offset + PAGE_CACHE_SIZE - 1) >> PAGE_CACHE_SHIFT;
1056     if (mapping->i_mmap != NULL)
1057         vmtruncate_list(mapping->i_mmap, pgoff);
1058     if (mapping->i_mmap_shared != NULL)
1059         vmtruncate_list(mapping->i_mmap_shared, pgoff);
1060
1061 out_unlock:
1062     spin_unlock(&mapping->i_shared_lock);
1063     truncate_inode_pages(mapping, offset);
1064     goto out_truncate;
1065
1066 do_expand:
1067     limit = current->rlim[RLIMIT_FSIZE].rlim_cur;
1068     if (limit != RLIM_INFINITY && offset > limit)
1069         goto out_sig;

```

```

1070     if (offset > inode->i_sb->s_maxbytes)
1071         goto out;
1072     inode->i_size = offset;
1073
1074 out_truncate:
1075     if (inode->i_op && inode->i_op->truncate) {
1076         lock_kernel();
1077         inode->i_op->truncate(inode);
1078         unlock_kernel();
1079     }
1080     return 0;
1081 out_sig:
1082     send_sig(SIGXFSZ, current, 0);
1083 out:
1084     return -EFBIG;
1085 }

```

1042 The parameters passed are the `inode` being truncated and the new `offset` marking the new end of the file. The old length of the file is stored in `inode->i_size`.

1045 Gets the `address_space` responsible for the `inode`.

1048-1049 If the new file size is larger than the old size, then `goto do_expand`, where the limits for the process will be checked before the file is grown.

1050 Here, the file is being shrunk, so it updates `inode->i_size` to match.

1051 Locks the spinlock, protecting the two lists of VMAs using this `inode`.

1052-1053 If no VMAs are mapping the `inode`, `goto out_unlock`, where the pages used by the file will be reclaimed by `truncate_inode_pages()` (See Section D.6.3.6).

1055 Calculates `pgoff` as the offset within the file in pages where the truncation will begin.

1056-1057 Truncates pages from all private mappings with `vmtruncate_list()` (See Section D.6.3.2).

1058-1059 Truncates pages from all shared mappings.

1062 Unlocks the spinlock protecting the VMA lists.

1063 Calls `truncate_inode_pages()` (See Section D.6.3.6) to reclaim the pages if they exist in the page cache for the file.

1064 Goto `out_truncate` to call the filesystem-specific `truncate()` function so the blocks used on disk will be freed.

1066-1071 If the file is being expanded, this makes sure that the process limits for maximum file size are not being exceeded and that the hosting filesystem is able to support the new filesize.

1072 If the limits are fine, this updates the inodes size and falls through to call the filesystem-specific truncate function, which will fill the expanded filesize with zeros.

1075-1079 If the filesystem provides a `truncate()` function, then this locks the kernel, calls it and unlocks the kernel again. Filesystems do not acquire the proper locks to prevent races between file truncation and file expansion due to writing or faulting so the big kernel lock is needed.

1080 Returns success.

1082-1084 If the file size grows too big, this sends the `SIGXFSZ` signal to the calling process and returns `-EFBIG`.

D.6.3.2 Function: `vmtruncate_list()` (*mm/memory.c*)

This function cycles through all VMAs in an `address_spaces` list and calls `zap_page_range()` for the range of addresses that map a file that is being truncated.

```

1006 static void vmtruncate_list(struct vm_area_struct *mpnt,
                                unsigned long pgoff)
1007 {
1008     do {
1009         struct mm_struct *mm = mpnt->vm_mm;
1010         unsigned long start = mpnt->vm_start;
1011         unsigned long end = mpnt->vm_end;
1012         unsigned long len = end - start;
1013         unsigned long diff;
1014
1015         /* mapping wholly truncated? */
1016         if (mpnt->vm_pgoff >= pgoff) {
1017             zap_page_range(mm, start, len);
1018             continue;
1019         }
1020
1021         /* mapping wholly unaffected? */
1022         len = len >> PAGE_SHIFT;
1023         diff = pgoff - mpnt->vm_pgoff;
1024         if (diff >= len)
1025             continue;
1026
1027         /* Ok, partially affected.. */
1028         start += diff << PAGE_SHIFT;
1029         len = (len - diff) << PAGE_SHIFT;

```

```

1030         zap_page_range(mm, start, len);
1031     } while ((mpnt = mpnt->vm_next_share) != NULL);
1032 }

```

1008-1031 Loops through all VMAs in the list.

1009 Gets the `mm_struct` that hosts this VMA.

1010-1012 Calculates the start, end and length of the VMA.

1016-1019 If the whole VMA is being truncated, this calls the function `zap_page_range()` (See Section D.6.3.3) with the start and length of the full VMA.

1022 Calculates the length of the VMA in pages.

1023-1025 Checks if the VMA maps any of the region being truncated. If the VMA is unaffected, it continues to the next VMA.

1028-1029 If the VMA is being partially truncated this calculates where the start and length of the region to truncate is in pages.

1030 Calls `zap_page_range()` (See Section D.6.3.3) to unmap the affected region.

D.6.3.3 Function: `zap_page_range()` (*mm/memory.c*)

This function is the top-level pagetable-walk function, which unmaps userpages in the specified range from an `mm_struct`.

```

360 void zap_page_range(struct mm_struct *mm,
                      unsigned long address, unsigned long size)
361 {
362     mmu_gather_t *tlb;
363     pgd_t *dir;
364     unsigned long start = address, end = address + size;
365     int freed = 0;
366
367     dir = pgd_offset(mm, address);
368
369     /*
370      * This is a long-lived spinlock. That's fine.
371      * There's no contention, because the page table
372      * lock only protects against kswapd anyway, and
373      * even if kswapd happened to be looking at this
374      * process we _want_ it to get stuck.
375      */
376     if (address >= end)
377         BUG();
378     spin_lock(&mm->page_table_lock);

```

```

379     flush_cache_range(mm, address, end);
380     tlb = tlb_gather_mmu(mm);
381
382     do {
383         freed += zap_pmd_range(tlb, dir, address, end - address);
384         address = (address + PGDIR_SIZE) & PGDIR_MASK;
385         dir++;
386     } while (address && (address < end));
387
388     /* this will flush any remaining tlb entries */
389     tlb_finish_mmu(tlb, start, end);
390
391     /*
392     * Update rss for the mm_struct (not necessarily current->mm)
393     * Notice that rss is an unsigned long.
394     */
395     if (mm->rss > freed)
396         mm->rss -= freed;
397     else
398         mm->rss = 0;
399     spin_unlock(&mm->page_table_lock);
400 }

```

364 Calculates the `start` and `end` address for zapping.

367 Calculates the PGD (`dir`) that contains the starting `address`.

376-377 Makes sure the start address is not after the end address.

378 Acquires the spinlock protecting the page tables. This is a very longheld lock and would normally be considered a bad idea, but the comment prior to the block explains why it is ok in this case.

379 Flushes the CPU cache for this range.

380 `tlb_gather_mmu()` records the MM that is being altered. Later, `tlb_remove_page()` will be called to unmap the PTE that stores the PTEs in a `struct free_pte_ctx` until the zapping is finished. This is to avoid having to constantly flush the TLB as PTEs are freed.

382-386 For each PMD affected by the zapping, this calls `zap_pmd_range()` until the end address has been reached. Note that `tlb` is passed as well for `tlb_remove_page()` to use later.

389 `tlb_finish_mmu()` frees all the PTEs that were unmapped by `tlb_remove_page()` and then flushes the TLBs. Doing the flushing this way avoids a storm of TLB flushing that would be otherwise required for each PTE unmapped.

395-398 Updates RSS count.

399 Releases the pagetable lock.

D.6.3.4 Function: `zap_pmd_range()` (*mm/memory.c*)

This function is unremarkable. It steps through the PMDs that are affected by the requested range and calls `zap_pte_range()` for each one.

```

331 static inline int zap_pmd_range(mmu_gather_t *tlb, pgd_t * dir,
                                unsigned long address,
                                unsigned long size)
332 {
333     pmd_t * pmd;
334     unsigned long end;
335     int freed;
336
337     if (pgd_none(*dir))
338         return 0;
339     if (pgd_bad(*dir)) {
340         pgd_ERROR(*dir);
341         pgd_clear(dir);
342         return 0;
343     }
344     pmd = pmd_offset(dir, address);
345     end = address + size;
346     if (end > ((address + PGDIR_SIZE) & PGDIR_MASK))
347         end = ((address + PGDIR_SIZE) & PGDIR_MASK);
348     freed = 0;
349     do {
350         freed += zap_pte_range(tlb, pmd, address, end - address);
351         address = (address + PMD_SIZE) & PMD_MASK;
352         pmd++;
353     } while (address < end);
354     return freed;
355 }
```

337-338 If no PGD exists, this returns.

339-343 If the PGD is bad, it flags the error and returns.

344 Gets the starting pmd.

345-347 Calculates the `end` address of the zapping. If it is beyond the end of this PGD, then set `end` to the end of the PGD.

349-353 Steps through all PMDs in this PGD. For each PMD, it calls `zap_pte_range()` (See Section D.6.3.5) to unmap the PTEs.

354 Returns how many pages were freed.

D.6.3.5 Function: zap_pte_range() (*mm/memory.c*)

This function calls `tlb_remove_page()` for each PTE in the requested pmd within the requested address range.

```

294 static inline int zap_pte_range(mmu_gather_t *tlb, pmd_t *pmd,
                                unsigned long address,
                                unsigned long size)
295 {
296     unsigned long offset;
297     pte_t *ptep;
298     int freed = 0;
299
300     if (pmd_none(*pmd))
301         return 0;
302     if (pmd_bad(*pmd)) {
303         pmd_ERROR(*pmd);
304         pmd_clear(pmd);
305         return 0;
306     }
307     ptep = pte_offset(pmd, address);
308     offset = address & ~PMD_MASK;
309     if (offset + size > PMD_SIZE)
310         size = PMD_SIZE - offset;
311     size &= PAGE_MASK;
312     for (offset=0; offset < size; ptep++, offset += PAGE_SIZE) {
313         pte_t pte = *ptep;
314         if (pte_none(pte))
315             continue;
316         if (pte_present(pte)) {
317             struct page *page = pte_page(pte);
318             if (VALID_PAGE(page) && !PageReserved(page))
319                 freed ++;
320             /* This will eventually call __free_pte on the pte. */
321             tlb_remove_page(tlb, ptep, address + offset);
322         } else {
323             free_swap_and_cache(pte_to_swp_entry(pte));
324             pte_clear(ptep);
325         }
326     }
327
328     return freed;
329 }

```

300-301 If the PMD does not exist, this returns.

302-306 If the PMD is bad, it flags the error and returns.

307 Gets the starting PTE offset.

308 Aligns the offset to a PMD boundary.

309 If the size of the region to unmap is past the PMD boundary, this fixes the size so that only this PMD will be affected.

311 Aligns `size` to a page boundary.

312-326 Steps through all PTEs in the region.

314-315 If no PTE exists, this continues to the next one.

316-322 If the PTE is present, this calls `tlb_remove_page()` to unmap the page. If the page is reclaimable, it increments the `freed` count.

322-325 If the PTE is in use, but the page is paged out or in the swap cache, this frees the swap slot and page with `free_swap_and_cache()` (See Section K.3.2.3). It is possible that a page is reclaimed if it was in the swap cache that is unaccounted for here, but it is not of paramount importance.

328 Returns the number of pages that were freed.

D.6.3.6 Function: `truncate_inode_pages()` (*mm/filemap.c*)

This is the top-level function responsible for truncating all pages from the page cache that occurred after `lstart` in a mapping.

```

327 void truncate_inode_pages(struct address_space * mapping,
                             loff_t lstart)
328 {
329     unsigned long start = (lstart + PAGE_CACHE_SIZE - 1) >>
                             PAGE_CACHE_SHIFT;
330     unsigned partial = lstart & (PAGE_CACHE_SIZE - 1);
331     int unlocked;
332
333     spin_lock(&pagecache_lock);
334     do {
335         unlocked = truncate_list_pages(&mapping->clean_pages,
                                         start, &partial);
336         unlocked |= truncate_list_pages(&mapping->dirty_pages,
                                         start, &partial);
337         unlocked |= truncate_list_pages(&mapping->locked_pages,
                                         start, &partial);
338     } while (unlocked);
339     /* Traversed all three lists without dropping the lock */
340     spin_unlock(&pagecache_lock);
341 }

```

329 Calculates where to `start` the truncation as an index in pages.

- 330 Calculates `partial` as an offset within the last page if it is being partially truncated.
- 333 Locks the page cache.
- 334 This will loop until none of the calls to `truncate_list_pages()` returns that a page was found that should have been reclaimed.
- 335 Uses `truncate_list_pages()` (See Section D.6.3.7) to truncate all pages in the `clean_pages` list.
- 336 Similarly, truncates pages in the `dirty_pages` list.
- 337 Similarly, truncates pages in the `locked_pages` list.
- 340 Unlocks the page cache.

D.6.3.7 Function: `truncate_list_pages()` (*mm/filemap.c*)

This function searches the requested list (`head`), which is part of an `address_space`. If pages are found after `start`, they will be truncated.

```

259 static int truncate_list_pages(struct list_head *head,
                                unsigned long start,
                                unsigned *partial)
260 {
261     struct list_head *curr;
262     struct page *page;
263     int unlocked = 0;
264
265     restart:
266     curr = head->prev;
267     while (curr != head) {
268         unsigned long offset;
269
270         page = list_entry(curr, struct page, list);
271         offset = page->index;
272
273         /* Is one of the pages to truncate? */
274         if ((offset >= start) ||
            (*partial && (offset + 1) == start)) {
275             int failed;
276
277             page_cache_get(page);
278             failed = TryLockPage(page);
279
280             list_del(head);
281             if (!failed)
282                 /* Restart after this page */

```

```

283         list_add_tail(head, curr);
284     else
285         /* Restart on this page */
286         list_add(head, curr);
287
288     spin_unlock(&pagecache_lock);
289     unlocked = 1;
290
291     if (!failed) {
292         if (*partial && (offset + 1) == start) {
293             truncate_partial_page(page, *partial);
294             *partial = 0;
295         } else
296             truncate_complete_page(page);
297
298         UnlockPage(page);
299     } else
300         wait_on_page(page);
301
302     page_cache_release(page);
303
304     if (current->need_resched) {
305         __set_current_state(TASK_RUNNING);
306         schedule();
307     }
308
309     spin_lock(&pagecache_lock);
310     goto restart;
311 }
312 curr = curr->prev;
313 }
314 return unlocked;
315 }

```

266-267 Records the start of the list and loops until the full list has been scanned.

270-271 Gets the page for this entry and what **offset** within the file it represents.

274 If the current page is after **start** or is a page that is to be partially truncated, this truncates this page or moves to the next one.

277-278 Takes a reference to the page and tries to lock it.

280 Removes the page from the list.

281-283 If we locked the page, this adds it back to the list where it will be skipped over on the next iteration of the loop.

- 284-286** If not, it adds it back where it will be found again immediately. Later in the function, `wait_on_page()` is called until the page is unlocked.
- 288** Releases the pagecache lock.
- 299** Sets `locked` to 1 to indicate a page was found that had to be truncated. This will force `truncate_inode_pages()` to call this function again to make sure there are no pages left behind. This looks like an oversight and was intended to have the functions recalled only if a *locked* page was found. However the way it is implemented means that it will be called whether the page was locked or not.
- 291-299** If we locked the page, this truncates it.
- 292-294** If the page is to be partially truncated, this calls `truncate_partial_page()` (See Section D.6.3.10) with the offset within the page where the truncation begins (`partial`).
- 296** If not, it calls `truncate_complete_page()` (See Section D.6.3.8) to truncate the whole page.
- 298** Unlocks the page.
- 300** If the page locking failed, this calls `wait_on_page()` to wait until the page can be locked.
- 302** Releases the reference to the page. If there are no more mappings for the page, it will be reclaimed.
- 304-307** Checks if the process should call `schedule()` before continuing. This is to prevent a truncating process from hogging the CPU.
- 309** Reacquires the spinlock and restarts the scanning for pages to reclaim.
- 312** The current page should not be reclaimed, so this moves to the next page.
- 314** Returns 1 if a page was found in the list that had to be truncated.

D.6.3.8 Function: `truncate_complete_page()` (*mm/filemap.c*)

This function truncates a full page, frees associated resources and reclaims the page.

```

239 static void truncate_complete_page(struct page *page)
240 {
241     /* Leave it on the LRU if it gets converted into
       * anonymous buffers */
242     if (!page->buffers || do_flushpage(page, 0))
243         lru_cache_del(page);
244
245     /*
246     * We remove the page from the page cache _after_ we have

```

```

247     * destroyed all buffer-cache references to it. Otherwise
248     * some other process might think this inode page is not in
249     * the page cache and creates a buffer-cache alias to it
250     * causing all sorts of fun problems ...
251     */
252     ClearPageDirty(page);
253     ClearPageUptodate(page);
254     remove_inode_page(page);
255     page_cache_release(page);
256 }

```

242 If the page has buffers, this calls `do_flushpage()` (See Section D.6.3.9) to flush all buffers associated with the page. The comments in the following lines describe the problem concisely.

243 Deletes the page from the LRU.

252-253 Clears the dirty and uptodate flags for the page.

254 Calls `remove_inode_page()` (See Section J.1.2.1) to delete the page from the page cache.

255 Drops the reference to the page. The page will be later reclaimed when `truncate_list_pages()` drops its own private reference to it.

D.6.3.9 Function: `do_flushpage()` (*mm/filemap.c*)

This function is responsible for flushing all buffers associated with a page.

```

223 static int do_flushpage(struct page *page, unsigned long offset)
224 {
225     int (*flushpage) (struct page *, unsigned long);
226     flushpage = page->mapping->a_ops->flushpage;
227     if (flushpage)
228         return (*flushpage)(page, offset);
229     return block_flushpage(page, offset);
230 }

```

226-228 If the `page→mapping` provides a `flushpage()` function, this calls it.

229 If not, this calls `block_flushpage()`, which is the generic function for flushing buffers associated with a page.

D.6.3.10 Function: `truncate_partial_page()` (*mm/filemap.c*)

This function partially truncates a page by zeroing out the higher bytes no longer in use and flushing any associated buffers.

```

232 static inline void truncate_partial_page(struct page *page,
                                         unsigned partial)

```

```

233 {
234     memclear_highpage_flush(page, partial, PAGE_CACHE_SIZE-partial);
235     if (page->buffers)
236         do_flushpage(page, partial);
237 }

```

234 `memclear_highpage_flush()` fills an address range with zeros. In this case, it will zero from `partial` to the end of the page.

235-236 If the page has any associated buffers, this flushes any buffers containing data in the truncated region.

D.6.4 Reading Pages for the Page Cache

D.6.4.1 Function: `filemap_nopage()` (*mm/filemap.c*)

This is the generic `nopage()` function used by many VMAs. This loops around itself with a large number of goto's, which can be difficult to trace, but there is nothing novel here. It is principally responsible for fetching the faulting page from either the pagecache or reading it from disk. If appropriate, it will also perform file readahead.

```

1994 struct page * filemap_nopage(struct vm_area_struct * area,
                                unsigned long address,
                                int unused)
1995 {
1996     int error;
1997     struct file *file = area->vm_file;
1998     struct address_space *mapping =
1999         file->f_dentry->d_inode->i_mapping;
2000     struct inode *inode = mapping->host;
2001     struct page *page, **hash;
2002     unsigned long size, pgoff, endoff;
2003     pgoff = ((address - area->vm_start) >> PAGE_CACHE_SHIFT) +
2004         area->vm_pgoff;
2005     endoff = ((area->vm_end - area->vm_start) >> PAGE_CACHE_SHIFT) +
2006         area->vm_pgoff;

```

This block acquires the `struct file`, `address_space` and `inode`, which are important for this page fault. It then acquires the starting offset within the file needed for this fault and the offset that corresponds to the end of this VMA. The offset is the end of the VMA instead of the end of the page in case file readahead is performed.

1997-1999 Acquires the `struct file`, `address_space` and `inode` required for this fault.